Discrete Methods for Modern AI: lecture 1

Kyle Richardson

Nov 2 2026

Course overview

Discrete Methods for Modern AI

Generative AI and Language Models



Prompt

Response

ChatGPT

User

Briefly tell me (in a few sentences) what a course on discrete methods for modern AI (i.e., generative AI and LLMs) would be about.

225t

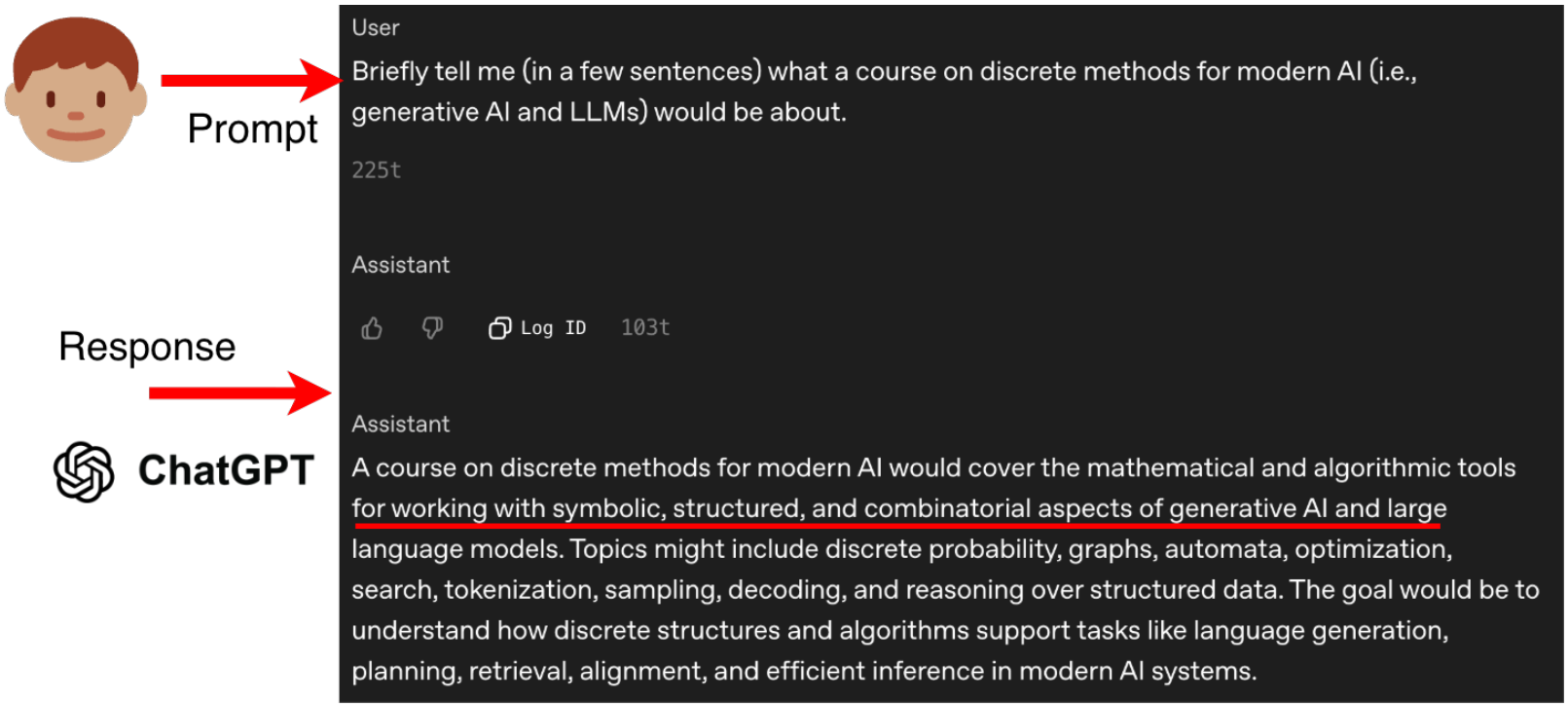
Assistant

Log ID 103t

Assistant

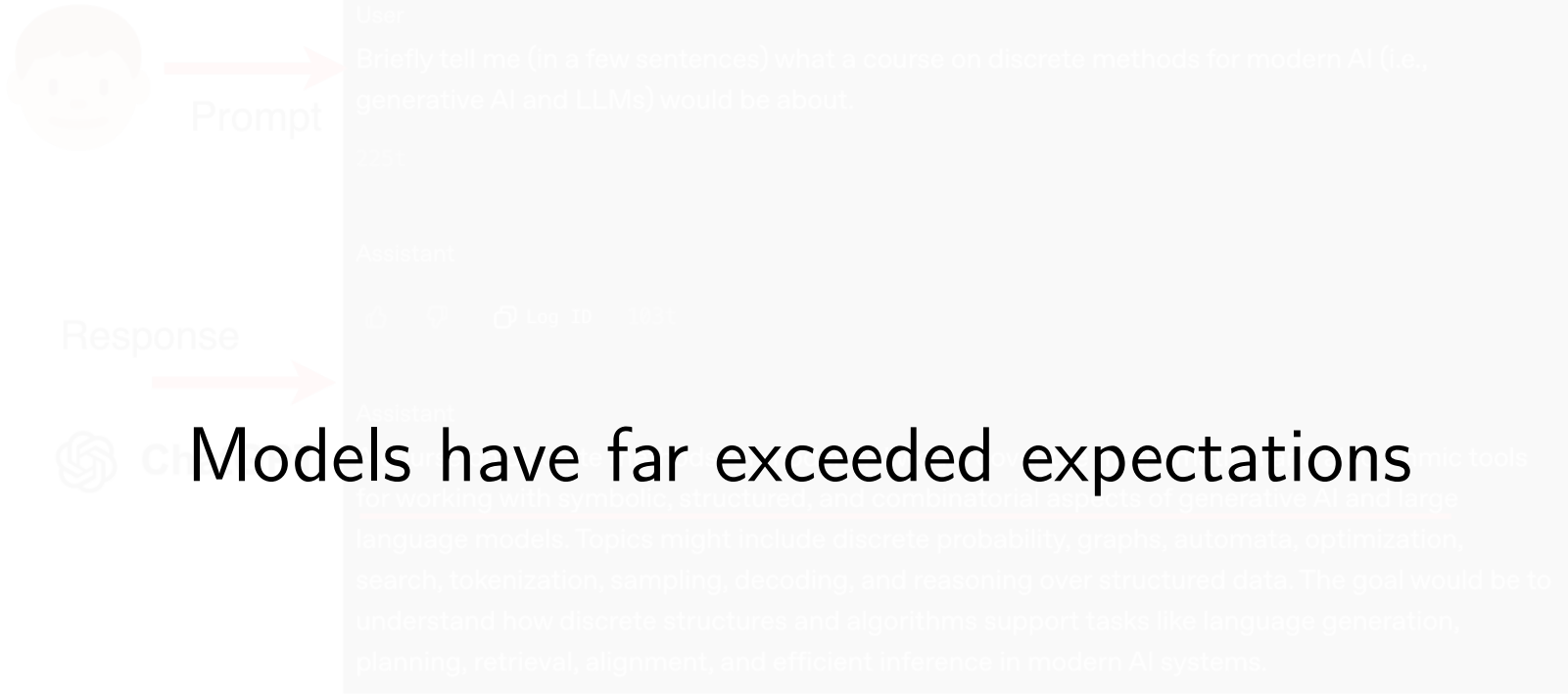
A course on discrete methods for modern AI would cover the mathematical and algorithmic tools for working with symbolic, structured, and combinatorial aspects of generative AI and large language models. Topics might include discrete probability, graphs, automata, optimization, search, tokenization, sampling, decoding, and reasoning over structured data. The goal would be to understand how discrete structures and algorithms support tasks like language generation, planning, retrieval, alignment, and efficient inference in modern AI systems.

Generative AI and Language Models



- **General purpose models**: trained at massive scales, used *as-is* and directly for a wide range of problems.

Generative AI and Language Models



Models have far exceeded expectations

- **General purpose models**: trained at massive scales, used *as-is* and directly for a wide range of problems.

Language models as agent simulators

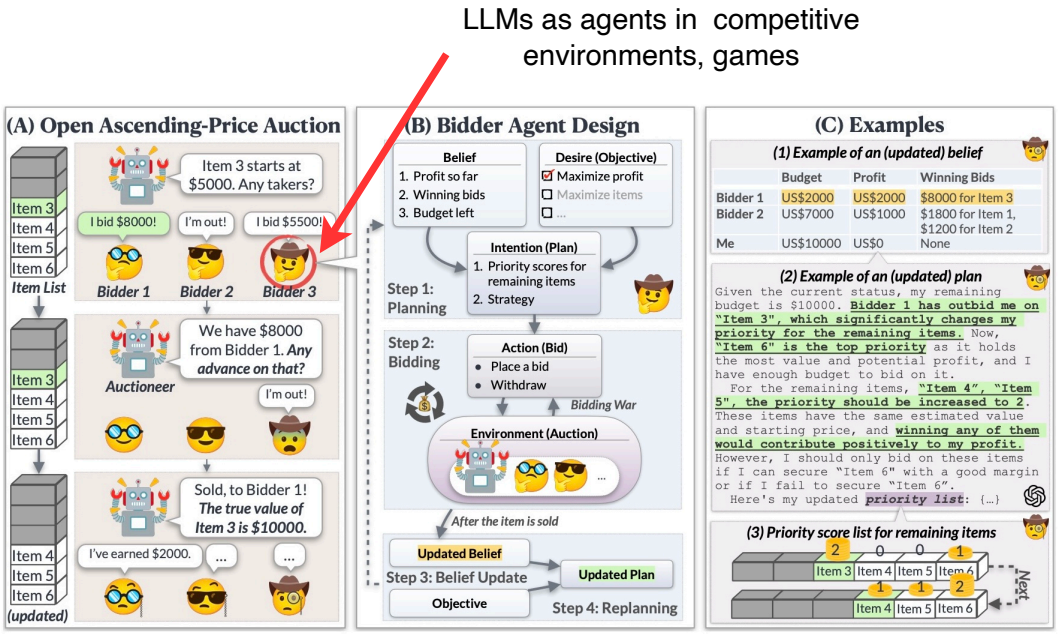


Figure 1: An illustration of AUCARENA: (A) shows a multi-round, ascending-price auction with an auctioneer announcing the highest bids, where bidders publicly decide after private reasoning. (B) presents a bidder agent structure using the Belief-Desire-Intention Model, involving planning, bidding, belief updating, and replanning, where beliefs and plans are adjusted with each auction development. (C) offers instances of updated beliefs and plans, illustrating a bidder allocating priority scores to items post-reasoning.

- Can we use LMs to simulate complex social dynamics? (Chen et al.2023; Zhang et al.2024; Yang et al.2025)

Language models as agent simulators

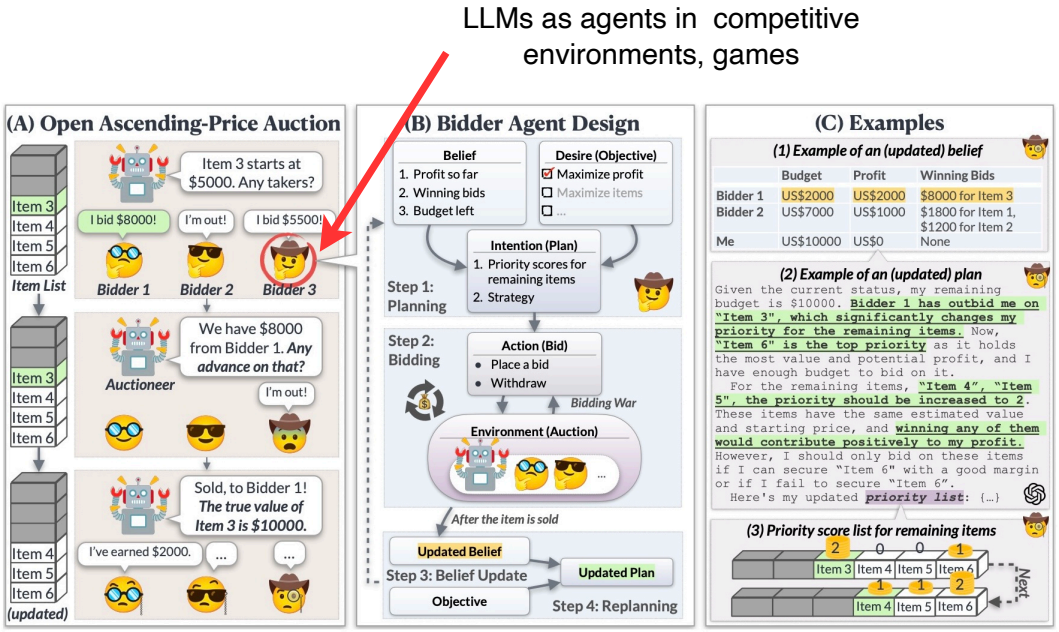
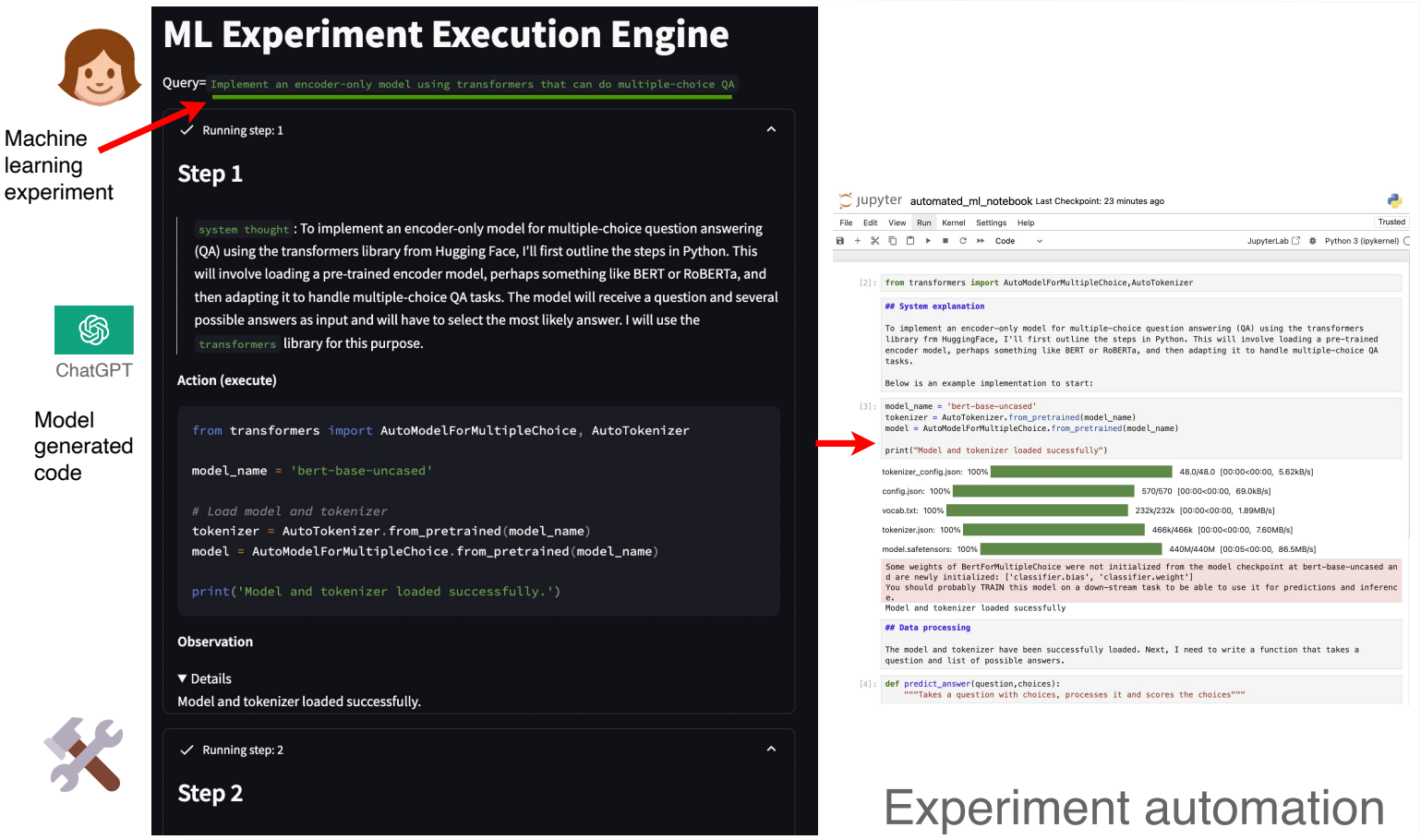


Figure 1: An illustration of AUCARENA: (A) shows a multi-round, ascending-price auction with an auctioneer announcing the highest bids, where bidders publicly decide after private reasoning. (B) presents a bidder agent structure using the Belief-Desire-Intention Model, involving planning, bidding, belief updating, and replanning, where beliefs and plans are adjusted with each auction development. (C) offers instances of updated beliefs and plans, illustrating a bidder allocating priority scores to items post-reasoning.

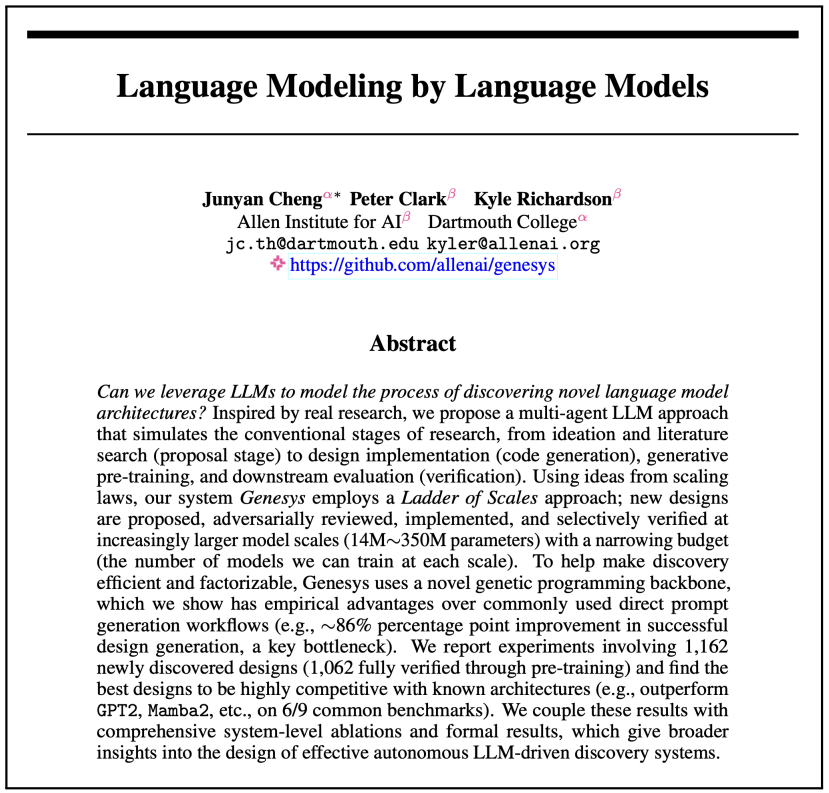
Valuable tool for running social science experiments, testing theories of language interaction, complex reasoning, adversarial language experts.

Language models as part of complex systems



- Automated research (Bogin et al.2024; Yu et al.2026) and general agentic benchmarking (Bragg et al.2026).

Language models as part of complex systems, **agents**



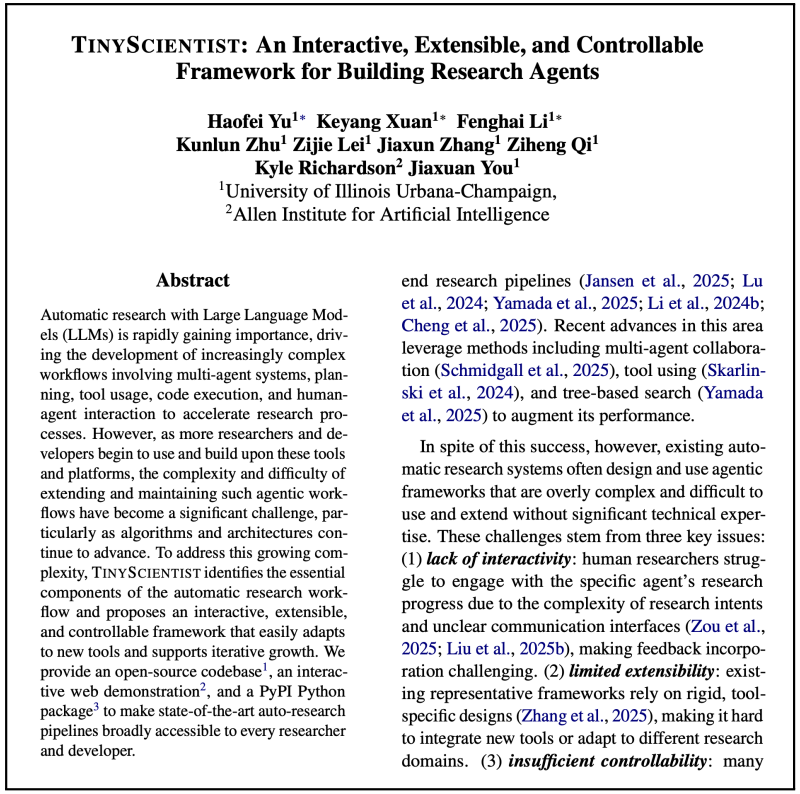
Language Modeling by Language Models

Junyan Cheng[α,*] **Peter Clark**[β] **Kyle Richardson**[β]
Allen Institute for AI[β] Dartmouth College[α]
jc.th@dartmouth.edu kyler@allenai.org
https://github.com/allenai/genesys

Abstract

Can we leverage LLMs to model the process of discovering novel language model architectures? Inspired by real research, we propose a multi-agent LLM approach that simulates the conventional stages of research, from ideation and literature search (proposal stage) to design implementation (code generation), generative pre-training, and downstream evaluation (verification). Using ideas from scaling laws, our system *Genesys* employs a *Ladder of Scales* approach; new designs are proposed, adversarially reviewed, implemented, and selectively verified at increasingly larger model scales (14M∼350M parameters) with a narrowing budget (the number of models we can train at each scale). To help make discovery efficient and factorizable, Genesys uses a novel genetic programming backbone, which we show has empirical advantages over commonly used direct prompt generation workflows (e.g., ∼86% percentage point improvement in successful design generation, a key bottleneck). We report experiments involving 1,162 newly discovered designs (1,062 fully verified through pre-training) and find the best designs to be highly competitive with known architectures (e.g., outperform GPT2, Mamba2, etc., on 6/9 common benchmarks). We couple these results with comprehensive system-level ablations and formal results, which give broader insights into the design of effective autonomous LLM-driven discovery systems.

(Cheng et al.2025)



TinyScientist: An Interactive, Extensible, and Controllable Framework for Building Research Agents

Haofei Yu[1*] **Keyang Xuan**[1*] **Fenghai Li**[1*]
Kunlun Zhu[1] **Zijie Lei**[1] **Jiaxun Zhang**[1] **Ziheng Qi**[1]
Kyle Richardson[2] **Jiaxuan You**[1]
[1]University of Illinois Urbana-Champaign,
[2]Allen Institute for Artificial Intelligence

Abstract

Automatic research with Large Language Models (LLMs) is rapidly gaining importance, driving the development of increasingly complex workflows involving multi-agent systems, planning, tool usage, code execution, and human-agent interaction to accelerate research processes. However, as more researchers and developers begin to use and build upon these tools and platforms, the complexity and difficulty of extending and maintaining such agentic workflows have become a significant challenge, particularly as algorithms and architectures continue to advance. To address this growing complexity, TinyScientist identifies the essential components of the automatic research workflow and proposes an interactive, extensible, and controllable framework that easily adapts to new tools and supports iterative growth. We provide an open-source codebase[1], an interactive web demonstration[2], and a PyPI Python package[3] to make state-of-the-art auto-research pipelines broadly accessible to every researcher and developer.

end research pipelines (Jansen et al., 2025; Lu et al., 2024; Yamada et al., 2025; Li et al., 2024b; Cheng et al., 2025). Recent advances in this area leverage methods including multi-agent collaboration (Schmidgall et al., 2025), tool using (Skarlinski et al., 2024), and tree-based search (Yamada et al., 2025) to augment its performance.

In spite of this success, however, existing automatic research systems often design and use agentic frameworks that are overly complex and difficult to use and extend without significant technical expertise. These challenges stem from three key issues: (1) ***lack of interactivity***: human researchers struggle to engage with the specific agent's research progress due to the complexity of research intents and unclear communication interfaces (Zou et al., 2025; Liu et al., 2025b), making feedback incorporation challenging. (2) ***limited extensibility***: existing representative frameworks rely on rigid, tool-specific designs (Zhang et al., 2025), making it hard to integrate new tools or adapt to different research domains. (3) ***insufficient controllability***: many

(Yu et al.2025)

A tool for scientific discovery (Yu et al.2026), automated experiment execution, helping non-experts engage in research.

A tool for scientific discovery (Yu et al.2026), automated experiment execution, helping non-experts engage in research.

Problems with Modern AI

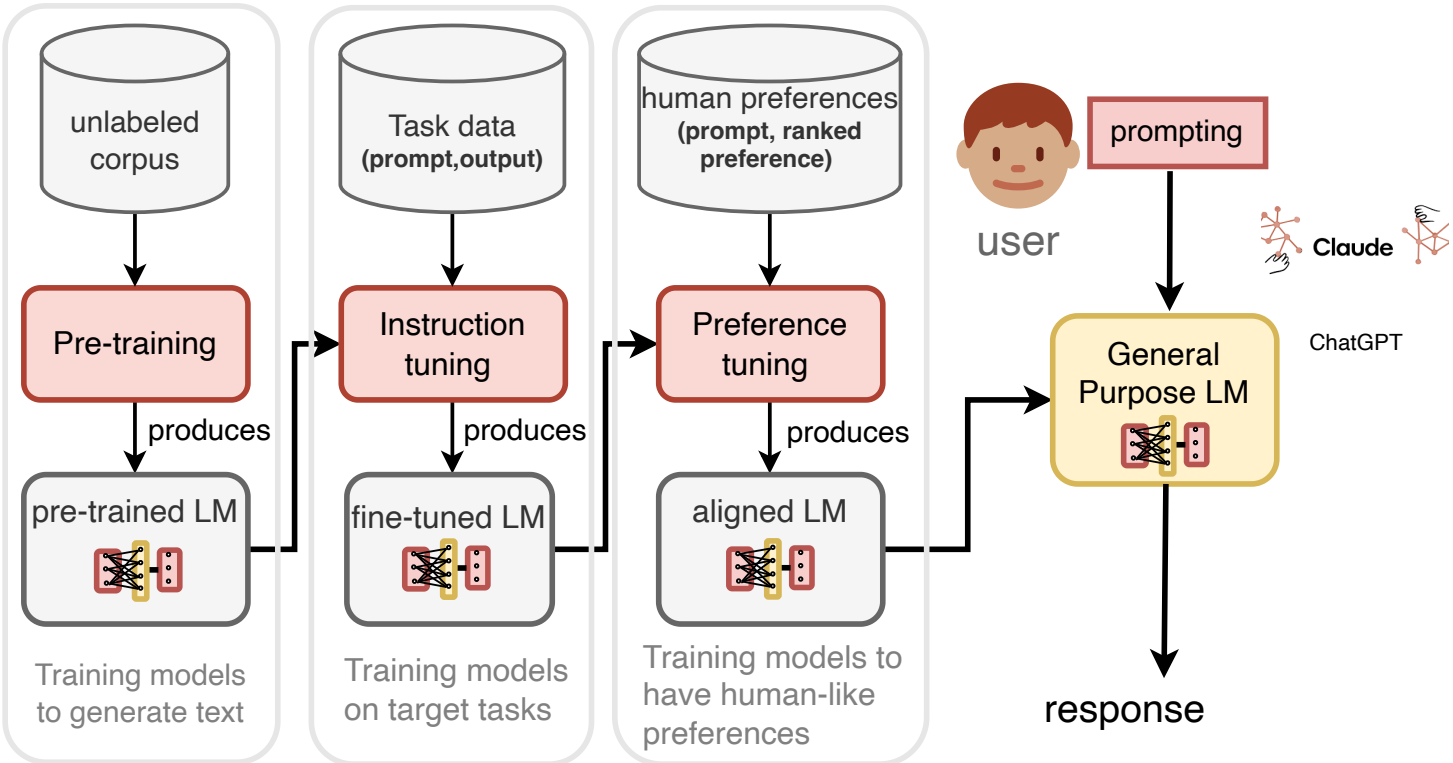
A tool for scientific discovery (Yu et al.2026), automated experiment execution, helping non-experts engage in research.

Missing **semantic** and **algorithmic** foundations.

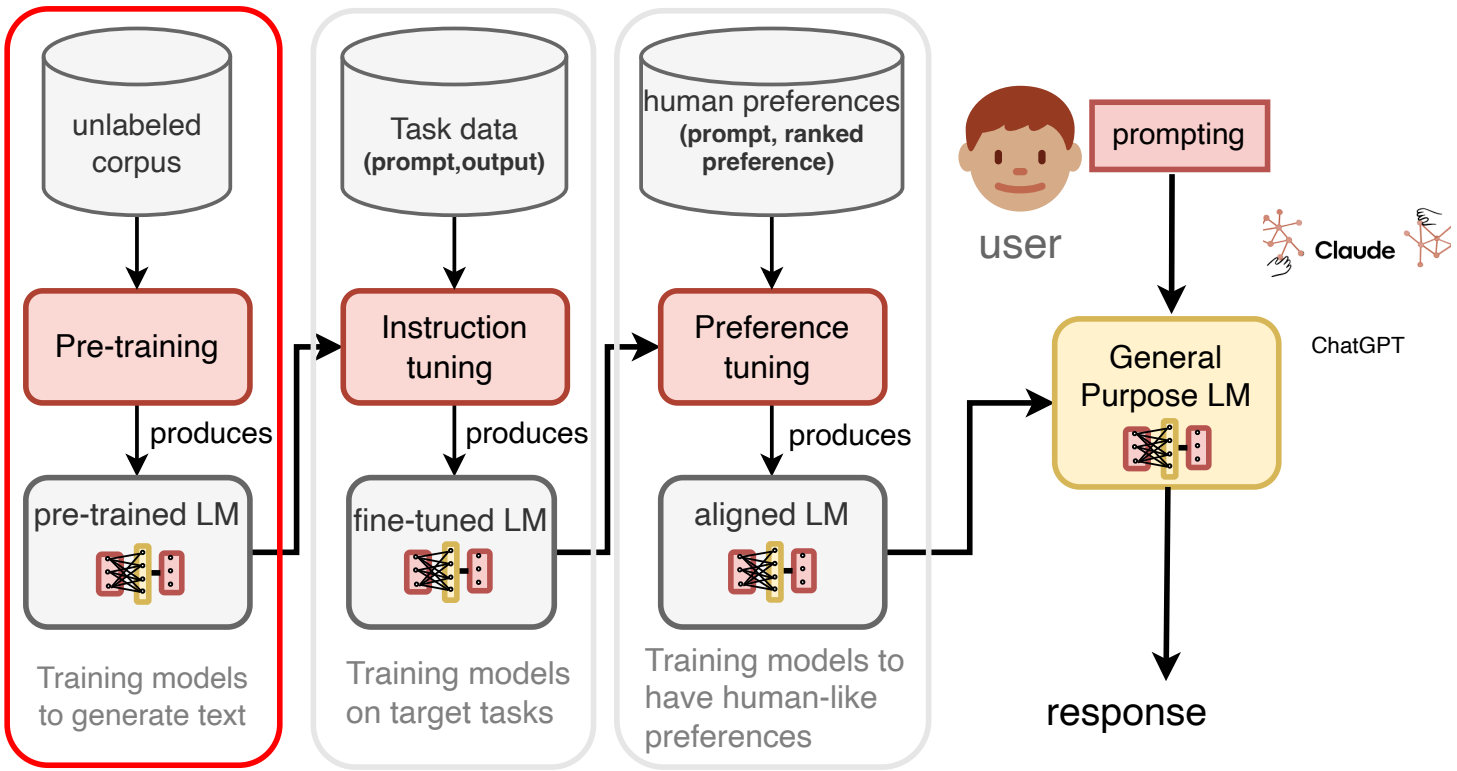
A tool for scientific discovery (Yu et al.2026), automated experiment execution, helping non-experts engage in research.

Missing **semantic** and **algorithmic** foundations.

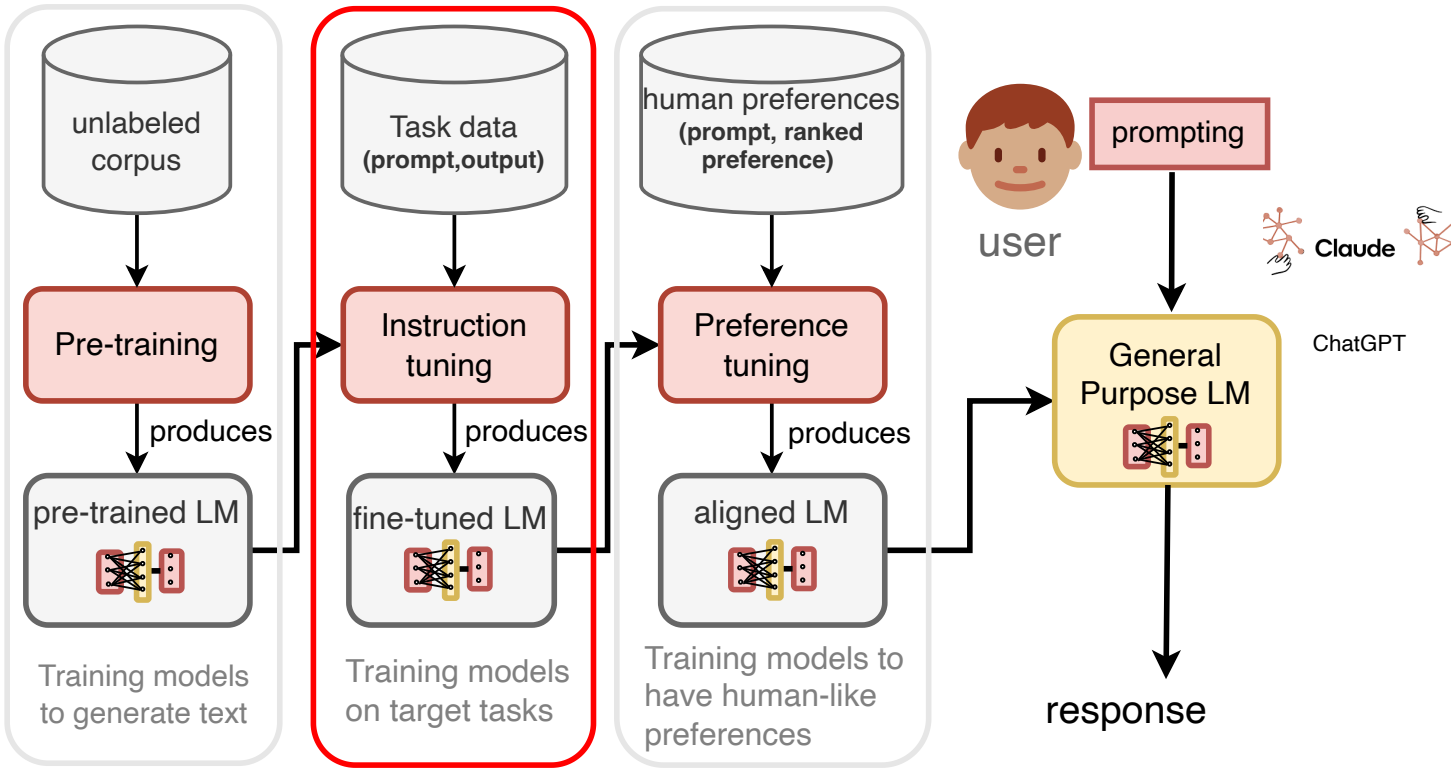
How do we get to general-purpose LLMs? **recipe**



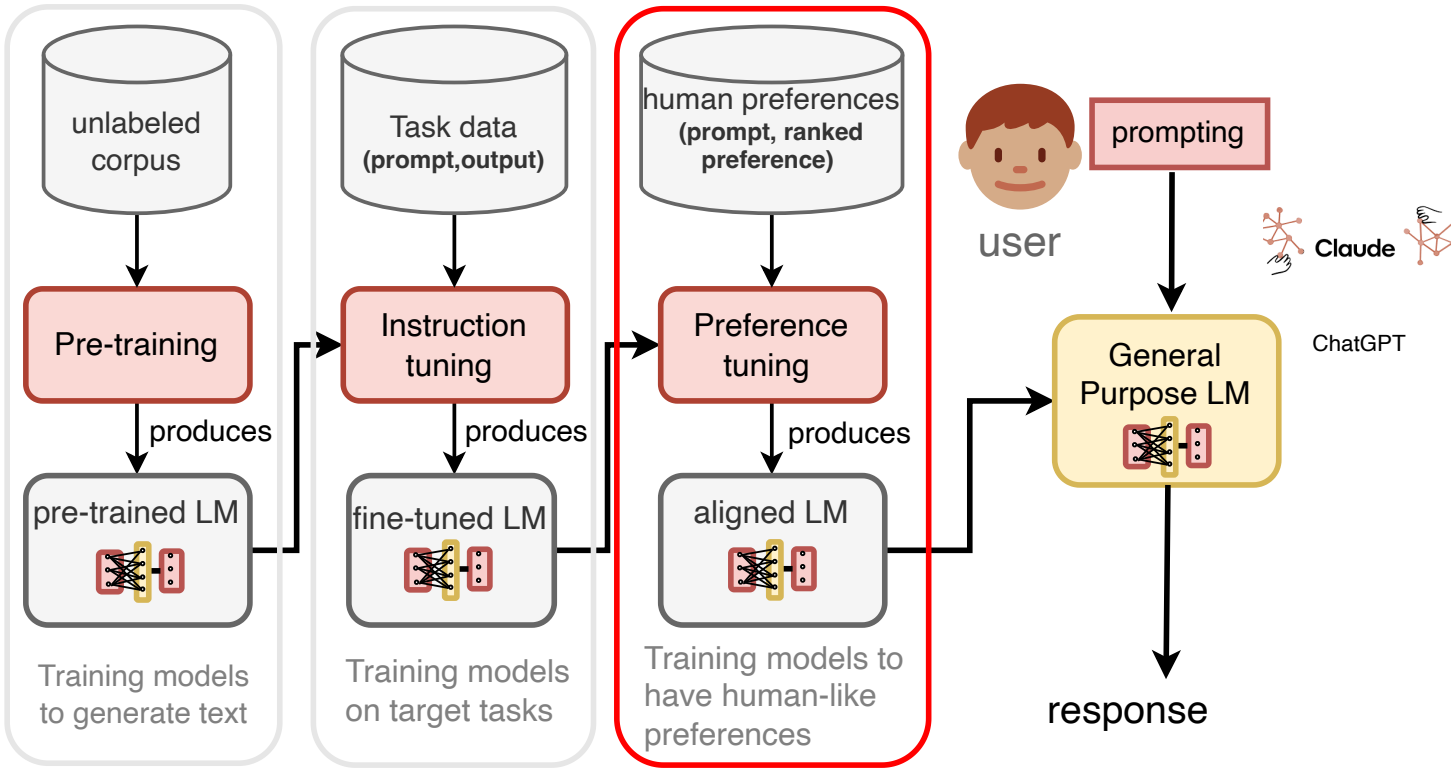
How do we get to general-purpose LLMs? **recipe**



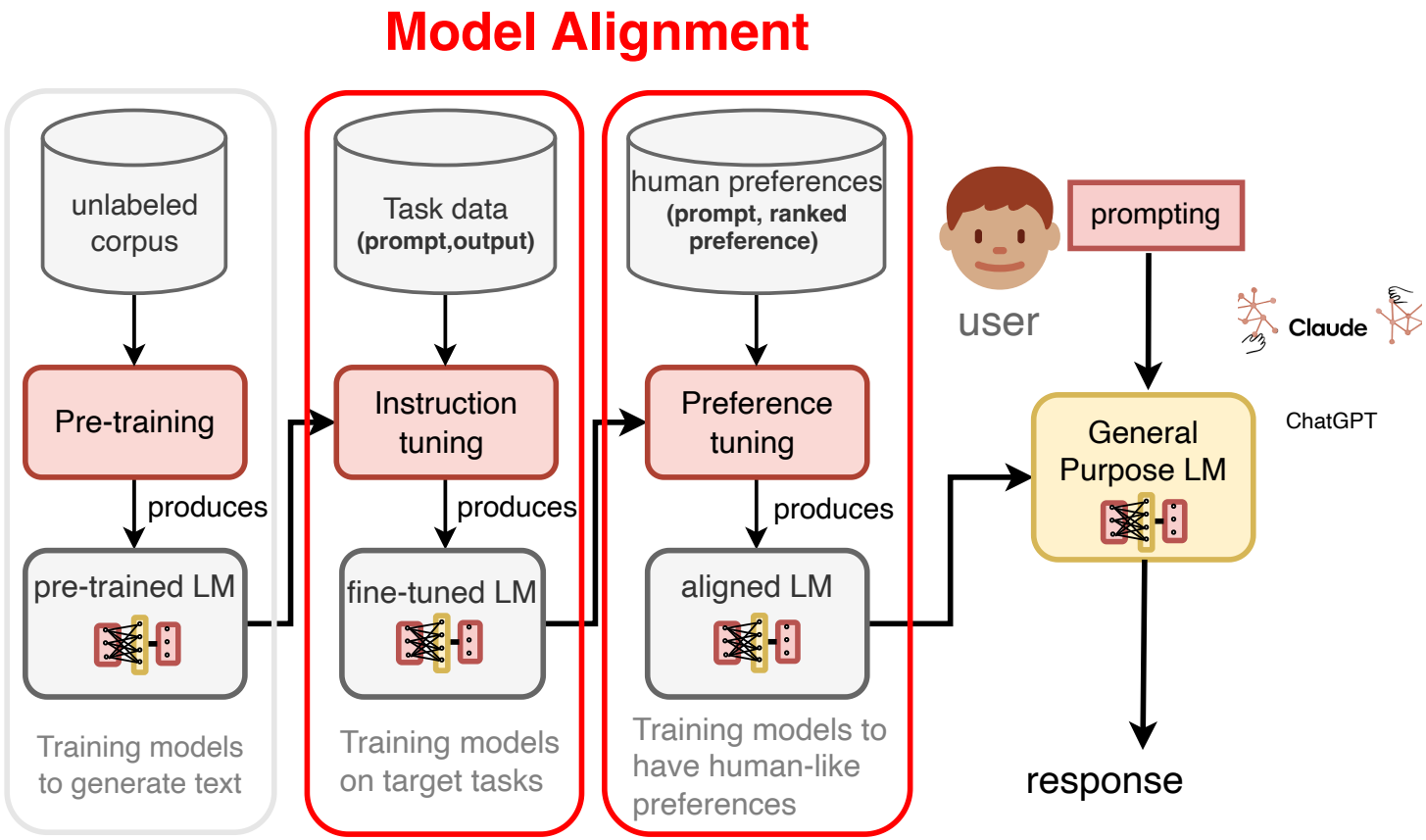
How do we get to general-purpose LLMs? **recipe**



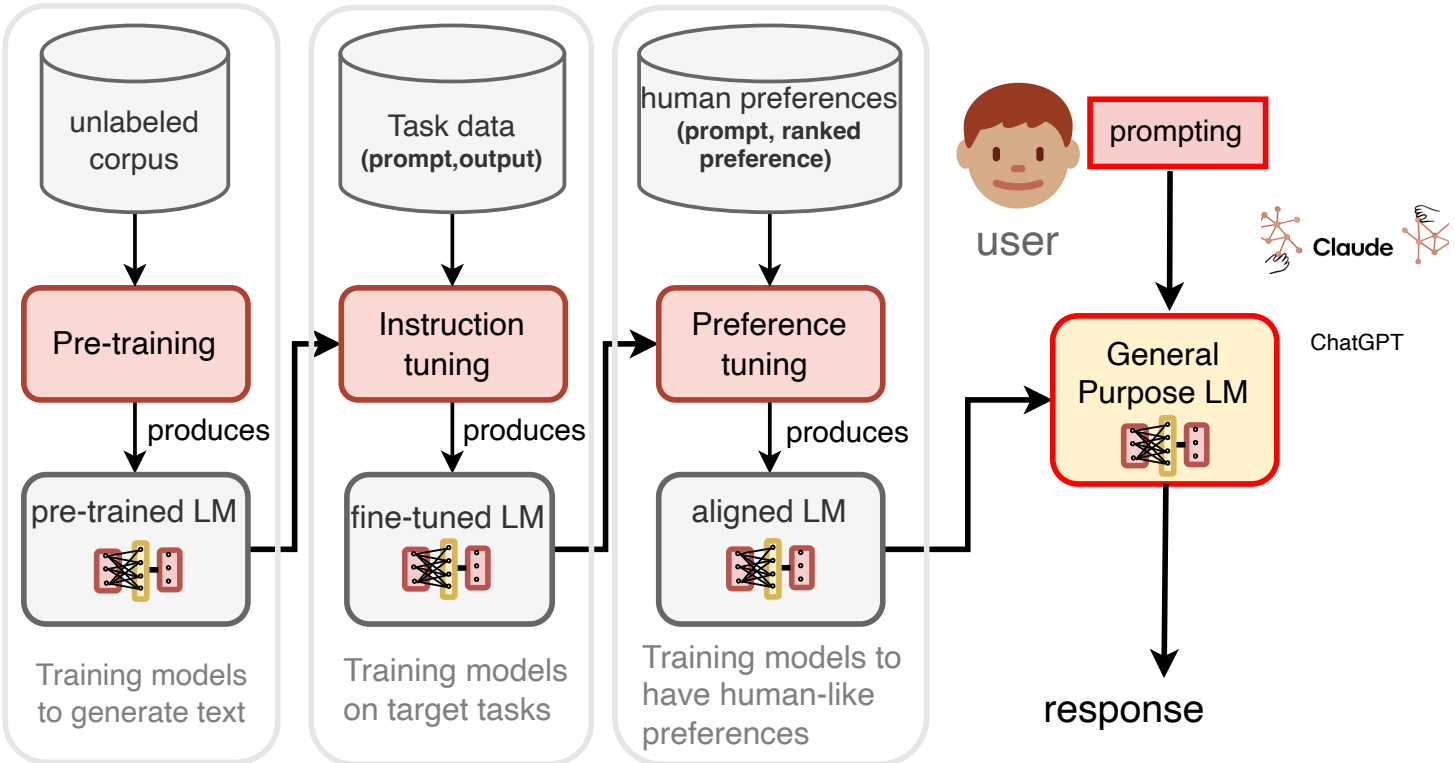
How do we get to general-purpose LLMs? **recipe**



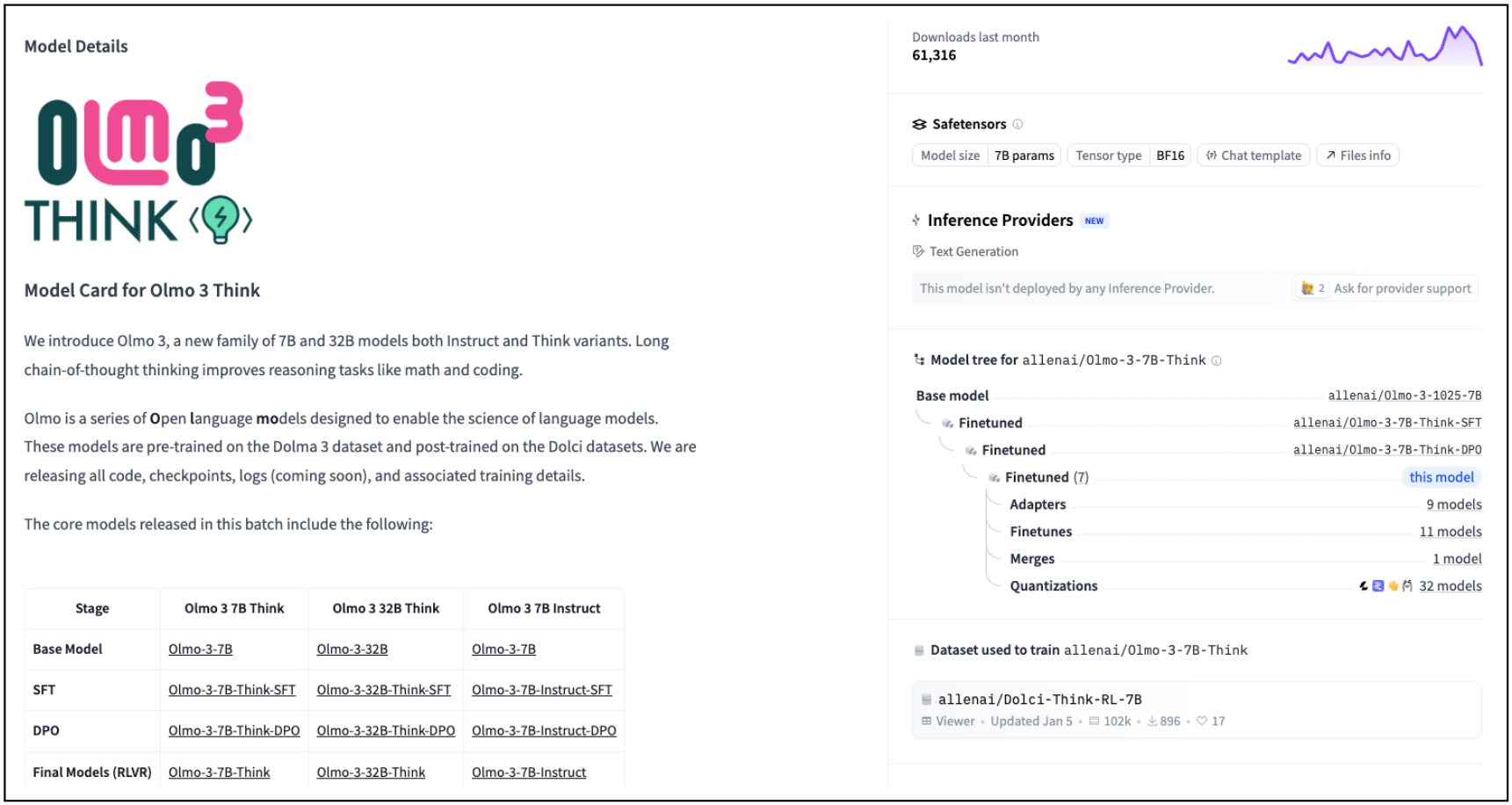
How do we get to general-purpose LLMs? **recipe**



How do we get to general-purpose LLMs? **recipe**



Open model development: **Olmo**



Model Details

Olmo 3 THINK

Model Card for Olmo 3 Think

We introduce Olmo 3, a new family of 7B and 32B models both Instruct and Think variants. Long chain-of-thought thinking improves reasoning tasks like math and coding.

Olmo is a series of **O**pen **l**anguage **mo**dels designed to enable the science of language models. These models are pre-trained on the Dolma 3 dataset and post-trained on the Dolci datasets. We are releasing all code, checkpoints, logs (coming soon), and associated training details.

The core models released in this batch include the following:

Stage	Olmo 3 7B Think	Olmo 3 32B Think	Olmo 3 7B Instruct
Base Model	Olmo-3-7B	Olmo-3-32B	Olmo-3-7B
SFT	Olmo-3-7B-Think-SFT	Olmo-3-32B-Think-SFT	Olmo-3-7B-Instruct-SFT
DPO	Olmo-3-7B-Think-DPO	Olmo-3-32B-Think-DPO	Olmo-3-7B-Instruct-DPO
Final Models (RLVR)	Olmo-3-7B-Think	Olmo-3-32B-Think	Olmo-3-7B-Instruct

Downloads last month
61,316

Safetensors

Model size | 7B params | Tensor type | BF16 | Chat template | Files info

Inference Providers NEW

Text Generation

This model isn't deployed by any Inference Provider. 2 Ask for provider support

Model tree for allenai/Olmo-3-7B-Think

Base model allenai/Olmo-3-1025-7B
Finetuned allenai/Olmo-3-7B-Think-SFT
Finetuned allenai/Olmo-3-7B-Think-DPO
Finetuned (7) this model
Adapters 9 models
Finetunes 11 models
Merges 1 model
Quantizations 32 models

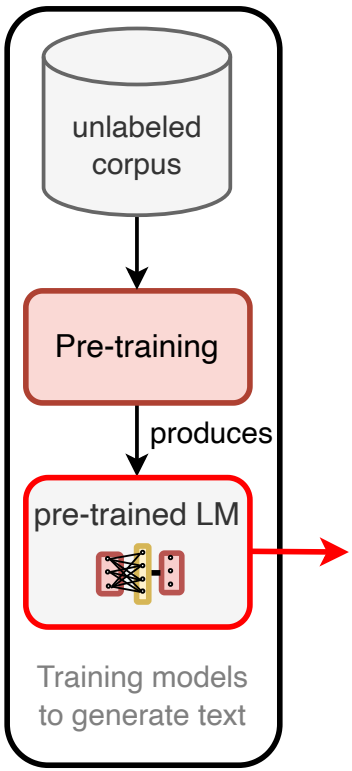
Dataset used to train allenai/Olmo-3-7B-Think

allenai/Dolci-Think-RL-7B
Viewer • Updated Jan 5 • 102k • 896 • 17

https://huggingface.co/collections/allenai/olmo-3

We will go through these different parts

The landscape of generative AI research: **Foundations**



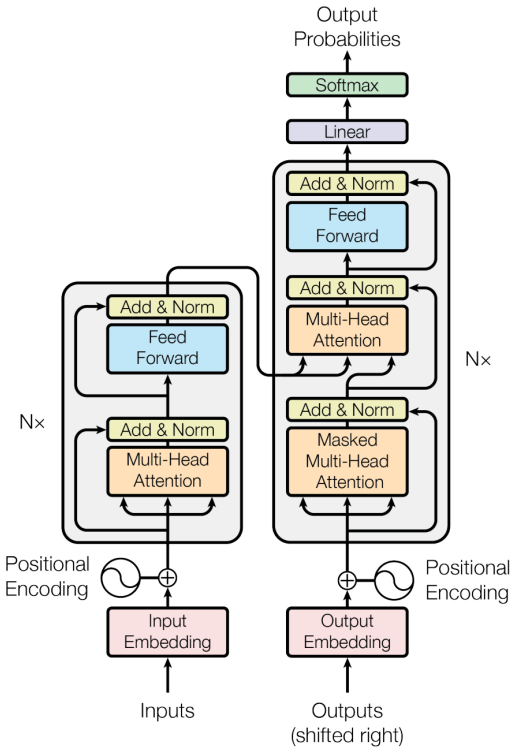
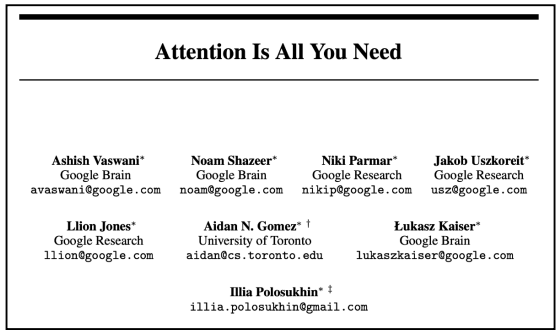


Figure 1: The Transformer - model architecture.

Model architectures

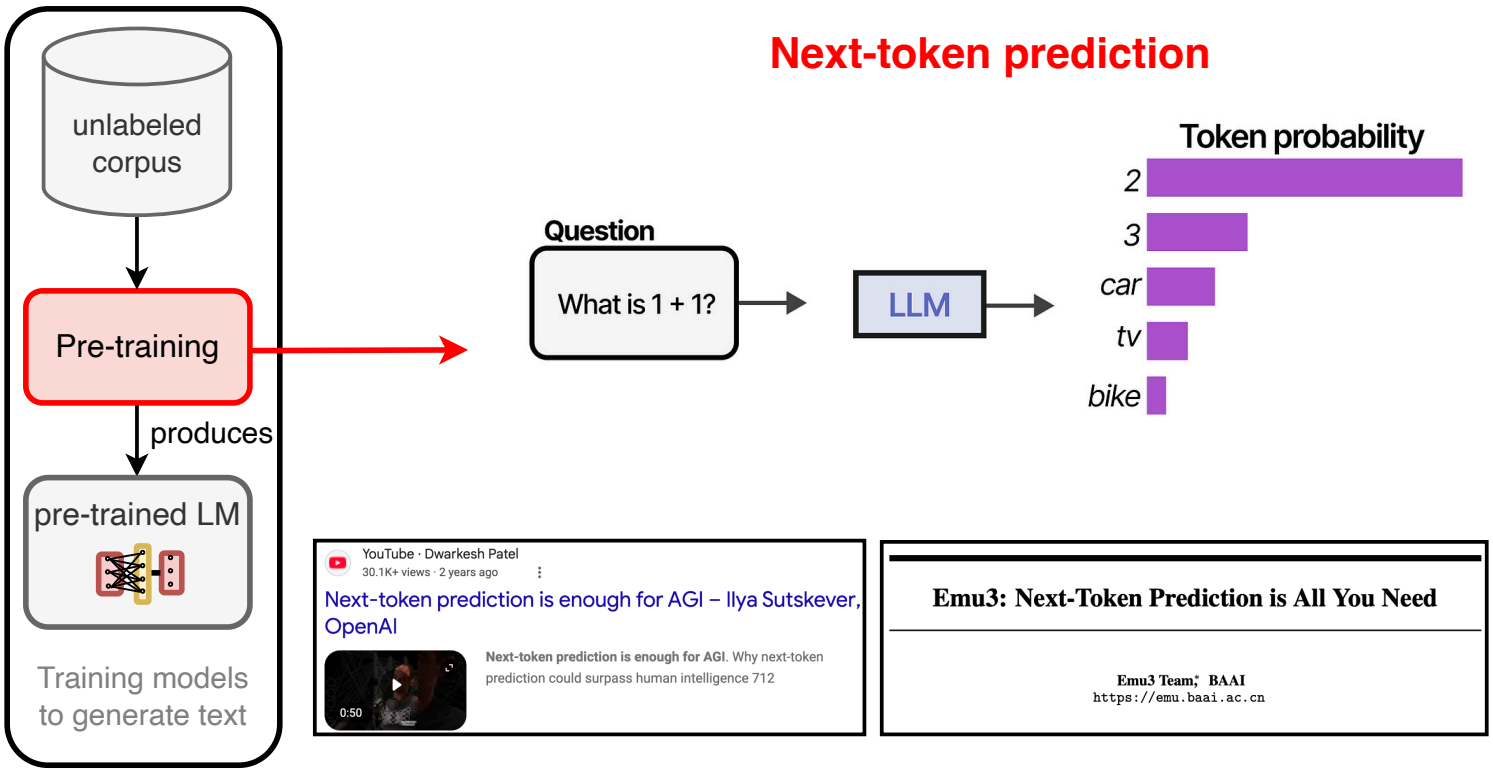


Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

- **Model architecture design**: what kinds of problems can specific architectures (e.g., transformers) express? Are there clear limitations?

The landscape of generative AI research: **Foundations**



- **Pre-training and evaluation**: Why is next-token prediction such an effective signal and how do we effectively measure model knowledge?

The landscape of generative AI research: **Alignment**

Loss function design

$$-\log\sigma\left(\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}-\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

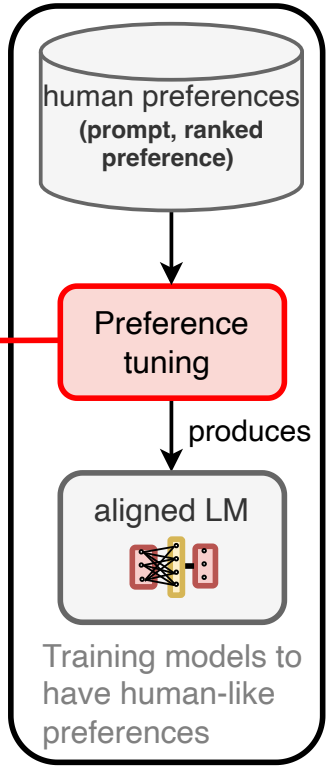
~

Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*
Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray

Direct Preference Optimization: Your Language Model is Secretly a Reward Model

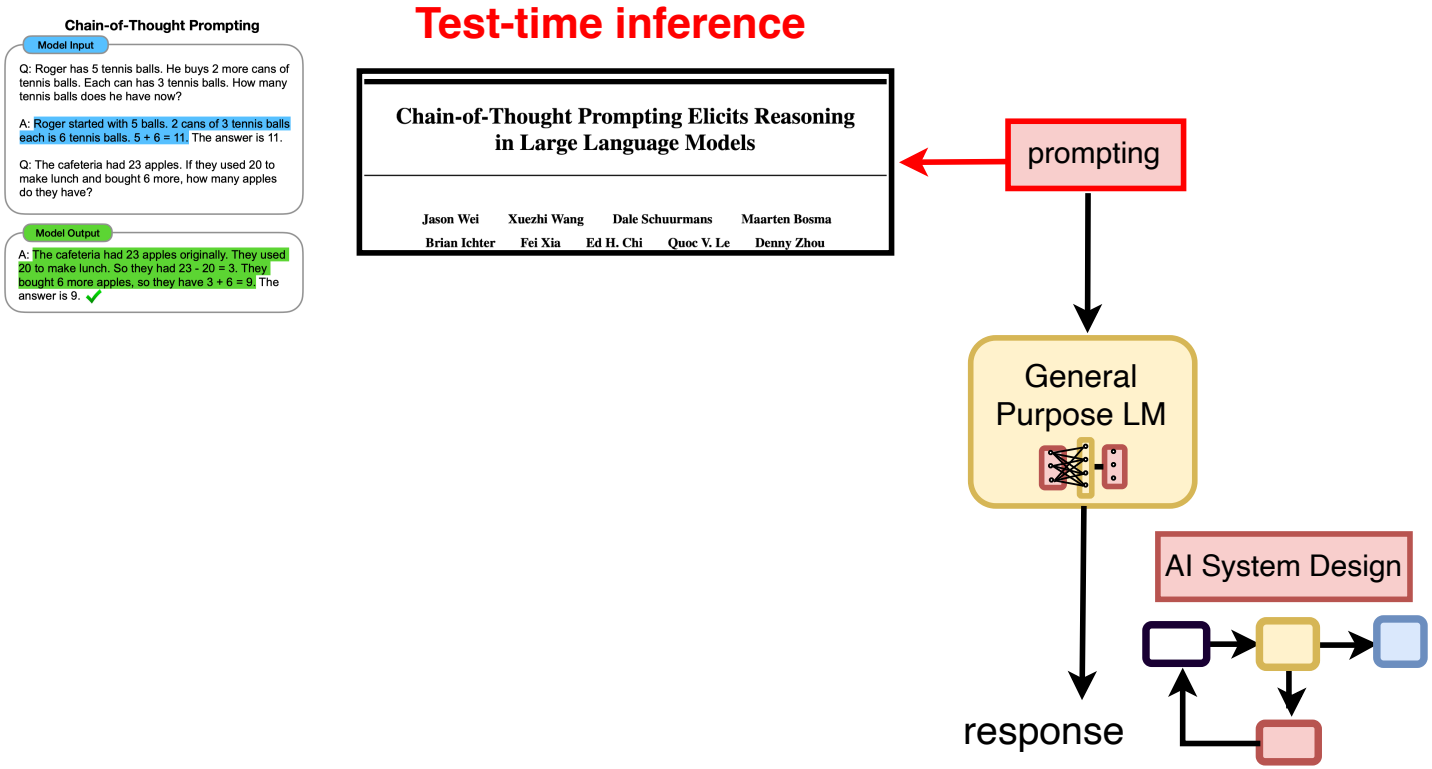
Rafael Rafailov*† Archit Sharma*† Eric Mitchell*†
Stefano Ermon†‡ Christopher D. Manning† Chelsea Finn†



- **Loss design**: What do we do when we train models to preferences? Does the optimization have an internal logic?

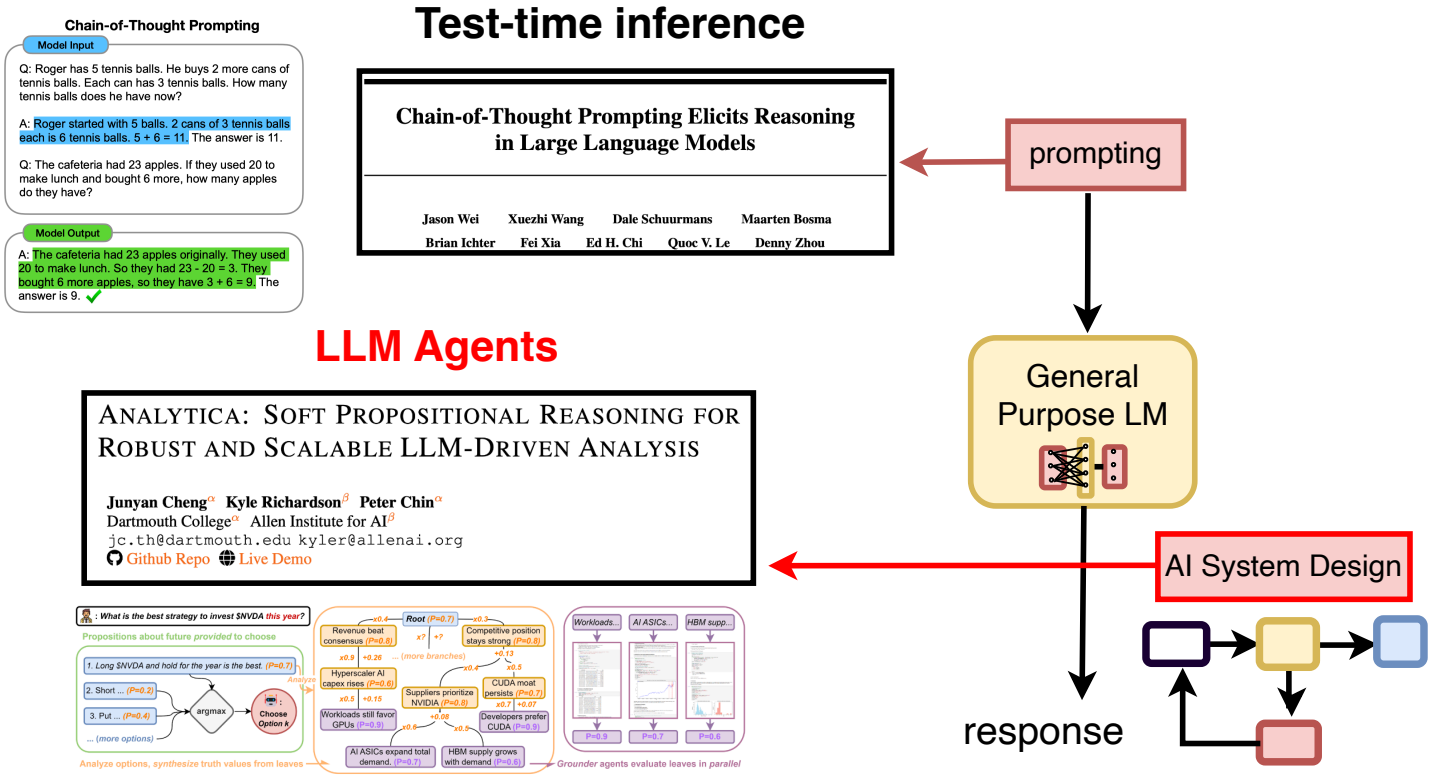
What do these computations mean?

The landscape of generative AI research: **Inference**



- **Test-time inference**: What kinds of reasoning structures do LLMs build? How can we know if the reasoning is correct and sound?

The landscape of generative AI research: **Inference**

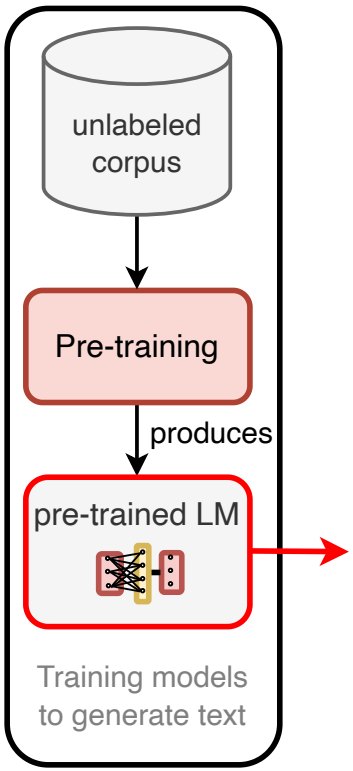


- **Test-time inference**: What kinds of reasoning structures do LLMs build?
  How can we know if the reasoning is correct and sound?

LLMs build structures, what are these structures?

How are all these areas connected to one another?

Programming languages for LLM research



Restricted Access Sequence Processing Language (Rasp)

Thinking Like Transformers

Gail Weiss [1] Yoav Goldberg [2 3] Eran Yahav [1]

```
1  def num_prevs(bools) {
2      prevs = select(indices,indices,<=);
3      return (indices+1) *
4             aggregate(prevs,
5                       indicator(bools));
6  }
7  n_opens = num_prevs(tokens=="(");
8  n_closes = num_prevs(tokens==")");
9  balance = n_opens - n_closes;
10 prev_imbalances = num_prevs(balance<0);
11 dyck1PTF = "F" if prev_imbalances > 0
12                   else
13            ("T" if balance==0 else "P");
```

Figure 15: RASP code for computing Dyck-1-PTF with the parentheses (and).

Masked Hard-Attention Transformers Recognize Exactly the Star-Free Languages

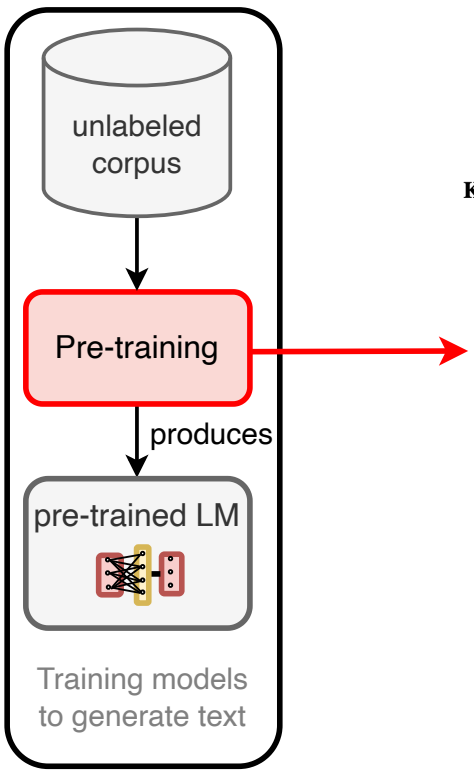
Andy Yang, University of Notre Dame; David Chiang, University of Notre Dame; Dana Angluin, Yale University

$$P_\ell(i) := \blacktriangleright_j\ [j < i, 1]\ \ Q_\ell(j) : 0$$
$$S_r(i) := \blacktriangleleft_i\ [j > i, 1]\ \ Q_r(j) : 0.$$
$$I(i) := (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)).$$
$$B_\ell(i) := \blacktriangleright_j\ [j < i, \neg I(j)]\ \ Q_\ell(j) : 0$$
$$A_r(i) := \blacktriangleleft_j\ [j > i, \neg I(j)]\ \ Q_r(j) : 0$$
$$C(i) := I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)).$$
$$Y(i) := \blacktriangleright_j\ [1, \neg C(j)]\ \ 0 : 1.$$

Figure: B-rasp for computing Dyck-1.

- **Rasp**: Languages that capture the underlying computational model of transformers (Weiss et al.2021; Yang and Chiang2024; Yang et al.2024; Huang et al.2026).

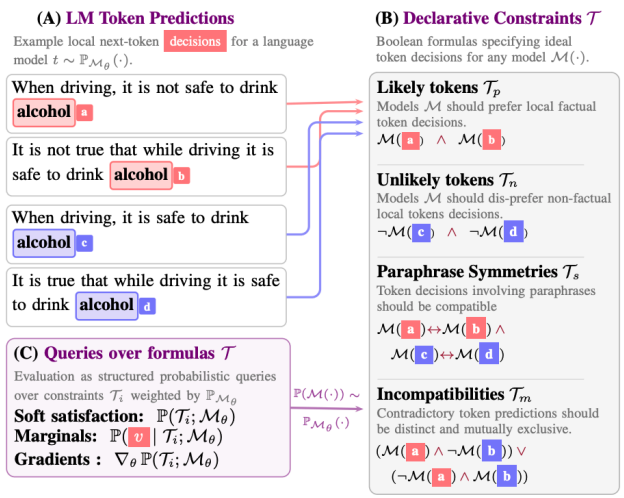
Programming languages for LLM research



ModelLog

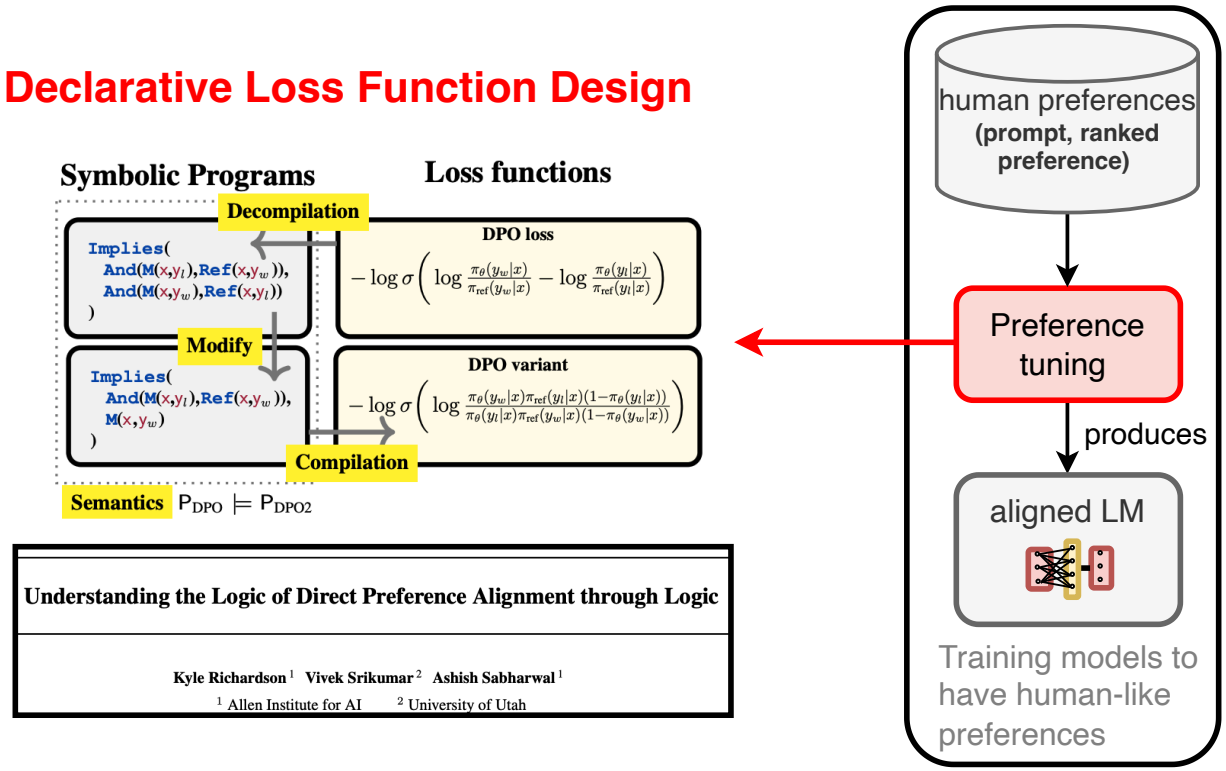
From Token Probabilities to Semantic Constraints: Towards Declarative Probabilistic Evaluation of Language Models

Kyle Richardson[1] Cullen Anderson[2] Pranav Balakrishnan[2] Takuto Ban[2] Daksha Ladia[2]
Ankita Gupta[2] Marisa Hudspeth[2]



- Declarative-style probabilistic programs, probabilistic logic, takes much inspiration from PLP (De Raedt and Kimmig2015; Fierens et al.2015; Manhaeve et al.2018).

Programming languages for LLM research



- Declarative-style probabilistic programs, probabilistic logic, takes much inspiration from PLP (De Raedt and Kimmig2015; Fierens et al.2015; Manhaeve et al.2018).

Languages provide a rich, declarative semantics

Programming languages for LLM research

LMQL

Prompting Is Programming: A Query Language for Large Language Models

LUCA BEURER-KELLNER, MARC FISCHER, and MARTIN VECHEV, ETH Zurich

```
@lmql.query
def meaning_of_life():
    '''lmql
    # top-level strings are prompts
    "Q: What is the answer to life, the \
     universe and everything?"

    # generation via (constrained) variables
    "A: [ANSWER]" where \
        len(ANSWER) < 120 and STOPS_AT(ANSWER, ".")

    # results are directly accessible
    print("LLM returned", ANSWER)

    # use typed variables for guaranteed
    # output format
    "The answer is [NUM: int]"

    # query programs are just functions
    return NUM
    '''

# so from Python, you can just do this
meaning_of_life() # 42
```

Copper

CoTs as Tractable Probabilistic Programs

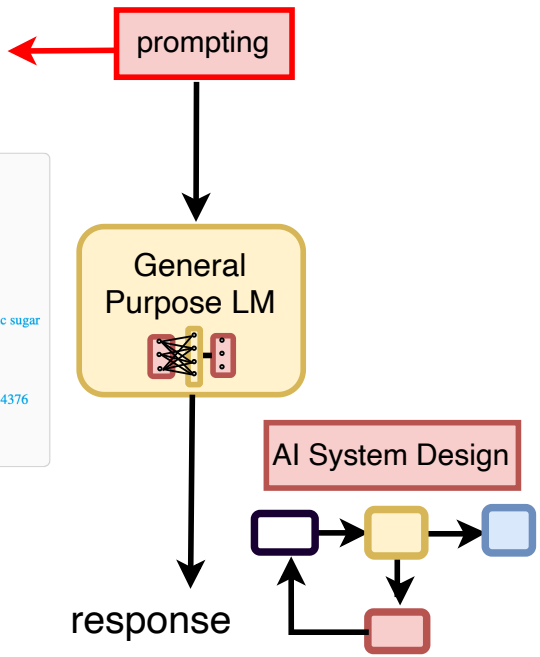
Kyle Richardson* (Allen Institute for Artificial Intelligence), Yu Feng* (University of Pennsylvania), Poorva Garg (University of California Los Angeles), Junyan Cheng (Dartmouth College), Guy Van den Broeck (University of California Los Angeles), Dan Roth (University of Pennsylvania)

```
@CotProgParser()
def pp():
  P1:  bool =~ Flip("step1", 0.86, [0.7])

  P2:  bool =~ Flip("step2", 0.96, [0.9])
  ## observe(P2) ## Optional condition, set P2=True

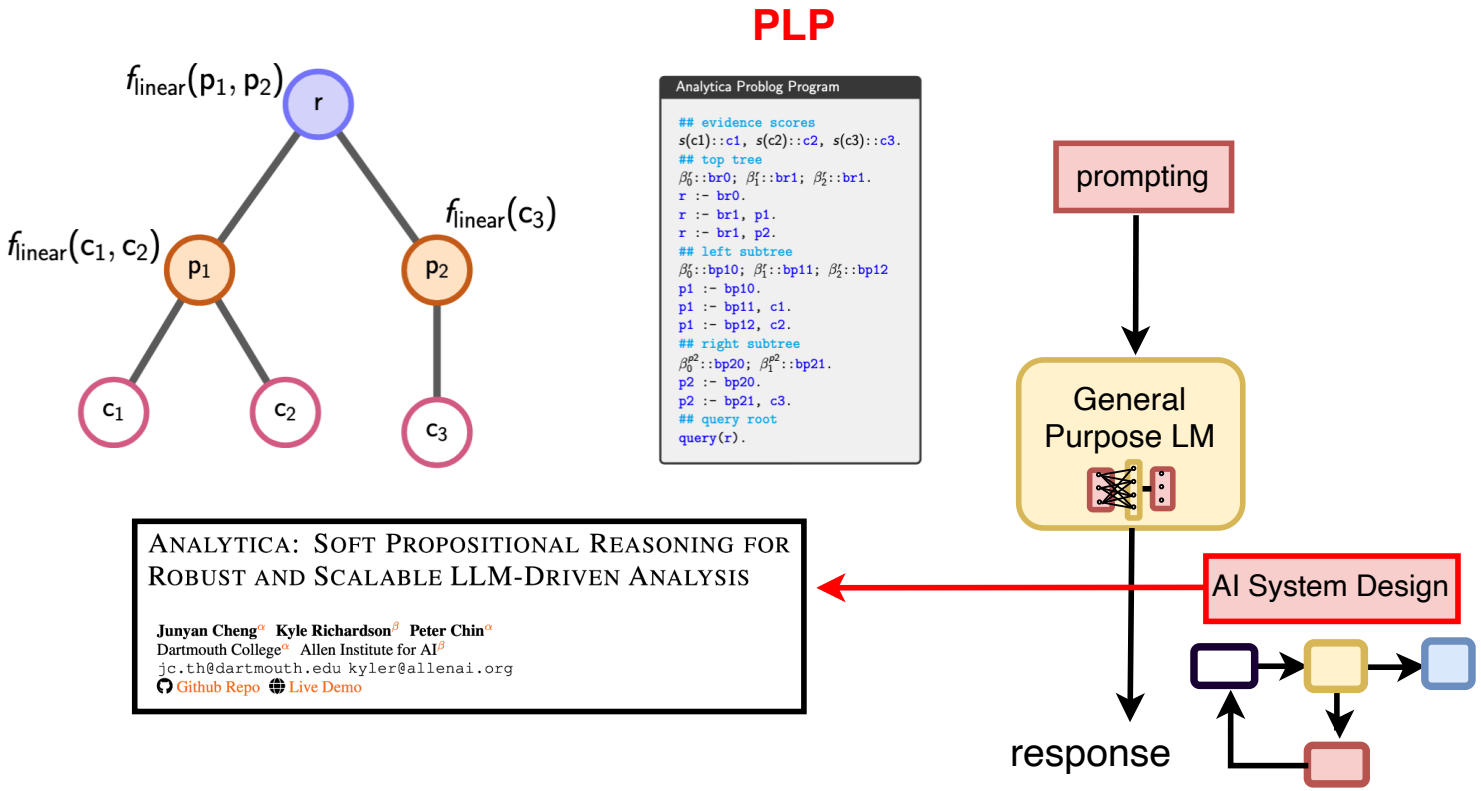
  if P1 and P2:
    P3:  bool =~ Flip("no", 0.53, [0.8])
    yield P3 ## returns "no" if P3 true, exception otherwise, syntactic sugar
  else:
    raise ValueError("Cannot establish P1 or P2")

## Program inference
pp.inference(query={"no"}, compute_grad=True) ## 0.4376
## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
pp.inference(query={"ValueError"}) ## 0.5624
pp.inference(query={"no"}, use_conf=True) ## 0.504
```



- Imperative-style programming, add structure to LLM workflows, mix with conventional programming (Beurer-Kellner et al.2023; Mandel et al.2026; Wu et al.2026).

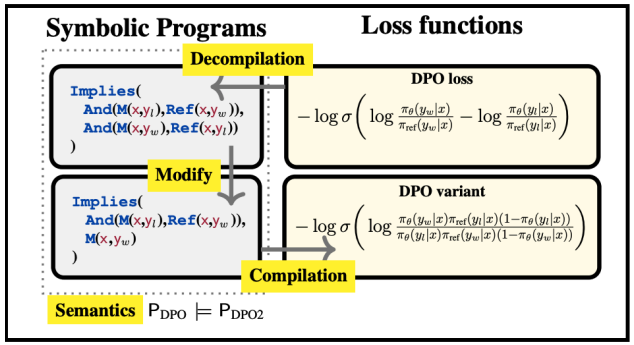
Programming languages for LLM research



- Modeling message passing, score aggregation in forecasting agents, LLM soft scoring; problog (De Raedt et al.2007; Dries et al.2015).

Language Model Programming

Language Model Programming




```python
@CotProgParser()
def pp():
  P1:  bool =~ Flip("step1", 0.86, [0.7])

  P2:  bool =~ Flip("step2", 0.96, [0.9])
  ## observe(P2) ## Optional condition, set P2=True

  if P1 and P2:
    P3:  bool =~ Flip("no", 0.53, [0.8])
    yield P3 ## returns "no" if P3 true, exception otherwise, syntactic sugar
  else:
    raise ValueError("Cannot establish P1 or P2")

## Program inference
pp.inference(query={"no"}, compute_grad=True)  ## 0.4376
## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
pp.inference(query={"ValueError"})  ## 0.5624
pp.inference(query={"no"}, use_conf=True)  ## 0.504
```

$$
\begin{aligned}
P_\ell(i) &:= \blacktriangleright_j\ [j < i, 1]\ \ Q_\ell(j) : 0 \\
S_r(i) &:= \blacktriangleleft_i\ [j > i, 1]\ \ Q_r(j) : 0. \\
I(i) &:= (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)). \\
B_\ell(i) &:= \blacktriangleright_j\ [j < i, \neg I(j)]\ \ Q_\ell(j) : 0 \\
A_r(i) &:= \blacktriangleleft_j\ [j > i, \neg I(j)]\ \ Q_r(j) : 0 \\
C(i) &:= I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)). \\
Y(i) &:= \blacktriangleright_j\ [1, \neg C(j)]\ \ 0 : 1.
\end{aligned}
$$

```python
@lmql.query
def meaning_of_life():
    '''lmql
    # top-level strings are prompts
    "Q: What is the answer to life, the \
     universe and everything?"

    # generation via (constrained) variables
    "A: [ANSWER]" where \
        len(ANSWER) < 120 and STOPS_AT(ANSWER, ".")

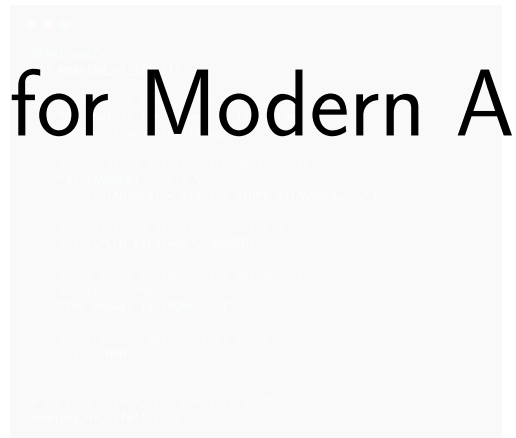
    # results are directly accessible
    print("LLM returned", ANSWER)

    # use typed variables for guaranteed
    # output format
    "The answer is [NUM: int]"

    # query programs are just functions
    return NUM
    '''

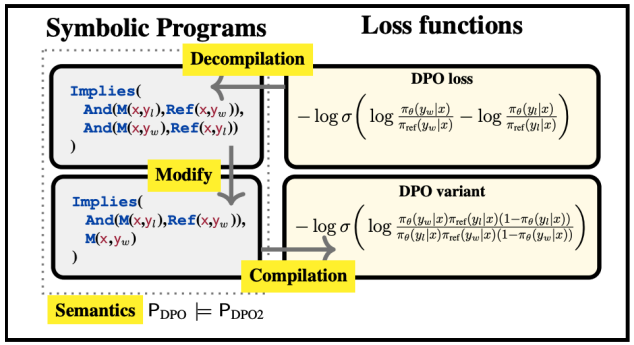
# so from Python, you can just do this
meaning_of_life() # 42
```

- **LM programming**: The study and design of languages for specifying, composing and reasoning about LLM computations.



Discrete Methods for Modern AI

Discrete Modeling Techniques




```python
@CotProgParser()
def pp():
  P1:  bool =~ Flip("step1", 0.86, [0.7])

  P2:  bool =~ Flip("step2", 0.96, [0.9])
  ## observe(P2) ## Optional condition, set P2=True

  if P1 and P2:
    P3:  bool =~ Flip("no", 0.53, [0.8])
    yield P3 ## returns "no" if P3 true, exception otherwise, syntactic sugar
  else:
    raise ValueError("Cannot establish P1 or P2")

## Program inference
pp.inference(query={"no"}, compute_grad=True)  ## 0.4376
## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
pp.inference(query={"ValueError"})  ## 0.5624
pp.inference(query={"no"}, use_conf=True)  ## 0.504
```

$$
\begin{aligned}
P_\ell(i) &:= \blacktriangleright_j\ [j < i, 1]\ \ Q_\ell(j) : 0 \\
S_r(i) &:= \blacktriangleleft_i\ [j > i, 1]\ \ Q_r(j) : 0. \\
I(i) &:= (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)). \\
B_\ell(i) &:= \blacktriangleright_j\ [j < i, \neg I(j)]\ \ Q_\ell(j) : 0 \\
A_r(i) &:= \blacktriangleleft_j\ [j > i, \neg I(j)]\ \ Q_r(j) : 0 \\
C(i) &:= I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)). \\
Y(i) &:= \blacktriangleright_j\ [1, \neg C(j)]\ \ 0 : 1.
\end{aligned}
$$

```python
@lmql.query
def meaning_of_life():
    '''lmql
    # top-level strings are prompts
    "Q: What is the answer to life, the \
     universe and everything?"

    # generation via (constrained) variables
    "A: [ANSWER]" where \
        len(ANSWER) < 120 and STOPS_AT(ANSWER, ".")

    # results are directly accessible
    print("LLM returned", ANSWER)

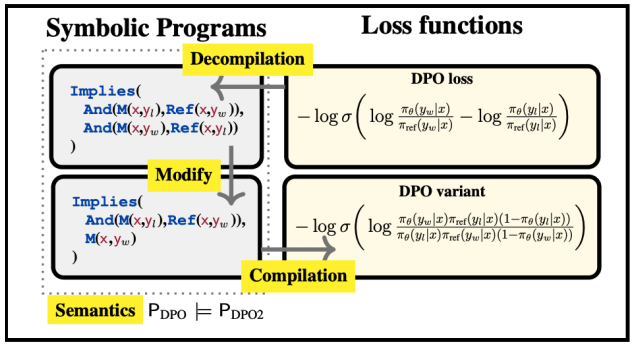
    # use typed variables for guaranteed
    # output format
    "The answer is [NUM: int]"

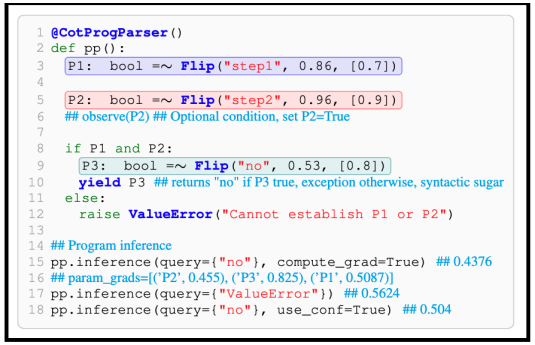
    # query programs are just functions
    return NUM
    '''

# so from Python, you can just do this
meaning_of_life() # 42
```

- **What this course is about**: The formal techniques needed for this kind of modeling, using and implementing symbolic programming languages.

Discrete Modeling Techniques





```python
@CotProgParser()
def pp():
  P1:  bool =~ Flip("step1", 0.86, [0.7])

  P2:  bool =~ Flip("step2", 0.96, [0.9])
  ## observe(P2) ## Optional condition, set P2=True

  if P1 and P2:
    P3:  bool =~ Flip("no", 0.53, [0.8])
    yield P3 ## returns "no" if P3 true, exception otherwise, syntactic sugar
  else:
    raise ValueError("Cannot establish P1 or P2")

## Program inference
pp.inference(query={"no"}, compute_grad=True)  ## 0.4376
## param_grads=[('P2', 0.455), ('P3', 0.825), ('P1', 0.5087)]
pp.inference(query={"ValueError"})  ## 0.5624
pp.inference(query={"no"}, use_conf=True)  ## 0.504
```

$$
\begin{aligned}
P_\ell(i) &:= \blacktriangleright_j\ [j < i, 1]\ \ Q_\ell(j) : 0 \\
S_r(i) &:= \blacktriangleleft_i\ [j > i, 1]\ \ Q_r(j) : 0. \\
I(i) &:= (Q_\ell(i) \wedge S_r(i)) \vee (Q_r(i) \wedge P_\ell(i)). \\
B_\ell(i) &:= \blacktriangleright_j\ [j < i, \neg I(j)]\ \ Q_\ell(j) : 0 \\
A_r(i) &:= \blacktriangleleft_j\ [j > i, \neg I(j)]\ \ Q_r(j) : 0 \\
C(i) &:= I(i) \vee (Q_\ell(i) \wedge A_r(i)) \vee (Q_r(i) \wedge B_\ell(i)). \\
Y(i) &:= \blacktriangleright_j\ [1, \neg C(j)]\ \ 0 : 1.
\end{aligned}
$$

```python
@lmql.query
def meaning_of_life():
    '''lmql
    # top-level strings are prompts
    "Q: What is the answer to life, the \
     universe and everything?"

    # generation via (constrained) variables
    "A: [ANSWER]" where \
        len(ANSWER) < 120 and STOPS_AT(ANSWER, ".")

    # results are directly accessible
    print("LLM returned", ANSWER)

    # use typed variables for guaranteed
    # output format
    "The answer is [NUM: int]"

    # query programs are just functions
    return NUM
    '''

# so from Python, you can just do this
meaning_of_life() # 42
```

- **Specific goals**: Follow the recent literature on LLM programming, build and implement symbolic programming languages for AI.

Course structure

Course structure: themes

Lecture	Discrete methods	Modern AI problem
FOUNDATIONS		
1. Discrete foundations	logic, SAT, model counting	*the toolkit (today)*
2. Language models	transformer basics, RASP	AI toolkit
APPLICATIONS		
3. Fine-tuning	semantic loss, weighted model counting	learning with constraints, encoders
4. Prediction & evaluation	probabilistic logic	pre-trained model evaluation, sampling
5. Alignment	semantics of preference modeling	direct preference optimization
6–. Inference	discrete probabilistic programming	prompting, chain-of-thought

- **Two parts**: first the discrete toolkit, then applications to each stage of the LLM pipeline.

Course structure: themes

Lecture	Discrete methods	Modern AI problem
FOUNDATIONS		
1. Discrete foundations	logic, SAT, model counting	*the toolkit (today)*
2. Language models	transformer basics, RASP	AI toolkit
APPLICATIONS		
3. Fine-tuning	semantic loss, weighted model counting	learning with constraints, encoders
4. Prediction & evaluation	probabilistic logic	pre-trained model evaluation, sampling
5. Alignment	semantics of preference modeling	direct preference optimization
6–. Inference	discrete probabilistic programming	prompting, chain-of-thought

- **Today**: the shared logical foundations, with SAT and model counting as the central algorithmic problems.

Course structure: themes

Lecture	Discrete methods	Modern AI problem
FOUNDATIONS		
1. Discrete foundations	logic, SAT, model counting	*the toolkit (today)*
2. Language models	transformer basics, RASP	AI toolkit
APPLICATIONS		
3. Fine-tuning	semantic loss, weighted model counting	learning with constraints, encoders
4. Prediction & evaluation	probabilistic logic	pre-trained model evaluation, sampling
5. Alignment	semantics of preference modeling	direct preference optimization
6–. Inference	discrete probabilistic programming	prompting, chain-of-thought

- What kinds of problems can transformers express? Viewing models as programs.

Course structure: themes

Lecture	Discrete methods	Modern AI problem
FOUNDATIONS		
1. Discrete foundations	logic, SAT, model counting	*the toolkit (today)*
2. Language models	transformer basics, RASP	AI toolkit
APPLICATIONS		
3. Fine-tuning	semantic loss, weighted model counting	learning with constraints, encoders
4. Prediction & evaluation	probabilistic logic	pre-trained model evaluation, sampling
5. Alignment	semantics of preference modeling	direct preference optimization
6–. Inference	discrete probabilistic programming	prompting, chain-of-thought

- How do we train models to satisfy constraints? Semantic loss is weighted model counting.

Course structure: themes

Lecture	Discrete methods	Modern AI problem
FOUNDATIONS		
1. Discrete foundations	logic, SAT, model counting	*the toolkit (today)*
2. Language models	transformer basics, RASP	AI toolkit
APPLICATIONS		
3. Fine-tuning	semantic loss, weighted model counting	learning with constraints, encoders
4. Prediction & evaluation	probabilistic logic	pre-trained model evaluation, sampling
5. Alignment	semantics of preference modeling	direct preference optimization
6–. Inference	discrete probabilistic programming	prompting, chain-of-thought

- Next-token prediction and evaluation as probabilistic constraint satisfaction.

Course structure: themes

Lecture	Discrete methods	Modern AI problem
FOUNDATIONS		
1. Discrete foundations	logic, SAT, model counting	*the toolkit (today)*
2. Language models	transformer basics, RASP	AI toolkit
APPLICATIONS		
3. Fine-tuning	semantic loss, weighted model counting	learning with constraints, encoders
4. Prediction & evaluation	probabilistic logic	pre-trained model evaluation, sampling
5. Alignment	semantics of preference modeling	direct preference optimization
6–. Inference	discrete probabilistic programming	prompting, chain-of-thought

- What do we do when we tune models to preferences? Does the optimization have an internal logic?

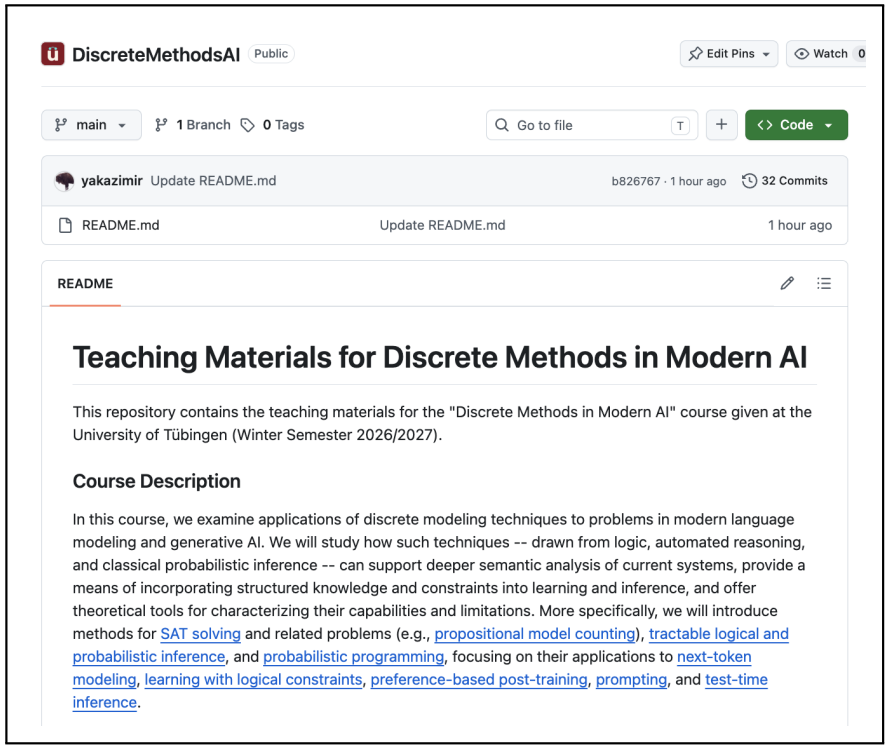
Course structure: themes

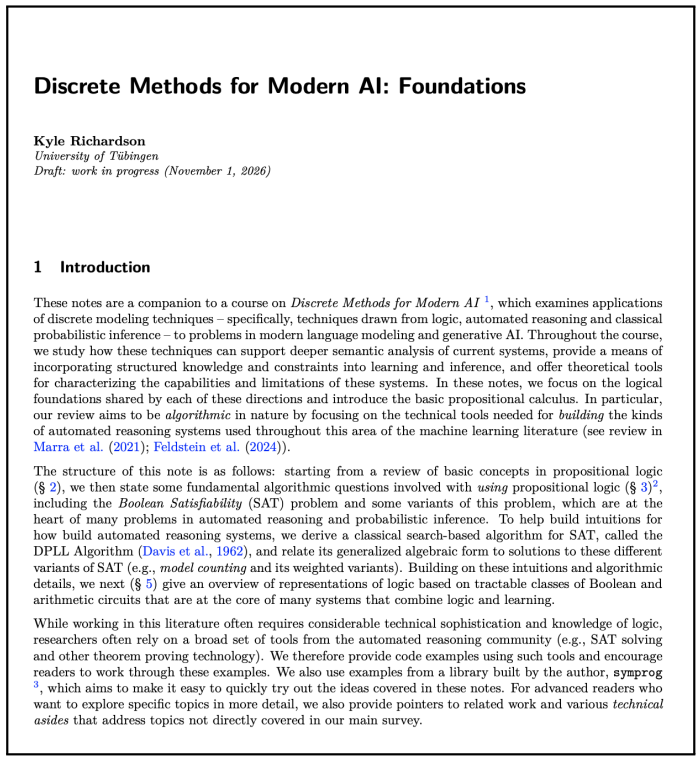
Lecture	Discrete methods	Modern AI problem
FOUNDATIONS		
1. Discrete foundations	logic, SAT, model counting	*the toolkit (today)*
2. Language models	transformer basics, RASP	AI toolkit
APPLICATIONS		
3. Fine-tuning	semantic loss, weighted model counting	learning with constraints, encoders
4. Prediction & evaluation	probabilistic logic	pre-trained model evaluation, sampling
5. Alignment	semantics of preference modeling	direct preference optimization
6–. Inference	discrete probabilistic programming	prompting, chain-of-thought

- What reasoning structures do LLMs build at test time? *(to be announced)*

Course materials and website

https://github.com/TuLLI-AI/DiscreteMethodsAI

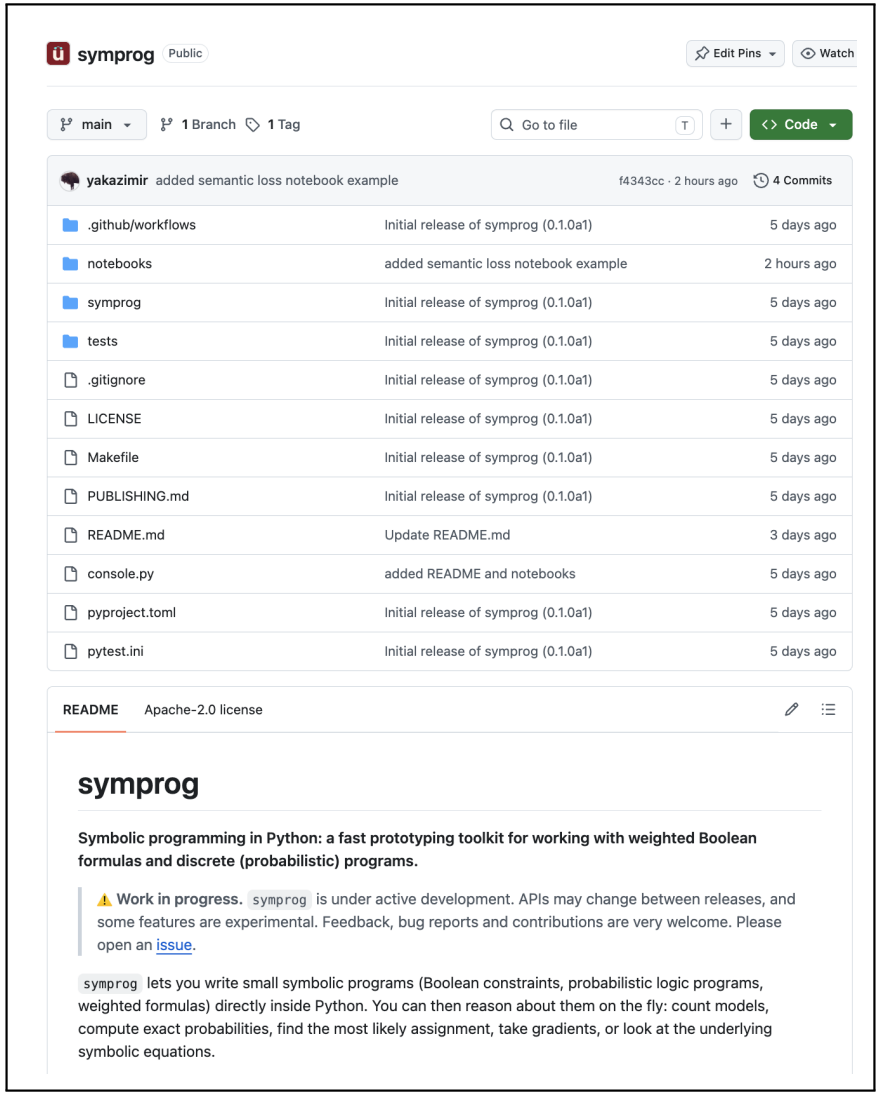




- **Expectations**: Attend weekly lectures, follow the readings and Python notebooks (on GitHub), quickly learn new topics.

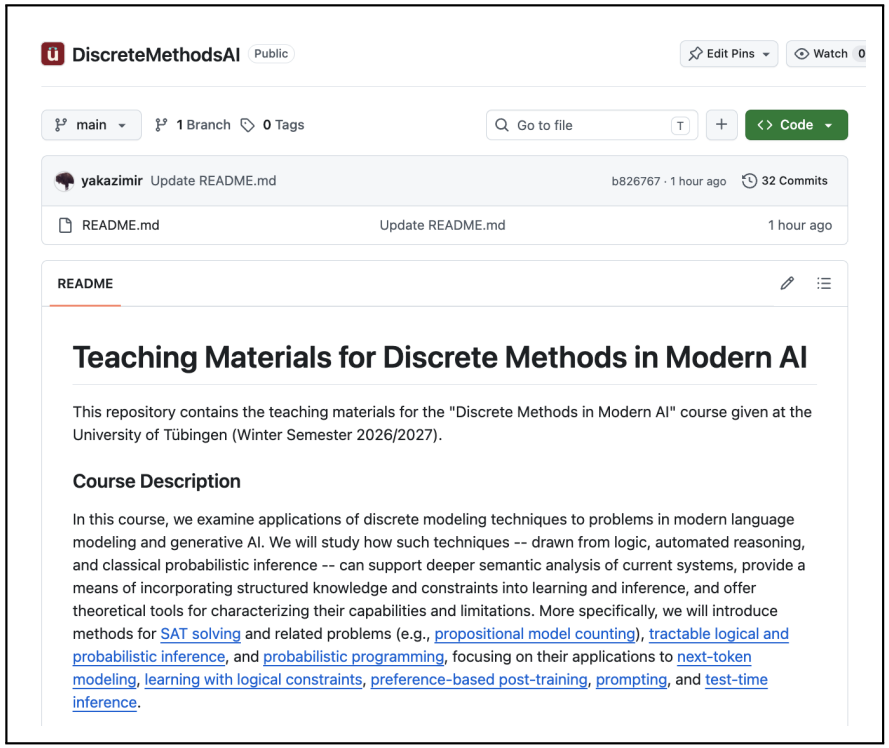
symprog: library for symbolic computations

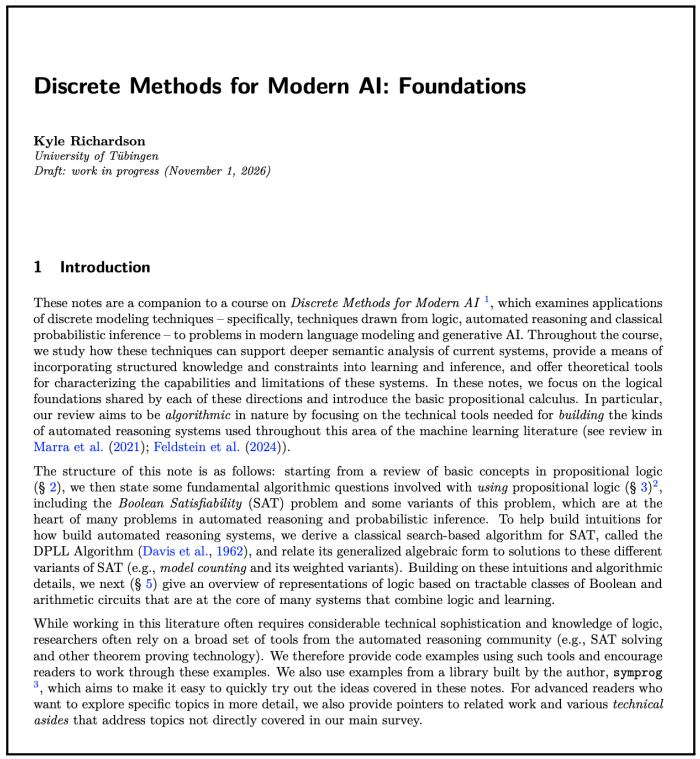
https://github.com/TuLLI-AI/DiscreteMethodsAI



Course evaluation

https://github.com/TuLLI-AI/DiscreteMethodsAI

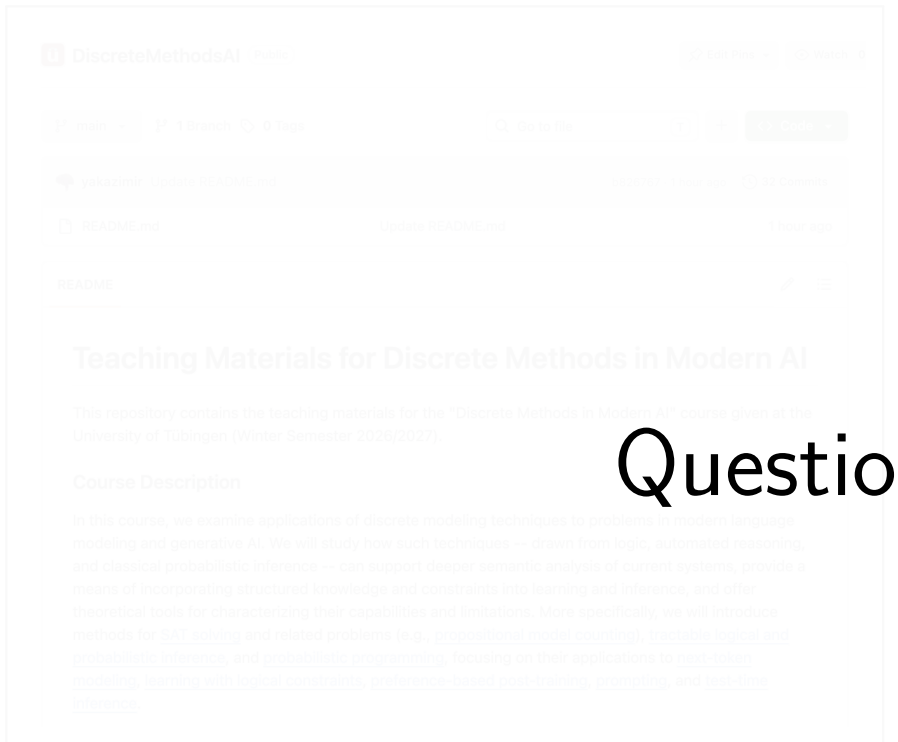




- **Evaluation**: In-class comprehension exam (20%, Dec. 7), project and presentation (*details forthcoming*).

Course evaluation

https://github.com/TuLLI-AI/DiscreteMethodsAI





Questions?

- **Evaluation**: In-class comprehension exam (20%, Dec. 7), project and presentation (*details forthcoming*).

(Propositional) Logic

Propositional logic in four questions

$F := \ldots$	$\overline{v}(F)$	$\Gamma \models \alpha$	SAT
How do we **write** formulas?	When is a formula **true**?	What **follows** from what?	How do we **compute** it?

Each question has a precise answer, and the last one, *computation*, is where most of this course lives.

Propositional logic in four questions

$F := \ldots$	$\overline{v}(F)$	$\Gamma \models \alpha$	SAT
How do we **write** formulas?	When is a formula **true**?	What **follows** from what?	How do we **compute** it?

Each question has a precise answer, and the last one, *computation*, is where most of this course lives.

Syntax: how do we write formulas?

Grammar

$$F \;:=\; P_i \mid \neg F \mid (F_1 \wedge F_2) \mid (F_1 \vee F_2) \mid (F_1 \to F_2) \mid (F_1 \leftrightarrow F_2) \mid \top \mid \bot$$

atoms

$P_1, P_2, P_3, \ldots$

basic propositions

connectives

$\neg \;\; \wedge \;\; \vee \;\; \to \;\; \leftrightarrow$

build bigger formulas

constants

$\top$ (**true**), $\bot$ (**false**)

always true / false

Syntax: how do we write formulas?

Grammar

$$F \;:=\; P_i \mid \neg F \mid (F_1 \wedge F_2) \mid (F_1 \vee F_2) \mid (F_1 \to F_2) \mid (F_1 \leftrightarrow F_2) \mid \top \mid \bot$$

atoms

$P_1, P_2, P_3, \ldots$

basic propositions

connectives

$\neg \;\; \wedge \;\; \vee \;\; \to \;\; \leftrightarrow$

build bigger formulas

constants

$\top$ (**true**), $\bot$ (**false**)

always true / false

✓ **well-formed**

$$(P_1 \wedge \neg P_2) \to (P_1 \vee \neg P_3)$$

$$(P_1 \vee \neg P_1) \wedge P_2$$

$$(P_1 \to \neg P_1) \wedge (P_2 \leftrightarrow \neg P_2)$$

✗ **not well-formed**

$$(P_1 \wedge \wedge \neg P_2)$$

$$(P_1 \vee P_2))$$

$$P_2 \vee \to \neg P_3$$

Syntax: how do we write formulas?

Grammar

$$F \;:=\; P_i \mid \neg F \mid (F_1 \wedge F_2) \mid (F_1 \vee F_2) \mid (F_1 \to F_2) \mid (F_1 \leftrightarrow F_2) \mid \top \mid \bot$$

atoms

$P_1, P_2, P_3, \ldots$
basic propositions

connectives

$\neg \;\; \wedge \;\; \vee \;\; \to \;\; \leftrightarrow$
build bigger formulas

constants

$\top$ **(true)**, $\bot$ **(false)**
always true / false

✓ **well-formed**

$(P_1 \wedge \neg P_2) \to (P_1 \vee \neg P_3)$

$(P_1 \vee \neg P_1) \wedge P_2$

$(P_1 \to \neg P_1) \wedge (P_2 \leftrightarrow \neg P_2)$

✗ **not well-formed**

$(P_1 \wedge \wedge \neg P_2)$

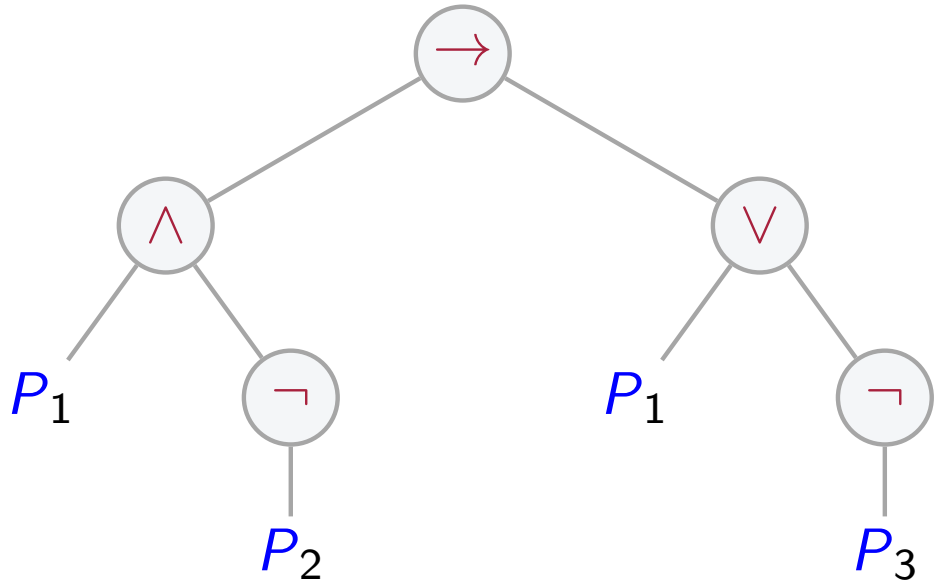
$(P_1 \vee P_2))$

$P_2 \vee \to \neg P_3$

Syntax only checks *shape*: $P_2 \leftrightarrow \neg P_2$ is well-formed, even though it can never be true.

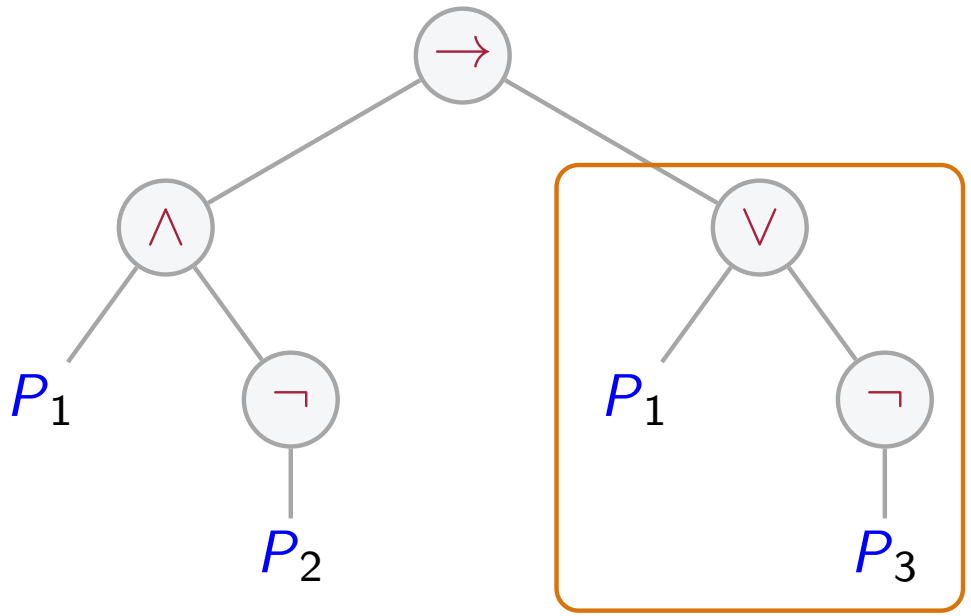
Syntax: formulas are trees

$$(P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$$



Syntax: formulas are trees

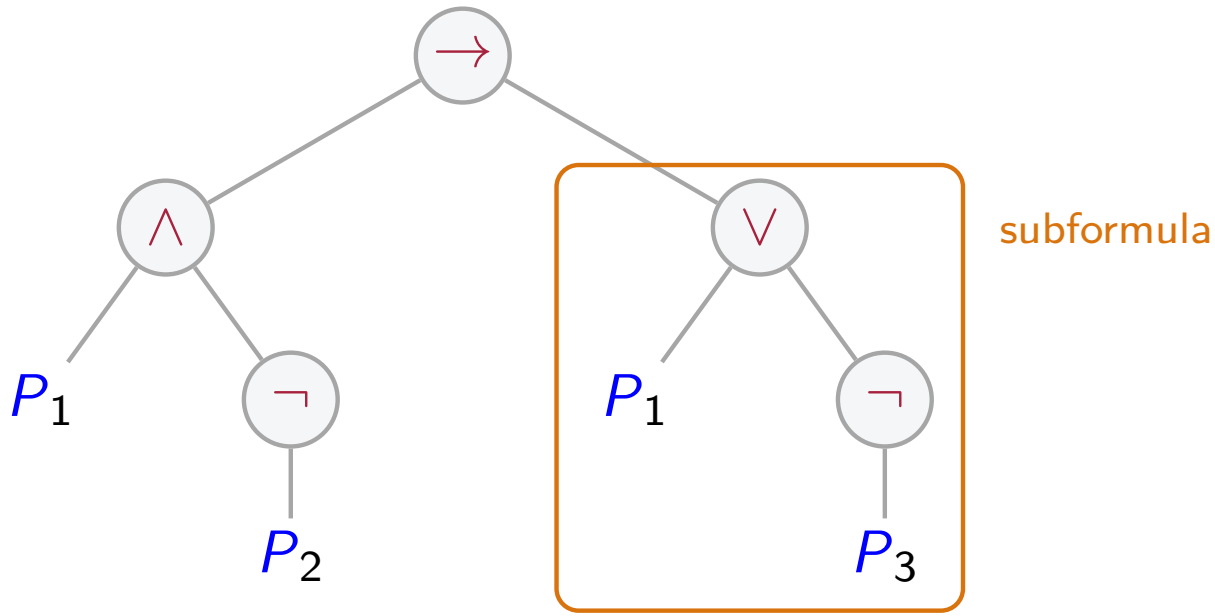
$$(P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$$



- **Subformulas** are subtrees, e.g., $P_1 \vee \neg P_3$.

Syntax: formulas are trees

$$(P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$$



- **Subformulas** are subtrees, e.g., $P_1 \vee \neg P_3$.
- **Convention**: drop parentheses, with $\neg$ binding tightest, then $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$:

  $P_1 \wedge \neg P_2 \rightarrow P_1 \vee \neg P_3$ is the same tree.

Semantics: when is a formula true?

$F := \ldots$	$\overline{v}(F)$	$\Gamma \models \alpha$	SAT
How do we **write** formulas?	When is a formula **true**?	What **follows** from what?	How do we **compute** it?

Each question has a precise answer, and the last one, *computation*, is where most of this course lives.

Semantics: when is a formula true?

Truth assignment

$v : \{\text{atoms}\} \to \{1, 0\}$ bivalent

$\overline{v}(\neg F) = \neg\, \overline{v}(F), \quad \overline{v}(F_1 \circ F_2) = \overline{v}(F_1) \circ \overline{v}(F_2)$ compositional

where $\circ$ is any of $\wedge, \vee, \to, \leftrightarrow$, computed by the table below

P_1	P_2	$\neg P_1$	$P_1 \wedge P_2$	$P_1 \vee P_2$	$P_1 \to P_2$	$P_1 \leftrightarrow P_2$
1	1	0	**1**	**1**	**1**	**1**
1	0	0	0	**1**	0	0
0	1	**1**	0	**1**	**1**	0
0	0	**1**	0	0	**1**	**1**

Semantics: when is a formula true?

Truth assignment

$v : \{\text{atoms}\} \to \{1, 0\}$ bivalent

$\overline{v}(\neg F) = \neg\, \overline{v}(F), \quad \overline{v}(F_1 \circ F_2) = \overline{v}(F_1) \circ \overline{v}(F_2)$ compositional

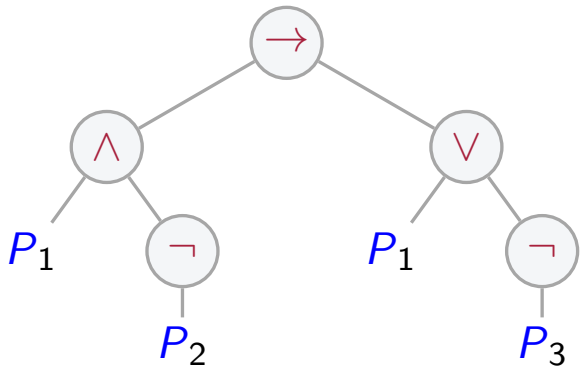
where $\circ$ is any of $\wedge, \vee, \to, \leftrightarrow$, computed by the table below

P_1	P_2	$\neg P_1$	$P_1 \wedge P_2$	$P_1 \vee P_2$	$P_1 \to P_2$	$P_1 \leftrightarrow P_2$
1	1	0	**1**	**1**	**1**	**1**
1	0	0	0	**1**	0	0
0	1	**1**	0	**1**	**1**	0
0	0	**1**	0	0	**1**	**1**

- **Read off in order**: $\wedge$ needs both, $\vee$ needs one, $\to$ fails only when $1 \to 0$, $\leftrightarrow$ needs agreement.

Semantics: computing a full truth table

$$F = (P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$$

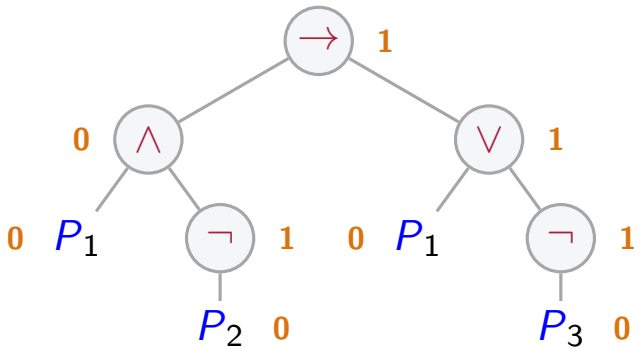


atoms			subformulas				root
P_1	P_2	P_3	$\neg P_2$	$P_1 \wedge \neg P_2$	$\neg P_3$	$P_1 \vee \neg P_3$	F

Recipe: one column per node of the tree, ordered bottom-up, and one row per world.

Semantics: computing a full truth table

$$F = (P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$$

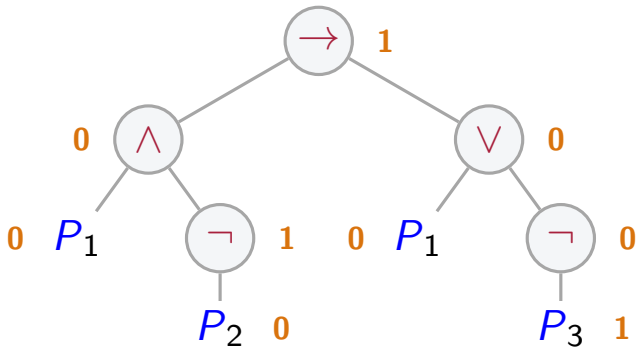


atoms			subformulas				root
P_1	P_2	P_3	$\neg P_2$	$P_1 \wedge \neg P_2$	$\neg P_3$	$P_1 \vee \neg P_3$	F
0	0	0	1	0	1	1	**1**

For each row: write the world on the leaves, then **propagate** values up the tree.

Semantics: computing a full truth table

$$F = (P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$$

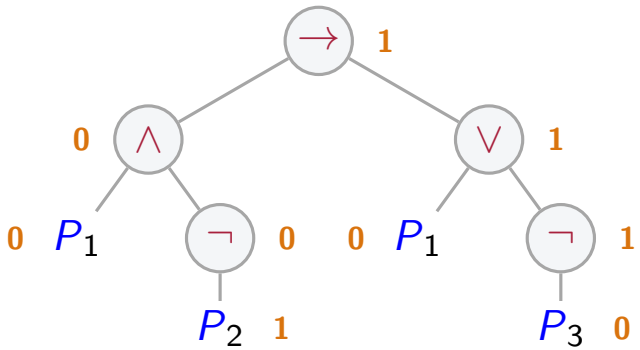


atoms			subformulas				root
P_1	P_2	P_3	$\neg P_2$	$P_1 \wedge \neg P_2$	$\neg P_3$	$P_1 \vee \neg P_3$	F
0	0	0	1	0	1	1	**1**
0	0	1	1	0	0	0	**1**

For each row: write the world on the leaves, then **propagate** values up the tree.

Semantics: computing a full truth table

$$F = (P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$$

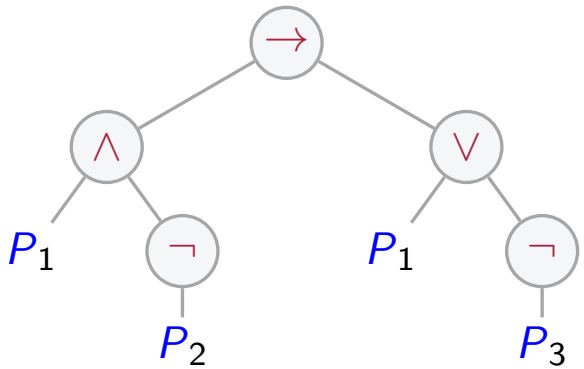


atoms			subformulas				root
P_1	P_2	P_3	$\neg P_2$	$P_1 \wedge \neg P_2$	$\neg P_3$	$P_1 \vee \neg P_3$	F
0	0	0	1	0	1	1	**1**
0	0	1	1	0	0	0	**1**
0	1	0	0	0	1	1	**1**

For each row: write the world on the leaves, then **propagate** values up the tree.

Semantics: computing a full truth table

$$F = (P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$$

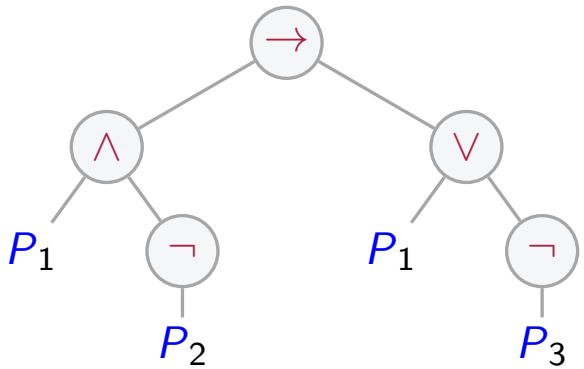


atoms			subformulas				root
P_1	P_2	P_3	$\neg P_2$	$P_1 \wedge \neg P_2$	$\neg P_3$	$P_1 \vee \neg P_3$	F
0	0	0	1	0	1	1	**1**
0	0	1	1	0	0	0	**1**
0	1	0	0	0	1	1	**1**
0	1	1	0	0	0	0	**1**
1	0	0	1	1	1	1	**1**
1	0	1	1	1	0	1	**1**
1	1	0	0	0	1	1	**1**
1	1	1	0	0	0	1	**1**

Repeat for all $2^3 = 8$ worlds: cost $= 2^n \times$ (size of the formula).

Semantics: computing a full truth table

$$F = (P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$$

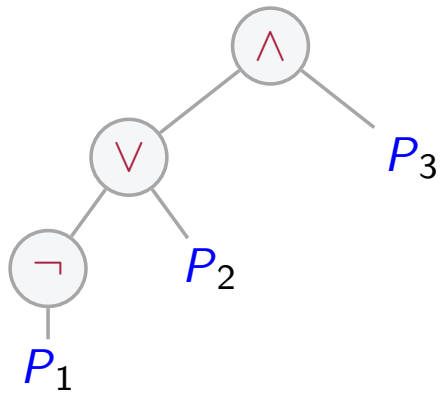


atoms			subformulas				root
P_1	P_2	P_3	$\neg P_2$	$P_1 \wedge \neg P_2$	$\neg P_3$	$P_1 \vee \neg P_3$	F
0	0	0	1	0	1	1	**1**
0	0	1	1	0	0	0	**1**
0	1	0	0	0	1	1	**1**
0	1	1	0	0	0	0	**1**
1	0	0	1	1	1	1	**1**
1	0	1	1	1	0	1	**1**
1	1	0	0	0	1	1	**1**
1	1	1	0	0	0	1	**1**

The root column is all 1s: F is true in every world. We will meet this formula again.

Worlds and models

$F = (\neg P_1 \vee P_2) \wedge P_3$ (our running example)

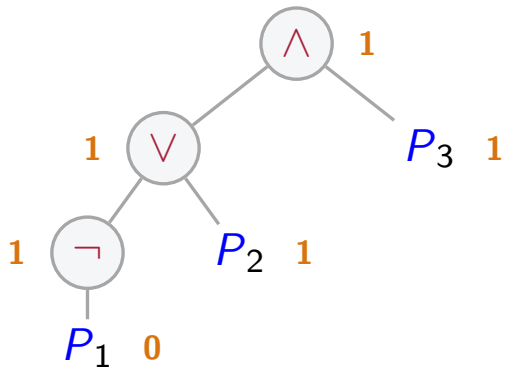


world	atoms			subformulas		root
	P_1	P_2	P_3	$\neg P_1$	$\neg P_1 \vee P_2$	F
w_1	0	0	0	1	1	0
w_2	0	0	1	1	1	**1**
w_3	0	1	0	1	1	0
w_4	0	1	1	1	1	**1**
w_5	1	0	0	0	0	0
w_6	1	0	1	0	0	0
w_7	1	1	0	0	1	0
w_8	1	1	1	0	1	**1**

Each row is a **world** w: one way of assigning 0/1 to all atoms. With n atoms there are 2^n worlds.

Worlds and models

$F = (\neg P_1 \vee P_2) \wedge P_3$ (our running example)

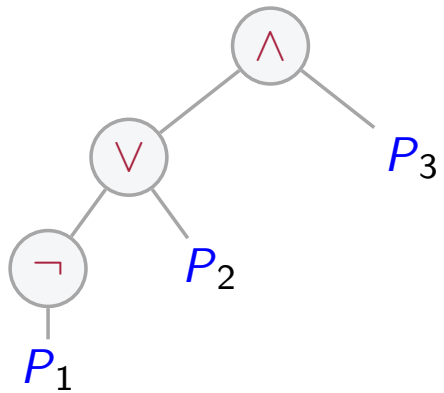


world	atoms			subformulas		root
	P_1	P_2	P_3	$\neg P_1$	$\neg P_1 \vee P_2$	F
w_1	0	0	0	1	1	0
w_2	0	0	1	1	1	**1**
w_3	0	1	0	1	1	0
w_4	0	1	1	1	1	**1**
w_5	1	0	0	0	0	0
w_6	1	0	1	0	0	0
w_7	1	1	0	0	1	0
w_8	1	1	1	0	1	**1**

F is true in w_4, so w_4 is a **model** of F, written $w_4 \models F$.

Worlds and models

$F = (\neg P_1 \vee P_2) \wedge P_3$ (our running example)



world	atoms			subformulas		root
	P_1	P_2	P_3	$\neg P_1$	$\neg P_1 \vee P_2$	F
w_1	0	0	0	1	1	0
w_2	0	0	1	1	1	**1**
w_3	0	1	0	1	1	0
w_4	0	1	1	1	1	**1**
w_5	1	0	0	0	0	0
w_6	1	0	1	0	0	0
w_7	1	1	0	0	1	0
w_8	1	1	1	0	1	**1**

The models of F are w_2, w_4, w_8. Keep this example in mind: we will *count* these later.

Truth tables in SymPy, a library for symbolic computation

```
## pip install sympy
from sympy import *
from sympy.logic.boolalg import truth_table

P1, P2, P3 = symbols("P1 P2 P3")
F = Implies(And(P1,Not(P2)), Or(P1,Not(P3)))

for world, value in truth_table(F, [P1, P2, P3]):
    print(world, value)
## [0, 0, 0] True
## [0, 0, 1] True
## ...
## [1, 1, 1] True
```

Logical equivalence

- $F_1 \equiv F_2$ when they are true in **exactly the same worlds**, i.e., their columns match.

Logical equivalence

- $F_1 \equiv F_2$ when they are true in **exactly the same worlds**, i.e., their columns match.

$$P_1 \rightarrow P_2 \;\equiv\; \neg P_1 \vee P_2$$

P_1	P_2	$P_1 \rightarrow P_2$	$\neg P_1 \vee P_2$
1	1	**1**	**1**
1	0	0	0
0	1	**1**	**1**
0	0	**1**	**1**

Logical equivalence

- $F_1 \equiv F_2$ when they are true in **exactly the same worlds**, i.e., their columns match.

$$P_1 \rightarrow P_2 \;\equiv\; \neg P_1 \vee P_2$$

P_1	P_2	$P_1 \rightarrow P_2$	$\neg P_1 \vee P_2$
1	1	**1**	**1**
1	0	0	0
0	1	**1**	**1**
0	0	**1**	**1**

$$P_1 \leftrightarrow P_2 \;\equiv\; (P_1 \rightarrow P_2) \wedge (P_2 \rightarrow P_1)$$

P_1	P_2	$P_1 \leftrightarrow P_2$	$(P_1 \rightarrow P_2) \wedge (P_2 \rightarrow P_1)$
1	1	**1**	**1**
1	0	0	0
0	1	0	0
0	0	**1**	**1**

Logical equivalence

- $F_1 \equiv F_2$ when they are true in **exactly the same worlds**, i.e., their columns match.

$$P_1 \rightarrow P_2 \equiv \neg P_1 \vee P_2$$

P_1	P_2	$P_1 \rightarrow P_2$	$\neg P_1 \vee P_2$
1	1	**1**	**1**
1	0	0	0
0	1	**1**	**1**
0	0	**1**	**1**

$$P_1 \leftrightarrow P_2 \equiv (P_1 \rightarrow P_2) \wedge (P_2 \rightarrow P_1)$$

P_1	P_2	$P_1 \leftrightarrow P_2$	$(P_1 \rightarrow P_2) \wedge (P_2 \rightarrow P_1)$
1	1	**1**	**1**
1	0	0	0
0	1	0	0
0	0	**1**	**1**

Consequence: $\rightarrow$ and $\leftrightarrow$ are shorthand, so we can work with $\neg, \wedge, \vee$ only.

The laws of Boolean algebra

De Morgan

$$\neg(F_1 \wedge F_2) \equiv \neg F_1 \vee \neg F_2$$
$$\neg(F_1 \vee F_2) \equiv \neg F_1 \wedge \neg F_2$$

Commutativity

$$F_1 \wedge F_2 \equiv F_2 \wedge F_1$$
$$F_1 \vee F_2 \equiv F_2 \vee F_1$$

Identity and complement

$$F \wedge \top \equiv F \qquad F \vee \bot \equiv F$$
$$F \wedge \neg F \equiv \bot \qquad F \vee \neg F \equiv \top$$

Associativity

$$F_1 \wedge (F_2 \wedge F_3) \equiv (F_1 \wedge F_2) \wedge F_3$$
$$F_1 \vee (F_2 \vee F_3) \equiv (F_1 \vee F_2) \vee F_3$$

Idempotency and double negation

$$F \wedge F \equiv F \qquad F \vee F \equiv F$$
$$\neg\neg F \equiv F$$

Distributivity

$$F_1 \wedge (F_2 \vee F_3) \equiv (F_1 \wedge F_2) \vee (F_1 \wedge F_3)$$
$$F_1 \vee (F_2 \wedge F_3) \equiv (F_1 \vee F_2) \wedge (F_1 \vee F_3)$$

The laws of Boolean algebra

De Morgan

$$\neg(F_1 \wedge F_2) \equiv \neg F_1 \vee \neg F_2$$
$$\neg(F_1 \vee F_2) \equiv \neg F_1 \wedge \neg F_2$$

Commutativity

$$F_1 \wedge F_2 \equiv F_2 \wedge F_1$$
$$F_1 \vee F_2 \equiv F_2 \vee F_1$$

Identity and complement

$$F \wedge \top \equiv F \qquad F \vee \bot \equiv F$$
$$F \wedge \neg F \equiv \bot \qquad F \vee \neg F \equiv \top$$

Associativity

$$F_1 \wedge (F_2 \wedge F_3) \equiv (F_1 \wedge F_2) \wedge F_3$$
$$F_1 \vee (F_2 \vee F_3) \equiv (F_1 \vee F_2) \vee F_3$$

Idempotency and double negation

$$F \wedge F \equiv F \qquad F \vee F \equiv F$$
$$\neg\neg F \equiv F$$

Distributivity

$$F_1 \wedge (F_2 \vee F_3) \equiv (F_1 \wedge F_2) \vee (F_1 \wedge F_3)$$
$$F_1 \vee (F_2 \wedge F_3) \equiv (F_1 \vee F_2) \wedge (F_1 \vee F_3)$$

Notation: by associativity and commutativity we can write $\bigwedge_{i=1}^{n} F_i = F_1 \wedge \cdots \wedge F_n$ and $\bigvee_{i=1}^{n} F_i = F_1 \vee \cdots \vee F_n$.

The laws of Boolean algebra

De Morgan

$$\neg(F_1 \wedge F_2) \equiv \neg F_1 \vee \neg F_2$$
$$\neg(F_1 \vee F_2) \equiv \neg F_1 \wedge \neg F_2$$

Commutativity

$$F_1 \wedge F_2 \equiv F_2 \wedge F_1$$
$$F_1 \vee F_2 \equiv F_2 \vee F_1$$

Identity and complement

$$F \wedge \top \equiv F \qquad F \vee \bot \equiv F$$
$$F \wedge \neg F \equiv \bot \qquad F \vee \neg F \equiv \top$$

Associativity

$$F_1 \wedge (F_2 \wedge F_3) \equiv (F_1 \wedge F_2) \wedge F_3$$
$$F_1 \vee (F_2 \vee F_3) \equiv (F_1 \vee F_2) \vee F_3$$

These should become second nature

Idempotency and double negation

$$F \wedge F \equiv F \qquad F \vee F \equiv F$$
$$\neg\neg F \equiv F$$

Distributivity

$$F_1 \wedge (F_2 \vee F_3) \equiv (F_1 \wedge F_2) \vee (F_1 \wedge F_3)$$
$$F_1 \vee (F_2 \wedge F_3) \equiv (F_1 \vee F_2) \wedge (F_1 \vee F_3)$$

Notation: by associativity and commutativity we can write $\bigwedge_{i=1}^{n} F_i = F_1 \wedge \cdots \wedge F_n$ and $\bigvee_{i=1}^{n} F_i = F_1 \vee \cdots \vee F_n$.

Using the laws: rewriting formulas

Pushing negation inwards

$\neg(P_1 \rightarrow P_2)$

Using the laws: rewriting formulas

Pushing negation inwards

$$\neg(P_1 \rightarrow P_2)$$
$$\equiv \neg(\neg P_1 \vee P_2) \qquad \rightarrow \text{ as } \vee$$

Using the laws: rewriting formulas

Pushing negation inwards

$$
\begin{aligned}
& \neg(P_1 \rightarrow P_2) \\
\equiv\ & \neg(\neg P_1 \vee P_2) && \rightarrow \text{ as } \vee \\
\equiv\ & \neg\neg P_1 \wedge \neg P_2 && \text{De Morgan}
\end{aligned}
$$

Using the laws: rewriting formulas

Pushing negation inwards

$$
\begin{aligned}
& \neg(P_1 \rightarrow P_2) && \\
\equiv\ & \neg(\neg P_1 \vee P_2) && \rightarrow \text{ as } \vee \\
\equiv\ & \neg\neg P_1 \wedge \neg P_2 && \text{De Morgan} \\
\equiv\ & P_1 \wedge \neg P_2 && \text{double negation}
\end{aligned}
$$

Using the laws: rewriting formulas

Pushing negation inwards

$$
\begin{aligned}
& \neg(P_1 \rightarrow P_2) \\
\equiv\ & \neg(\neg P_1 \vee P_2) && \rightarrow \text{ as } \vee \\
\equiv\ & \neg\neg P_1 \wedge \neg P_2 && \text{De Morgan} \\
\equiv\ & P_1 \wedge \neg P_2 && \text{double negation}
\end{aligned}
$$

Simplifying

$$(P_1 \wedge P_2) \vee (P_1 \wedge \neg P_2)$$

Using the laws: rewriting formulas

Pushing negation inwards

$$
\begin{aligned}
& \neg(P_1 \rightarrow P_2) \\
\equiv\ & \neg(\neg P_1 \vee P_2) && \rightarrow \text{ as } \vee \\
\equiv\ & \neg\neg P_1 \wedge \neg P_2 && \text{De Morgan} \\
\equiv\ & P_1 \wedge \neg P_2 && \text{double negation}
\end{aligned}
$$

Simplifying

$$
\begin{aligned}
& (P_1 \wedge P_2) \vee (P_1 \wedge \neg P_2) \\
\equiv\ & P_1 \wedge (P_2 \vee \neg P_2) && \text{distributivity}
\end{aligned}
$$

Using the laws: rewriting formulas

Pushing negation inwards

$$
\begin{aligned}
& \neg(P_1 \rightarrow P_2) \\
\equiv\ & \neg(\neg P_1 \vee P_2) && \rightarrow \text{ as } \vee \\
\equiv\ & \neg\neg P_1 \wedge \neg P_2 && \text{De Morgan} \\
\equiv\ & P_1 \wedge \neg P_2 && \text{double negation}
\end{aligned}
$$

Simplifying

$$
\begin{aligned}
& (P_1 \wedge P_2) \vee (P_1 \wedge \neg P_2) \\
\equiv\ & P_1 \wedge (P_2 \vee \neg P_2) && \text{distributivity} \\
\equiv\ & P_1 \wedge \top && \text{complement}
\end{aligned}
$$

Using the laws: rewriting formulas

Pushing negation inwards

$\neg(P_1 \rightarrow P_2)$

$\equiv \neg(\neg P_1 \vee P_2)$ $\rightarrow$ as $\vee$

$\equiv \neg\neg P_1 \wedge \neg P_2$ De Morgan

$\equiv P_1 \wedge \neg P_2$ double negation

Simplifying

$(P_1 \wedge P_2) \vee (P_1 \wedge \neg P_2)$

$\equiv P_1 \wedge (P_2 \vee \neg P_2)$ distributivity

$\equiv P_1 \wedge \top$ complement

$\equiv P_1$ identity

Using the laws: rewriting formulas

Pushing negation inwards

$$
\begin{aligned}
& \neg(P_1 \to P_2) \\
\equiv\ & \neg(\neg P_1 \vee P_2) && \to \text{ as } \vee \\
\equiv\ & \neg\neg P_1 \wedge \neg P_2 && \text{De Morgan} \\
\equiv\ & P_1 \wedge \neg P_2 && \text{double negation}
\end{aligned}
$$

Simplifying

$$
\begin{aligned}
& (P_1 \wedge P_2) \vee (P_1 \wedge \neg P_2) \\
\equiv\ & P_1 \wedge (P_2 \vee \neg P_2) && \text{distributivity} \\
\equiv\ & P_1 \wedge \top && \text{complement} \\
\equiv\ & P_1 && \text{identity}
\end{aligned}
$$

Rewriting into a fixed shape, a *normal form*, is the first step of most reasoning algorithms (next part).

Inference: satisfiability and validity

$F := \ldots$	$\overline{v}(F)$	$\Gamma \models \alpha$	SAT
How do we **write** formulas?	When is a formula **true**?	What **follows** from what?	How do we **compute** it?

Each question has a precise answer, and the last one, *computation*, is where most of this course lives.

Inference: satisfiability and validity

Definitions

Γ is **satisfiable**	$\exists w : \; w \models F$ for all $F \in \Gamma$	*some* world makes Γ true
Γ is **unsatisfiable**	no such w exists	*no* world makes Γ true
F is **valid** ($\models F$)	$\forall w : \; w \models F$	*every* world, a **tautology**

Inference: satisfiability and validity

Definitions

Γ is **satisfiable**	$\exists w : \; w \models F$ for all $F \in \Gamma$	*some* world makes Γ true
Γ is **unsatisfiable**	no such w exists	*no* world makes Γ true
F is **valid** ($\models F$)	$\forall w : \; w \models F$	*every* world, a **tautology**

running example

$(\neg P_1 \vee P_2) \wedge P_3$

satisfiable ✓
valid ✗

3 of 8 worlds

Inference: satisfiability and validity

Definitions

Γ is **satisfiable**	$\exists w : \; w \models F$ for all $F \in \Gamma$	*some* world makes Γ true
Γ is **unsatisfiable**	no such w exists	*no* world makes Γ true
F is **valid** ($\models F$)	$\forall w : \; w \models F$	*every* world, a **tautology**

running example

$(\neg P_1 \vee P_2) \wedge P_3$

satisfiable ✓
valid ✗
3 of 8 worlds

contradiction

$P_1 \wedge \neg P_1$

satisfiable ✗
valid ✗
0 of 2 worlds

Inference: satisfiability and validity

Definitions

Γ is **satisfiable**	$\exists w: \; w \models F$ for all $F \in \Gamma$	*some* world makes Γ true
Γ is **unsatisfiable**	no such w exists	*no* world makes Γ true
F is **valid** ($\models F$)	$\forall w: \; w \models F$	*every* world, a **tautology**

running example

$(\neg P_1 \vee P_2) \wedge P_3$

satisfiable ✓
valid ✗
3 of 8 worlds

contradiction

$P_1 \wedge \neg P_1$

satisfiable ✗
valid ✗
0 of 2 worlds

our tree from before

$(P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$

satisfiable ✓
valid ✓8 of 8 worlds

Inference: satisfiability and validity

Definitions

Γ is **satisfiable**	$\exists w: \; w \models F$ for all $F \in \Gamma$	*some* world makes Γ true
Γ is **unsatisfiable**	no such w exists	*no* world makes Γ true
F is **valid** ($\models F$)	$\forall w: \; w \models F$	*every* world, a **tautology**

running example

$(\neg P_1 \vee P_2) \wedge P_3$

satisfiable ✓
valid ✗
3 of 8 worlds

contradiction

$P_1 \wedge \neg P_1$

satisfiable ✗
valid ✗
0 of 2 worlds

our tree from before

$(P_1 \wedge \neg P_2) \rightarrow (P_1 \vee \neg P_3)$

satisfiable ✓
valid ✓8 of 8 worlds

The tree from before is a tautology: its antecedent is only true when P_1 is, which already makes the consequent true.

Satisfiability: an example

$$\Gamma = \{\ P_1 \rightarrow P_2,\ \ P_3 \wedge \neg P_2 \wedge \neg P_4\ \}$$

Γ is satisfiable iff *some* world makes *both* formulas true.

Satisfiability: an example

$$\Gamma = \{\ P_1 \rightarrow P_2,\ \ P_3 \wedge \neg P_2 \wedge \neg P_4\ \}$$

A candidate assignment

$v(P_1) = 0,\ \ v(P_2) = 0$
$v(P_3) = 1,\ \ v(P_4) = 0$

P_1	P_2	P_3	P_4	$P_1 \rightarrow P_2$	2nd formula	Γ
0	0	1	0	1	1	**1**

Try an assignment, and evaluate each formula under it.

Satisfiability: an example

$$\Gamma = \{\ P_1 \to P_2,\ \ P_3 \wedge \neg P_2 \wedge \neg P_4\ \}$$

A candidate assignment

$v(P_1) = 0,\ \ v(P_2) = 0$
$v(P_3) = 1,\ \ v(P_4) = 0$

P_1	P_2	P_3	P_4	$P_1 \to P_2$	2nd formula	Γ
0	0	1	0	1	1	**1**

Both formulas are true: v is a **satisfying assignment**, Γ is **satisfiable** ✓

Satisfiability: an example

$$\Gamma' = \{\ P_1 \rightarrow P_2,\ \ P_3 \wedge \neg P_2 \wedge \neg P_4 \wedge \underline{P_1}\ \}$$

The only candidate

$v(P_1) = 1,\ \ v(P_2) = 0$
$v(P_3) = 1,\ \ v(P_4) = 0$

P_1	P_2	P_3	P_4	$P_1 \rightarrow P_2$	2nd formula	Γ'
1	0	1	0	0	1	0

Now the second formula is *only* true when
$P_1 = 1, P_2 = 0, P_3 = 1, P_4 = 0 \ldots$

Satisfiability: an example

$$\Gamma' = \{\ P_1 \to P_2,\quad P_3 \wedge \neg P_2 \wedge \neg P_4 \wedge \underline{P_1}\ \}$$

The only candidate

$v(P_1) = 1,\quad v(P_2) = 0$
$v(P_3) = 1,\quad v(P_4) = 0$

P_1	P_2	P_3	P_4	$P_1 \to P_2$	2nd formula	Γ'
1	0	1	0	0	1	0

. . . but there $P_1 \to P_2$ is false. No assignment makes both true: Γ' is **unsatisfiable** ✗

satisfiability testing

Satisfiability as *consistency*: an ML example

Constraint (what should hold)

P_1: model predicts **A** on input x_1
P_2: model predicts **B** on input x_2

$$P_1 \rightarrow P_2$$

e.g., *if x_1 is a dog, then its paraphrase x_2 is an animal*

Satisfiability as *consistency*: an ML example

Constraint (what should hold)

P_1: model predicts **A** on input x_1
P_2: model predicts **B** on input x_2

$$P_1 \rightarrow P_2$$

e.g., *if x_1 is a dog, then its paraphrase x_2 is an animal*

Predictions (model behavior)

predicts **A** on x_1 P_1
does not predict **B** on x_2 $\neg P_2$

$$P_1 \wedge \neg P_2$$

e.g., *x_1 is a dog, but its paraphrase x_2 is not an animal*

Satisfiability as *consistency*: an ML example

Constraint (what should hold)

P_1: model predicts **A** on input x_1
P_2: model predicts **B** on input x_2

$$P_1 \rightarrow P_2$$

e.g., *if x_1 is a dog, then its paraphrase x_2 is an animal*

Predictions (model behavior)

predicts **A** on x_1 P_1
does not predict **B** on x_2 $\neg P_2$

$$P_1 \wedge \neg P_2$$

e.g., *x_1 is a dog, but its paraphrase x_2 is not an animal*

Are the predictions consistent with the constraint?

$\iff$ Is $\{P_1 \rightarrow P_2,\ P_1 \wedge \neg P_2\}$ satisfiable? **No**: the only world that makes the predictions true violates the constraint.

Satisfiability as *consistency*: an ML example

Constraint (what should hold)

P_1: model predicts **A** on input x_1
P_2: model predicts **B** on input x_2

$$P_1 \rightarrow P_2$$

e.g., *if x_1 is a dog, then its paraphrase x_2 is an animal*

Predictions (model behavior)

predicts **A** on x_1 P_1
does not predict **B** on x_2 $\neg P_2$

$$P_1 \wedge \neg P_2$$

e.g., *x_1 is a dog, but its paraphrase x_2 is not an animal*

Are the predictions consistent with the constraint?

$\iff$ Is $\{P_1 \rightarrow P_2,\ P_1 \wedge \neg P_2\}$ satisfiable? **No**: the only world that makes the predictions true violates the constraint.

Lecture 3: turn this yes/no question into a *probability*, and train the model to make it high.

Inference: logical entailment

Definition

Γ **entails** α, $\quad \Gamma \models \alpha \qquad \forall w :$ if $w \models \Gamma$ then $w \models \alpha$

i.e., *no* world makes Γ true and α false

Examples

$\{P_1, P_2\} \models P_1$ ✓

$P_1 \to P_2 \models (P_1 \wedge P_3) \to P_2$ ✓

$P_1 \wedge P_2 \models P_1 \to P_2$ ✓

$P_1 \to P_2 \not\models P_1 \wedge P_2$ ✗

To check $\Gamma \models \alpha$: look at the worlds where Γ is true, and check that α is true in all of them.

Inference: logical entailment

Definition

Γ **entails** α, $\quad \Gamma \models \alpha \qquad \forall w :$ if $w \models \Gamma$ then $w \models \alpha$

i.e., *no* world makes Γ true and α false

Examples

$\{P_1, P_2\} \models P_1$ ✓

$P_1 \to P_2 \models (P_1 \wedge P_3) \to P_2$ ✓

$P_1 \wedge P_2 \models P_1 \to P_2$ ✓

$P_1 \to P_2 \not\models P_1 \wedge P_2$ ✗

worlds		Γ	α
P_1	P_2	$P_1 \wedge P_2$	$P_1 \to P_2$
1	1	**1**	**1**
1	0	0	0
0	1	0	1
0	0	0	1

Γ is true in only one world, and α is true there too: entailment holds ✓

Inference: logical entailment

Definition

Γ **entails** α, $\quad \Gamma \models \alpha \qquad \forall w :$ if $w \models \Gamma$ then $w \models \alpha$

i.e., *no* world makes Γ true and α false

Examples

$\{P_1, P_2\} \models P_1$ ✓

$P_1 \rightarrow P_2 \models (P_1 \wedge P_3) \rightarrow P_2$ ✓

$P_1 \wedge P_2 \models P_1 \rightarrow P_2$ ✓

$P_1 \rightarrow P_2 \not\models P_1 \wedge P_2$ ✗

worlds		Γ	α
P_1	P_2	$P_1 \rightarrow P_2$	$P_1 \wedge P_2$
1	1	1	1
1	0	0	0
0	1	**1**	**0**
0	0	**1**	**0**

The red worlds make Γ true and α false: they are **counterexamples**, so the entailment fails ✗

Relating entailment, validity and satisfiability

Theorem: relating these different notions

For $\Gamma = \{F_1, \ldots, F_m\}$ and any formula α, the following are equivalent:

$$\Gamma \models \alpha \qquad \text{entailment}$$

$$\iff \quad (F_1 \wedge \cdots \wedge F_m) \rightarrow \alpha \quad \text{is } \textbf{valid} \qquad \text{validity}$$

$$\iff \quad F_1 \wedge \cdots \wedge F_m \wedge \neg\alpha \quad \text{is } \textbf{unsatisfiable} \qquad \text{satisfiability}$$

Relating entailment, validity and satisfiability

Theorem: relating these different notions

For $\Gamma = \{F_1, \ldots, F_m\}$ and any formula α, the following are equivalent:

	$\Gamma \models \alpha$	entailment
$\iff$	$(F_1 \wedge \cdots \wedge F_m) \rightarrow \alpha$ is **valid**	validity
$\iff$	$F_1 \wedge \cdots \wedge F_m \wedge \neg\alpha$ is **unsatisfiable**	satisfiability

Why?

$\Gamma \models \alpha$: no world makes Γ true and α false

same as: no world makes $F_1 \wedge \cdots \wedge F_m \wedge \neg\alpha$ true,

i.e., it has **no model**.

Relating entailment, validity and satisfiability

Theorem: relating these different notions

For $\Gamma = \{F_1, \ldots, F_m\}$ and any formula α, the following are equivalent:

	$\Gamma \models \alpha$	entailment
$\iff$	$(F_1 \wedge \cdots \wedge F_m) \rightarrow \alpha$ is **valid**	validity
$\iff$	$F_1 \wedge \cdots \wedge F_m \wedge \neg\alpha$ is **unsatisfiable**	satisfiability

Why?

$\Gamma \models \alpha$: no world makes Γ true and α false

same as: no world makes $F_1 \wedge \cdots \wedge F_m \wedge \neg\alpha$ true,

i.e., it has **no model**.

$P_1 \wedge P_2 \models P_1 \rightarrow P_2$ becomes:

P_1	P_2	$P_1 \wedge P_2 \wedge \neg(P_1 \rightarrow P_2)$
1	1	0
1	0	0
0	1	0
0	0	0

no model, so the entailment holds ✓

Relating entailment, validity and satisfiability

Theorem: relating these different notions

For $\Gamma = \{F_1, \ldots, F_m\}$ and any formula α, the following are equivalent:

	$\Gamma \models \alpha$	entailment
$\iff$	$(F_1 \wedge \cdots \wedge F_m) \rightarrow \alpha$ is **valid**	validity
$\iff$	$F_1 \wedge \cdots \wedge F_m \wedge \neg\alpha$ is **unsatisfiable**	satisfiability

Why?

$\Gamma \models \alpha$: no world makes Γ true and α false

same as: no world makes $F_1 \wedge \cdots \wedge F_m \wedge \neg\alpha$ true,

i.e., it has **no model**.

$P_1 \wedge P_2 \models P_1 \rightarrow P_2$ becomes:

P_1	P_2	$P_1 \wedge P_2 \wedge \neg(P_1 \rightarrow P_2)$
1	1	0
1	0	0
0	1	0
0	0	0

no model, so the entailment holds ✓

Take-away: Inference is reducible to satisfiability testing.

Theorem: relating these different notions

Entailment as satisfiability in Z3, applying Theorem 1

```
## pip install z3-solver
from z3 import *

P1, P2 = Bools("P1 P2")
solver = Solver()

solver.add(And(P1, P2))           ## Γ
solver.check()                    ## => sat
solver.model()                    ## => [P1 = True, P2 = True]

solver.add(Not(Implies(P1, P2))) ## ¬α
solver.check()                    ## => unsat: Γ ⊨ α
```

Take-away: Inference is reducible to satisfiability testing.

So far: the truth-table method

what: satisfiability, validity, entailment are **semantic**: what to check, not how

how: **enumerate all worlds**, but each new atom **doubles** them

Time needed, if a computer checks a billion worlds per second

atoms n	worlds 2^n	time
3	8	instant: our running example

So far: the truth-table method

what: satisfiability, validity, entailment are **semantic**: what to check, not how

how: **enumerate all worlds**, but each new atom **doubles** them

Time needed, if a computer checks a billion worlds per second

atoms n	worlds 2^n	time
3	8	instant: our running example
20	$\approx 10^6$	a millisecond

So far: the truth-table method

what: satisfiability, validity, entailment are **semantic**: what to check, not how

how: **enumerate all worlds**, but each new atom **doubles** them

Time needed, if a computer checks a billion worlds per second

atoms n	worlds 2^n	time
3	8	instant: our running example
20	$\approx 10^6$	a millisecond
30	$\approx 10^9$	about a second

So far: the truth-table method

what: satisfiability, validity, entailment are **semantic**: what to check, not how

how: **enumerate all worlds**, but each new atom **doubles** them

Time needed, if a computer checks a billion worlds per second

atoms n	worlds 2^n	time
3	8	instant: our running example
20	$\approx 10^6$	a millisecond
30	$\approx 10^9$	about a second
50	$\approx 10^{15}$	about two weeks

So far: the truth-table method

what: satisfiability, validity, entailment are **semantic**: what to check, not how

how: **enumerate all worlds**, but each new atom **doubles** them

Time needed, if a computer checks a billion worlds per second

atoms n	worlds 2^n	time
3	8	instant: our running example
20	$\approx 10^6$	a millisecond
30	$\approx 10^9$	about a second
50	$\approx 10^{15}$	about two weeks
100	$\approx 10^{30}$	$\approx$ 3000 times the age of the universe

So far: the truth-table method

what: satisfiability, validity, entailment are **semantic**: what to check, not how

how: **enumerate all worlds**, but each new atom **doubles** them

Time needed, if a computer checks a billion worlds per second

atoms n	worlds 2^n	time
3	8	instant: our running example
20	$\approx 10^6$	a millisecond
30	$\approx 10^9$	about a second
50	$\approx 10^{15}$	about two weeks
100	$\approx 10^{30}$	$\approx$ 3000 times the age of the universe
10^6	2^{10^6}	industrial SAT instances

So far: the truth-table method

what: satisfiability, validity, entailment are **semantic**: what to check, not how

how: **enumerate all worlds**, but each new atom **doubles** them

Time needed, if a computer checks a billion worlds per second

atoms n	worlds 2^n	time
3	8	instant: our running example
20	$\approx 10^6$	a millisecond
30	$\approx 10^9$	about a second
50	$\approx 10^{15}$	about two weeks
100	$\approx 10^{30}$	$\approx$ 3000 times the age of the universe
10^6	2^{10^6}	industrial SAT instances

Truth tables always work, but they grow **exponentially**: real problems (chips, software, schedules) have thousands or millions of atoms.

So far: the truth-table method

what: satisfiability, validity, entailment are **semantic**: what to check, not how

how: **enumerate all worlds**, but each new atom **doubles** them

Time needed, if a computer checks a billion worlds per second

atoms n	worlds 2^n	time
3	8	instant: our running example
20	$\approx 10^6$	a blink
30	$\approx 10^9$	about a second
100	$\approx 10^{30}$	$\approx$ 3000 times the age of the universe
10^6	2^{10^6}	industrial SAT instances

$$2^{100} \text{ worlds}$$

$$1\,267\,650\,600\,228\,229\,401\,496\,703\,205\,376$$

> Truth tables always work, but they grow **exponentially**: real problems (chips, software, schedules) have thousands or millions of atoms.

So far: the truth-table method

what: satisfiability, validity, entailment are **semantic**: what to check, not how

how: **enumerate all worlds**, but each new atom **doubles** them

Time needed, if a computer checks a billion worlds per second

atoms n	worlds 2^n	time
3	8	instant: our running example
20	$\approx 10^6$	a millisecond
30	$\approx 10^9$	a second
50	$\approx 10^{15}$	about two weeks
100	$\approx 10^{30}$	$\approx$ 3000 times the age of the universe
10^6	2^{10^6}	industrial SAT instances

> Truth tables always work, but they grow **exponentially**: real problems (chips, software, schedules) have thousands or millions of atoms.

We need better algorithms: SAT

Interim summary

1. **Syntax: formulas are trees**

$$(P_1 \wedge \neg P_2) \rightarrow P_3$$

atoms joined by connectives

Interim summary

1. **Syntax: formulas are trees**

$$(P_1 \wedge \neg P_2) \rightarrow P_3$$

atoms joined by connectives

2. **Semantics: worlds**

$$v(P_1) = 1 \rightsquigarrow \overline{v}(P_1 \vee P_2) = 1$$

fix the atoms; go bottom-up

Interim summary

1. **Syntax: formulas are trees**

$$(P_1 \wedge \neg P_2) \rightarrow P_3$$

atoms joined by connectives

2. **Semantics: worlds**

$$v(P_1) = 1 \rightsquigarrow \overline{v}(P_1 \vee P_2) = 1$$

fix the atoms; go bottom-up

3. **Equivalence and laws**

$$P_1 \rightarrow P_2 \equiv \neg P_1 \vee P_2$$

same truth table; rewrite by laws

Interim summary

1. **Syntax: formulas are trees**

$(P_1 \wedge \neg P_2) \rightarrow P_3$

atoms joined by connectives

2. **Semantics: worlds**

$v(P_1) = 1 \rightsquigarrow \overline{v}(P_1 \vee P_2) = 1$

fix the atoms; go bottom-up

3. **Equivalence and laws**

$P_1 \rightarrow P_2 \equiv \neg P_1 \vee P_2$

same truth table; rewrite by laws

4. **Satisfiable and valid**

$P_1 \wedge \neg P_2$ sat. $P_1 \vee \neg P_1$ valid

has a model / every world is one

Interim summary

1. Syntax: formulas are trees

$(P_1 \wedge \neg P_2) \rightarrow P_3$

atoms joined by connectives

2. Semantics: worlds

$v(P_1) = 1 \;\rightsquigarrow\; \overline{v}(P_1 \vee P_2) = 1$

fix the atoms; go bottom-up

3. Equivalence and laws

$P_1 \rightarrow P_2 \;\equiv\; \neg P_1 \vee P_2$

same truth table; rewrite by laws

4. Satisfiable and valid

$P_1 \wedge \neg P_2$ sat. $\quad$ $P_1 \vee \neg P_1$ valid

has a model / every world is one

5. Entailment

$P_1 \wedge P_2 \models P_1 \rightarrow P_2$

every model of Γ is a model of α

Interim summary

1. Syntax: formulas are trees

$(P_1 \wedge \neg P_2) \rightarrow P_3$

atoms joined by connectives

2. Semantics: worlds

$v(P_1) = 1 \;\rightsquigarrow\; \overline{v}(P_1 \vee P_2) = 1$

fix the atoms; go bottom-up

3. Equivalence and laws

$P_1 \rightarrow P_2 \;\equiv\; \neg P_1 \vee P_2$

same truth table; rewrite by laws

4. Satisfiable and valid

$P_1 \wedge \neg P_2$ sat. $\quad$ $P_1 \vee \neg P_1$ valid

has a model / every world is one

5. Entailment

$P_1 \wedge P_2 \models P_1 \rightarrow P_2$

every model of Γ is a model of α

6. Everything reduces to sat

$\Gamma \models \alpha \iff \Gamma \cup \{\neg\alpha\}$ unsat.

truth tables: 2^n worlds, too slow

Interim summary

1. **Syntax: formulas are trees**
   $(P_1 \wedge \neg P_2) \rightarrow P_3$
   atoms joined by connectives

2. **Semantics: worlds**
   $v(P_1) = 1 \rightsquigarrow \overline{v}(P_1 \vee P_2) = 1$
   fix the atoms, go bottom-up

3. **Equivalence and laws**
   $P_1 \rightarrow P_2 \equiv \neg P_1 \vee P_2$
   same truth table; rewrite by laws

4. **Satisfiable and valid**
   $P_1 \wedge \neg P_2$ sat. $P_1 \vee \neg P_1$ valid
   has a model / every world is one

5. **Entailment**
   $P_1 \wedge P_2 \models P_1 \rightarrow P_2$
   every model of Γ is a model of α

6. **Everything reduces to sat**
   $\Gamma \models \alpha \iff \Gamma \cup \{\neg\alpha\}$ unsat.
   truth tables: 2^n worlds, too slow

Questions?

Boolean Satisfiability (SAT)

Computation: the SAT problem

$F := \ldots$	$\overline{v}(F)$	$\Gamma \models \alpha$	SAT
How do we **write** formulas?	When is a formula **true**?	What **follows** from what?	How do we **compute** it?

The last question: deciding satisfiability *without* enumerating all 2^n worlds.

Computation: the SAT problem

The SAT problem

input: a formula F

output: a model of F, or UNSAT if it has none

Computation: the SAT problem

The SAT problem

input: a formula F

output: a model of F, or $\textsc{unsat}$ if it has none

All of inference

$\Gamma \models \alpha$, validity, ...

all reduce to $\textsc{sat}$

our theorem

Computation: the SAT problem

The SAT problem

input: a formula F

output: a model of F, or $\textsc{unsat}$ if it has none

All of inference

$\Gamma \models \alpha$, validity, ...

all reduce to $\textsc{sat}$

our theorem

Hard in theory

the first problem shown

NP-complete

Cook, 1971

Computation: the SAT problem

The SAT problem

input: a formula F

output: a model of F, or $\textsc{unsat}$ if it has none

All of inference

$\Gamma \models \alpha$, validity, ...

all reduce to $\textsc{sat}$

our theorem

Hard in theory

the first problem shown

NP-complete

Cook, 1971

SAT solvers

solvers handle

millions of atoms

chips, software, schedules

Computation: the SAT problem

The SAT problem

input: a formula F

output: a model of F, or $\textsc{unsat}$ if it has none

All of inference

$\Gamma \models \alpha$, validity, ...

all reduce to $\textsc{sat}$

our theorem

Hard in theory

the first problem shown

NP-complete

Cook, 1971

SAT solvers

solvers handle

millions of atoms

chips, software, schedules

Plan: normal forms $\rightarrow$ conditioning $\rightarrow$ the **DPLL** algorithm

Normal forms: formulas in a fixed shape

Definitions

literal an atom or its negation: P_1, $\neg P_1$

all formulas

$\neg(P_1 \to P_2)$

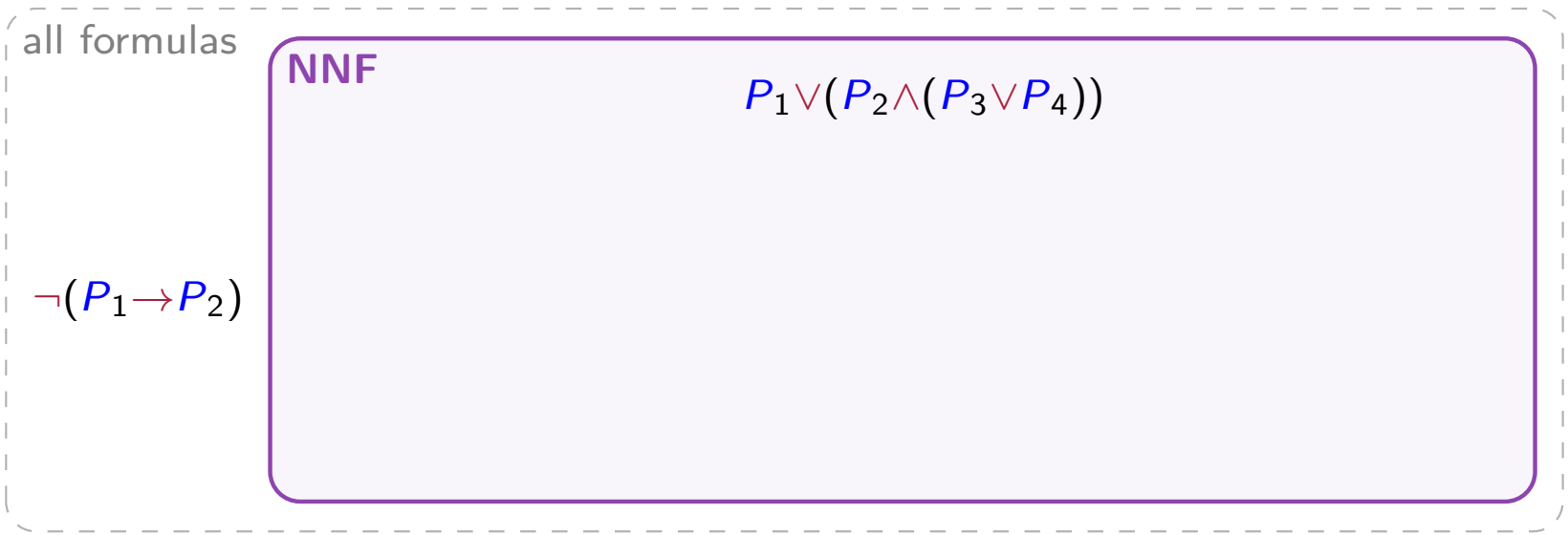
Literals are the building blocks of all three forms.

Normal forms: formulas in a fixed shape

Definitions

literal an atom or its negation: P_1, $\neg P_1$

NNF $\neg$ only on atoms, otherwise only $\wedge, \vee$



NNF: the laws push every $\neg$ down to the atoms.

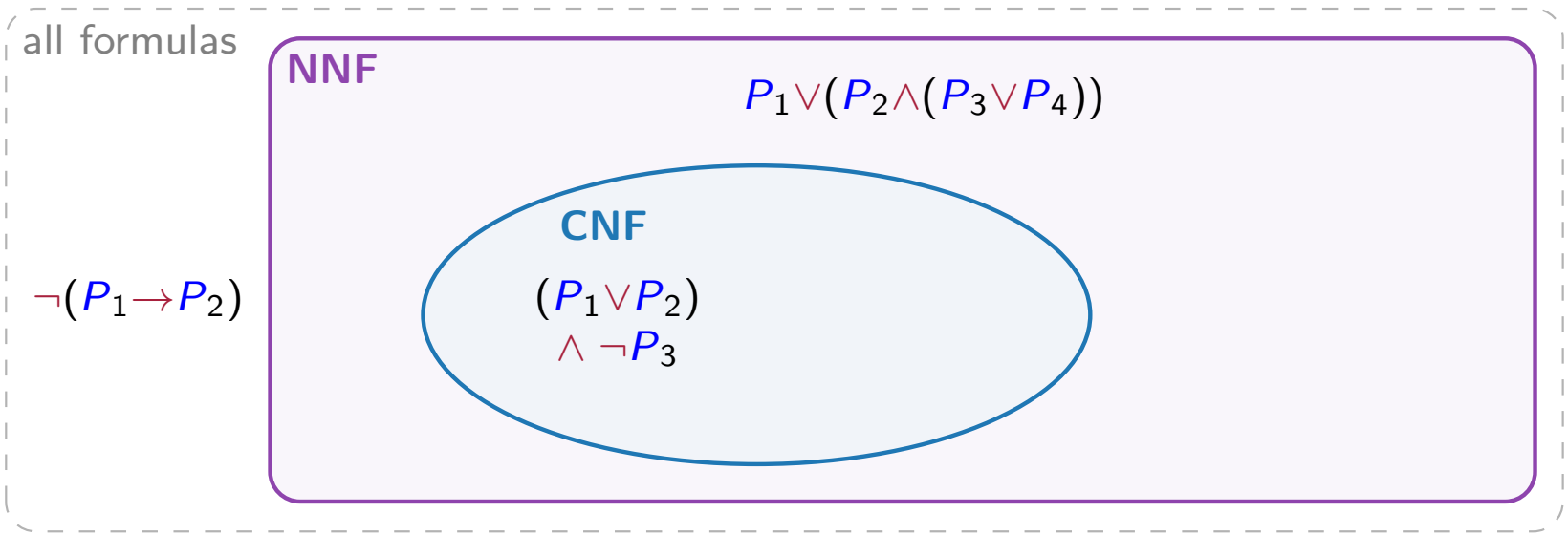
Normal forms: formulas in a fixed shape

Definitions

literal an atom or its negation: P_1, $\neg P_1$

NNF $\neg$ only on atoms, otherwise only $\wedge, \vee$

CNF an $\wedge$ of **clauses**, clause $=$ an $\vee$ of literals



CNF: a list of constraints that *all* have to hold.

Normal forms: formulas in a fixed shape

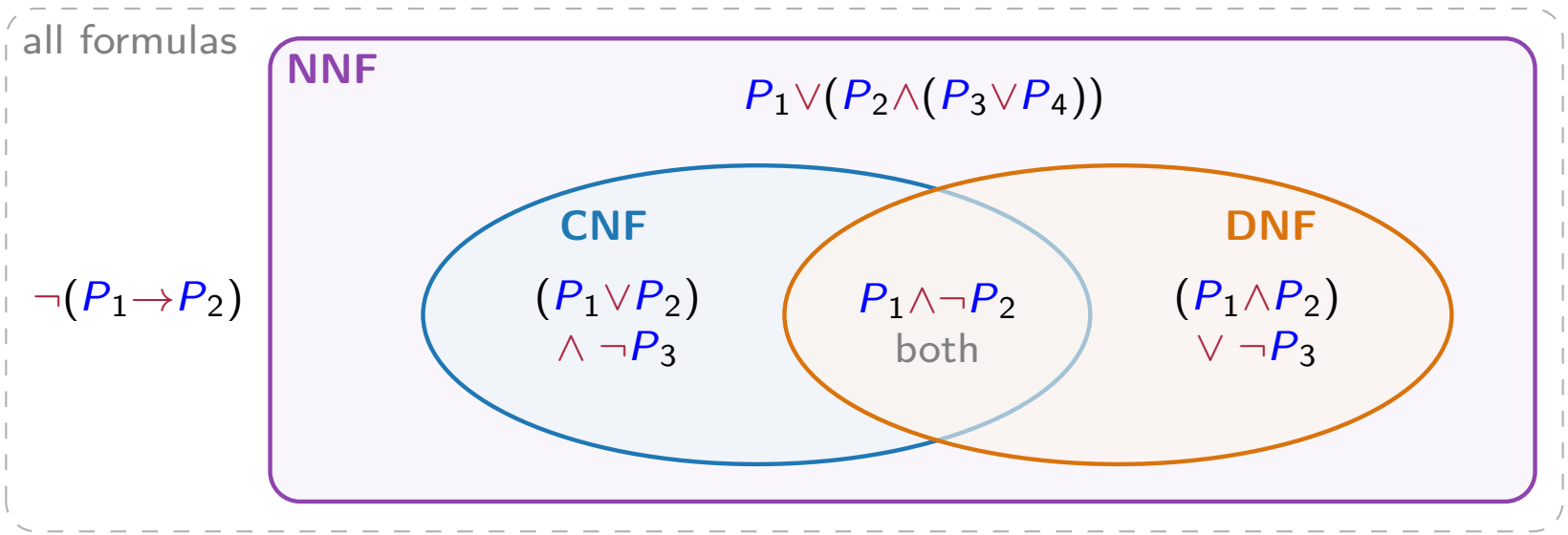
Definitions

literal an atom or its negation: P_1, $\neg P_1$

NNF $\neg$ only on atoms, otherwise only $\wedge, \vee$

CNF an $\wedge$ of **clauses**, clause = an $\vee$ of literals

DNF an $\vee$ of **terms**, term = an $\wedge$ of literals



DNF: a list of cases, *one* of which has to hold.

Normal forms: formulas in a fixed shape

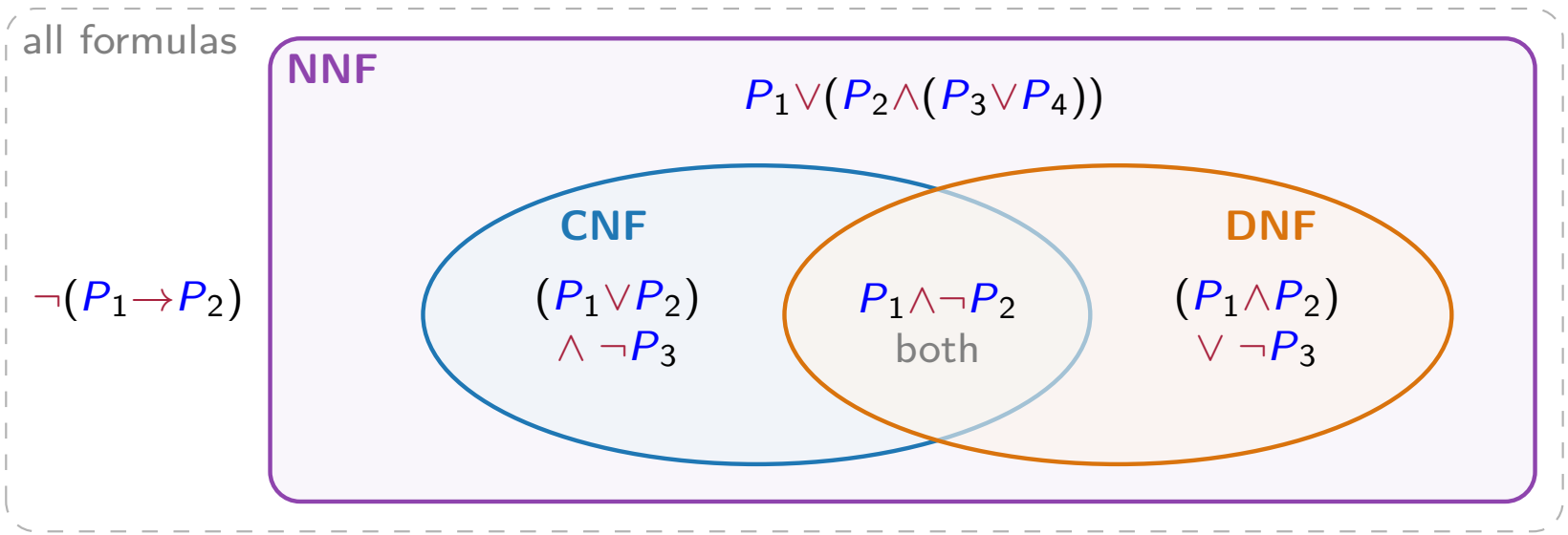
Definitions

literal an atom or its negation: P_1, $\neg P_1$

NNF $\neg$ only on atoms, otherwise only $\wedge, \vee$

CNF an $\wedge$ of **clauses**, clause = an $\vee$ of literals

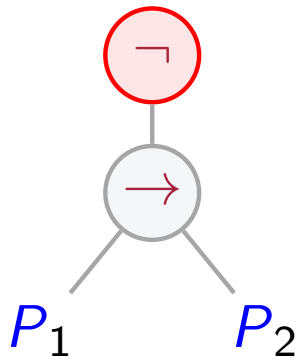
DNF an $\vee$ of **terms**, term = an $\wedge$ of literals



Every formula has an equivalent formula in NNF, in CNF and in DNF: **no expressive power is lost**.

Getting to NNF: push negations down to the leaves

$$\neg(P_1 \rightarrow P_2) \equiv P_1 \wedge \neg P_2$$

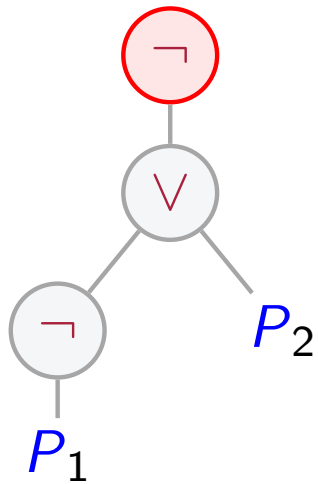


formula	law
$\neg(P_1 \rightarrow P_2)$	

Not NNF: a $\neg$ sits above a whole subformula, and $\rightarrow$ is used.

Getting to NNF: push negations down to the leaves

$$\neg(P_1 \rightarrow P_2) \equiv P_1 \wedge \neg P_2$$

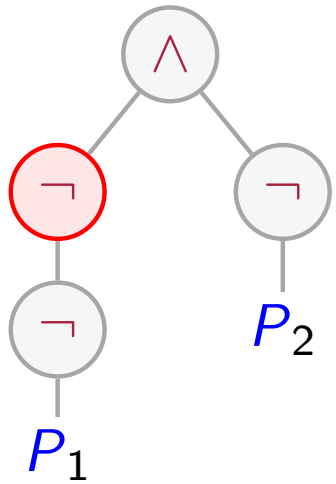


formula	law
$\neg(P_1 \rightarrow P_2)$	
$\equiv \neg(\neg P_1 \vee P_2)$	remove $\rightarrow$

Step 1: rewrite $\rightarrow$ (and $\leftrightarrow$) with $\neg, \wedge, \vee$.

Getting to NNF: push negations down to the leaves

$$\neg(P_1 \rightarrow P_2) \equiv P_1 \wedge \neg P_2$$

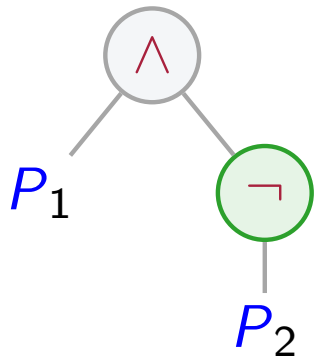


formula		law
	$\neg(P_1 \rightarrow P_2)$	
$\equiv$	$\neg(\neg P_1 \vee P_2)$	remove $\rightarrow$
$\equiv$	$\neg\neg P_1 \wedge \neg P_2$	De Morgan

Step 2: De Morgan moves the $\neg$ one level down, and flips $\vee$ into $\wedge$.

Getting to NNF: push negations down to the leaves

$$\neg(P_1 \rightarrow P_2) \equiv P_1 \wedge \neg P_2$$



formula		law
	$\neg(P_1 \rightarrow P_2)$	
$\equiv$	$\neg(\neg P_1 \vee P_2)$	remove $\rightarrow$
$\equiv$	$\neg\neg P_1 \wedge \neg P_2$	De Morgan
$\equiv$	$P_1 \wedge \neg P_2$	double negation

Step 3: drop double negations. NNF: every $\neg$ sits directly on an atom.

CNF and DNF: two shapes for the same formula

The general shapes

CNF $\bigwedge_{j=1}^{m} \left(\bigvee_{i=1}^{k_j} l_i^j \right)$ an $\wedge$ of m clauses, each an $\vee$ of k_j literals

DNF $\bigvee_{j=1}^{m} \left(\bigwedge_{i=1}^{k_j} l_i^j \right)$ an $\vee$ of m terms, each an $\wedge$ of k_j literals

CNF and DNF: two shapes for the same formula

The general shapes

CNF $\bigwedge_{j=1}^{m} \left(\bigvee_{i=1}^{k_j} l_i^j \right)$ an $\wedge$ of m clauses, each an $\vee$ of k_j literals

DNF $\bigvee_{j=1}^{m} \left(\bigwedge_{i=1}^{k_j} l_i^j \right)$ an $\vee$ of m terms, each an $\wedge$ of k_j literals

Example: $F = P_1 \rightarrow (P_2 \wedge P_3)$

CNF of F

$$\underbrace{(\neg P_1 \vee P_2)}_{\text{clause}} \wedge \underbrace{(\neg P_1 \vee P_3)}_{\text{clause}}$$

DNF of F

$$\underbrace{\neg P_1}_{\text{term}} \vee \underbrace{(P_2 \wedge P_3)}_{\text{term}}$$

CNF and DNF: two shapes for the same formula

The general shapes

CNF $\bigwedge_{j=1}^{m} \left(\bigvee_{i=1}^{k_j} l_i^j \right)$ an $\wedge$ of m clauses, each an $\vee$ of k_j literals

DNF $\bigvee_{j=1}^{m} \left(\bigwedge_{i=1}^{k_j} l_i^j \right)$ an $\vee$ of m terms, each an $\wedge$ of k_j literals

Example: $F = P_1 \rightarrow (P_2 \wedge P_3)$

CNF of F

$$\underbrace{(\neg P_1 \vee P_2)}_{\text{clause}} \wedge \underbrace{(\neg P_1 \vee P_3)}_{\text{clause}}$$

DNF of F

$$\underbrace{\neg P_1}_{\text{term}} \vee \underbrace{(P_2 \wedge P_3)}_{\text{term}}$$

atoms			formulas		
P_1	P_2	P_3	F	CNF	DNF
0	0	0	1	1	1
0	0	1	1	1	1
0	1	0	1	1	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	0	0	0
1	1	1	1	1	1

CNF and DNF: two shapes for the same formula

The general shapes

CNF $\bigwedge_{j=1}^{m} \left(\bigvee_{i=1}^{k_j} l_i^j \right)$ an $\wedge$ of m clauses, each an $\vee$ of k_j literals

DNF $\bigvee_{j=1}^{m} \left(\bigwedge_{i=1}^{k_j} l_i^j \right)$ an $\vee$ of m terms, each an $\wedge$ of k_j literals

Example: $F = P_1 \rightarrow (P_2 \wedge P_3)$

CNF of F

$$\underbrace{(\neg P_1 \vee P_2)}_{\text{clause}} \wedge \underbrace{(\neg P_1 \vee P_3)}_{\text{clause}}$$

DNF of F

$$\underbrace{\neg P_1}_{\text{term}} \vee \underbrace{(P_2 \wedge P_3)}_{\text{term}}$$

atoms			formulas		
P_1	P_2	P_3	F	CNF	DNF
0	0	0	1	1	1
0	0	1	1	1	1
0	1	0	1	1	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	0	0	0
1	1	1	1	1	1

CNF and DNF: two shapes for the same formula

The general shapes

CNF $\bigwedge_{j=1}^{m} \left(\bigvee_{i=1}^{k_j} l_i^j \right)$ an $\wedge$ of m clauses, each an $\vee$ of k_j literals

DNF $\bigvee_{j=1}^{m} \left(\bigwedge_{i=1}^{k_j} l_i^j \right)$ an $\vee$ of m terms, each an $\wedge$ of k_j literals

Example: $F = P_1 \rightarrow (P_2 \wedge P_3)$

CNF of F

$$\underbrace{(\neg P_1 \vee P_2)}_{\text{clause}} \wedge \underbrace{(\neg P_1 \vee P_3)}_{\text{clause}}$$

DNF of F

$$\underbrace{\neg P_1}_{\text{term}} \vee \underbrace{(P_2 \wedge P_3)}_{\text{term}}$$

atoms			formulas		
P_1	P_2	P_3	F	CNF	DNF
0	0	0	1	1	1
0	0	1	1	1	1
0	1	0	1	1	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	0	0	0
1	1	1	1	1	1

Why normal forms matter

Is a DNF satisfiable?

$$(P_1 \wedge \neg P_1) \vee (P_2 \wedge \neg P_3)$$

A term is satisfiable unless it contains a literal and its negation.

Why normal forms matter

Is a DNF satisfiable?

$$(P_1 \wedge \neg P_1) \vee (P_2 \wedge \neg P_3)$$

term 1 clashes, term 2 does not:

satisfiable ✓

A term is satisfiable unless it contains a literal and its negation.

Why normal forms matter

Is a DNF satisfiable?

$$(P_1 \wedge \neg P_1) \vee (P_2 \wedge \neg P_3)$$

term 1 clashes, term 2 does not:
satisfiable ✓

one pass: look for a term without both P and $\neg P$

A term is satisfiable unless it contains a literal and its negation.

Why normal forms matter

Is a DNF satisfiable?

$$(P_1 \wedge \neg P_1) \vee (P_2 \wedge \neg P_3)$$

term 1 clashes, term 2 does not:
satisfiable ✓

one pass: look for a term without both P and $\neg P$

Is a CNF valid?

$$(P_1 \vee \neg P_1) \wedge (P_2 \vee P_3)$$

A clause is valid only if it contains a literal and its negation.

Why normal forms matter

Is a DNF satisfiable?

$$(P_1 \wedge \neg P_1) \vee (P_2 \wedge \neg P_3)$$

term 1 clashes, term 2 does not:
satisfiable ✓

one pass: look for a term without both P and $\neg P$

Is a CNF valid?

$$(P_1 \vee \neg P_1) \wedge (P_2 \vee P_3)$$

clause 2 can be false: valid ✗

A clause is valid only if it contains a literal and its negation.

Why normal forms matter

Is a DNF satisfiable?

$$(P_1 \wedge \neg P_1) \vee (P_2 \wedge \neg P_3)$$

term 1 clashes, term 2 does not: satisfiable ✓

one pass: look for a term without both P and $\neg P$

Is a CNF valid?

$$(P_1 \vee \neg P_1) \wedge (P_2 \vee P_3)$$

clause 2 can be false: valid ✗

one pass: every clause needs both P and $\neg P$

A clause is valid only if it contains a literal and its negation.

Why normal forms matter

Is a DNF satisfiable?

$(P_1 \wedge \neg P_1) \vee (P_2 \wedge \neg P_3)$

term 1 clashes, term 2 does not: satisfiable ✓

one pass: look for a term without both P and $\neg P$

Is a CNF valid?

$(P_1 \vee \neg P_1) \wedge (P_2 \vee P_3)$

clause 2 can be false: valid ✗

one pass: every clause needs both P and $\neg P$

	satisfiable?	valid?
DNF	**easy**	hard
CNF	hard	**easy**

The same question can be easy or hard, depending on the **shape** of the formula, algorithms are often formulated based on a normal form.

Why normal forms matter

Is a DNF satisfiable?

$$(P_1 \wedge \neg P_1) \vee (P_2 \wedge \neg P_3)$$

term 1 clashes, term 2 does not: satisfiable ✓

one pass: look for a term without both P and $\neg P$

Is a CNF valid?

$$(P_1 \vee \neg P_1) \wedge (P_2 \vee P_3)$$

clause 2 can be false: not valid ✗

one pass: every clause needs both P and $\neg P$

So: convert to DNF, and SAT is easy?

	satisfiable?	valid?
DNF	**easy**	hard
CNF	hard	**easy**

The same question can be easy or hard, depending on the **shape** of the formula, algorithms are often formulated based on a normal form.

The catch: converting can blow up

$$(P_0^1 \vee P_1^1) \wedge (P_0^2 \vee P_1^2) \wedge \cdots \wedge (P_0^n \vee P_1^n)$$ a CNF with n clauses

Its DNF, for $n = 2$

$$(P_0^1 \wedge P_0^2) \vee (P_0^1 \wedge P_1^2)$$
$$\vee\ (P_1^1 \wedge P_0^2) \vee (P_1^1 \wedge P_1^2)$$

one term per way of picking
one literal from each clause

The catch: converting can blow up

$$(P_0^1 \vee P_1^1) \wedge (P_0^2 \vee P_1^2) \wedge \cdots \wedge (P_0^n \vee P_1^n)$$ a CNF with n clauses

Its DNF, for $n = 2$

$$(P_0^1 \wedge P_0^2) \vee (P_0^1 \wedge P_1^2)$$
$$\vee\ (P_1^1 \wedge P_0^2) \vee (P_1^1 \wedge P_1^2)$$

one term per way of picking one literal from each clause

n	CNF literals	DNF terms
2	4	4
3	6	8
10	20	1 024
30	60	$\approx 10^9$
n	$2n$	2^n

The catch: converting can blow up

$$(P_0^1 \vee P_1^1) \wedge (P_0^2 \vee P_1^2) \wedge \cdots \wedge (P_0^n \vee P_1^n)$$ a CNF with n clauses

Its DNF, for $n = 2$

$$(P_0^1 \wedge P_0^2) \vee (P_0^1 \wedge P_1^2)$$
$$\vee\ (P_1^1 \wedge P_0^2) \vee (P_1^1 \wedge P_1^2)$$

one term per way of picking one literal from each clause

n	CNF literals	DNF terms
2	4	4
3	6	8
10	20	1 024
30	60	$\approx 10^9$
n	$2n$	2^n

Converting to DNF can make a formula **exponentially** larger, and the easy check is lost again. SAT solvers therefore work directly on **CNF**.

The catch: converting can blow up

$(P_0^1 \vee P_1^1) \wedge (P_0^2 \vee P_1^2) \wedge \cdots \wedge (P_0^n \vee P_1^n)$ a CNF with n clauses

Watching the blowup in SymPy

```
## pip install sympy
from sympy import symbols, And, Or
from sympy.logic.boolalg import to_dnf

P0 = symbols("P0_1:5")   ## P0_1, ..., P0_4
P1 = symbols("P1_1:5")   ## P1_1, ..., P1_4

cnf = And(*[Or(a, b) for a, b in zip(P0, P1)])   ## 4 clauses
dnf = to_dnf(cnf)

len(cnf.args), len(dnf.args)   ## => (4, 16)
```

CNF formulas as sets of clauses

$$(P_1 \vee P_2) \wedge (P_1 \vee \neg P_2 \vee \neg P_3) \wedge (\neg P_1 \vee P_2 \vee \neg P_3)$$

As sets, the order of clauses and literals and any repeats no longer matter.

CNF formulas as sets of clauses

$$(P_1 \vee P_2) \wedge (P_1 \vee \neg P_2 \vee \neg P_3) \wedge (\neg P_1 \vee P_2 \vee \neg P_3)$$

$$\Delta = \{\ \{P_1, P_2\},\ \ \{P_1, \neg P_2, \neg P_3\},\ \ \{\neg P_1, P_2, \neg P_3\}\ \}$$

As sets, the order of clauses and literals and any repeats no longer matter.

CNF formulas as sets of clauses

$$(P_1 \vee P_2) \wedge (P_1 \vee \neg P_2 \vee \neg P_3) \wedge (\neg P_1 \vee P_2 \vee \neg P_3)$$

$$\Delta = \{\ \{P_1, P_2\},\ \ \{P_1, \neg P_2, \neg P_3\},\ \ \{\neg P_1, P_2, \neg P_3\}\ \}$$

A clause is a set of literals

$$C = \{P_1, \neg P_2, \neg P_3\}$$

read as: $P_1 \vee \neg P_2 \vee \neg P_3$

As sets, the order of clauses and literals and any repeats no longer matter.

CNF formulas as sets of clauses

$$(P_1 \vee P_2) \wedge (P_1 \vee \neg P_2 \vee \neg P_3) \wedge (\neg P_1 \vee P_2 \vee \neg P_3)$$

$$\Delta = \{\ \{P_1, P_2\},\ \ \{P_1, \neg P_2, \neg P_3\},\ \ \{\neg P_1, P_2, \neg P_3\}\ \}$$

A clause is a set of literals

$C = \{P_1, \neg P_2, \neg P_3\}$

read as: $P_1 \vee \neg P_2 \vee \neg P_3$

A CNF is a set of clauses

$\Delta = \{C_1, C_2, C_3\}$

read as: $C_1 \wedge C_2 \wedge C_3$

As sets, the order of clauses and literals and any repeats no longer matter.

CNF formulas as sets of clauses

$$(P_1 \vee P_2) \wedge (P_1 \vee \neg P_2 \vee \neg P_3) \wedge (\neg P_1 \vee P_2 \vee \neg P_3)$$

$$\Delta = \{\ \{P_1, P_2\},\ \ \{P_1, \neg P_2, \neg P_3\},\ \ \{\neg P_1, P_2, \neg P_3\}\ \}$$

A clause is a set of literals

$C = \{P_1, \neg P_2, \neg P_3\}$

read as: $P_1 \vee \neg P_2 \vee \neg P_3$

A CNF is a set of clauses

$\Delta = \{C_1, C_2, C_3\}$

read as: $C_1 \wedge C_2 \wedge C_3$

The empty clause $\{\}$

no literal left to make it true

$\{\} \in \Delta$: Δ is **unsatisfiable** ✗

An empty $\vee$ is false, an empty $\wedge$ is true: the two places where a search can stop.

CNF formulas as sets of clauses

$$(P_1 \vee P_2) \wedge (P_1 \vee \neg P_2 \vee \neg P_3) \wedge (\neg P_1 \vee P_2 \vee \neg P_3)$$

$$\Delta = \{\ \{P_1, P_2\},\ \ \{P_1, \neg P_2, \neg P_3\},\ \ \{\neg P_1, P_2, \neg P_3\}\ \}$$

A clause is a set of literals

$C = \{P_1, \neg P_2, \neg P_3\}$

read as: $P_1 \vee \neg P_2 \vee \neg P_3$

A CNF is a set of clauses

$\Delta = \{C_1, C_2, C_3\}$

read as: $C_1 \wedge C_2 \wedge C_3$

The empty clause $\{\}$

no literal left to make it true

$\{\} \in \Delta$: Δ is **unsatisfiable** ✗

The empty CNF $\Delta = \{\}$

no constraint left to violate

$\Delta = \{\}$: Δ is **satisfied** ✓

An empty $\vee$ is false, an empty $\wedge$ is true: the two places where a search can stop.

Conditioning: assume a literal is true

Conditioning Δ on a literal l

$$\Delta|l \;=\; \left\{\, C - \{\neg l\} \;\middle|\; C \in \Delta,\;\; l \notin C \,\right\}$$

clauses with l become 1: drop them; $\neg l$ becomes 0: remove it

Conditioning: assume a literal is true

Conditioning Δ on a literal l

$$\Delta|l \;=\; \left\{\, C - \{\neg l\} \;\middle|\; C \in \Delta,\;\; l \notin C \,\right\}$$

clauses with l become 1: drop them; $\neg l$ becomes 0: remove it

$\Delta = \big\{\{P_1, P_2\},\ \{P_1, \neg P_2, \neg P_3\},\ \{\neg P_1, P_2, \neg P_3\}\big\}$ a set of three clauses

clause $C \in \Delta$	plug in $P_1 = 1$
$\{P_1, P_2\}$	$1 \vee P_2 \;=\; 1$
$\{P_1, \neg P_2, \neg P_3\}$	$1 \vee \neg P_2 \vee \neg P_3 \;=\; 1$
$\{\neg P_1, P_2, \neg P_3\}$	$0 \vee P_2 \vee \neg P_3$
result	$\Delta\|P_1 = \{\{P_2, \neg P_3\}\}$

$P_1 = 1$: clauses with P_1 become 1 and always hold, so they are dropped; the $\neg P_1$ in the last one is 0, so it goes.

Conditioning: assume a literal is true

Conditioning Δ on a literal l

$$\Delta|l \;=\; \left\{\, C - \{\neg l\} \;\middle|\; C \in \Delta,\;\; l \notin C \,\right\}$$

clauses with l become 1: drop them; $\neg l$ becomes 0: remove it

$\Delta = \big\{\{P_1, P_2\},\ \{P_1, \neg P_2, \neg P_3\},\ \{\neg P_1, P_2, \neg P_3\}\big\}$ a set of three clauses

clause $C \in \Delta$	plug in $P_1 = 1$	plug in $P_1 = 0$
$\{P_1, P_2\}$	$1 \vee P_2 \;=\; 1$	$0 \vee P_2$
$\{P_1, \neg P_2, \neg P_3\}$	$1 \vee \neg P_2 \vee \neg P_3 \;=\; 1$	$0 \vee \neg P_2 \vee \neg P_3$
$\{\neg P_1, P_2, \neg P_3\}$	$0 \vee P_2 \vee \neg P_3$	$1 \vee P_2 \vee \neg P_3 \;=\; 1$
result	$\Delta\|P_1 = \{\{P_2, \neg P_3\}\}$	$\Delta\|\neg P_1 = \{\{P_2\}, \{\neg P_2, \neg P_3\}\}$

$P_1 = 0$: the opposite. Now $0 \vee P_2$ shrinks to the clause $\{P_2\}$.

Conditioning: assume a literal is true

Conditioning Δ on a literal l

$$\Delta|l \;=\; \left\{\, C - \{\neg l\} \;\middle|\; C \in \Delta,\;\; l \notin C \,\right\}$$

clauses with l become 1: drop them; $\neg l$ becomes 0: remove it

$\Delta = \big\{\{P_1, P_2\},\ \{P_1, \neg P_2, \neg P_3\},\ \{\neg P_1, P_2, \neg P_3\}\big\}$ a set of three clauses

clause $C \in \Delta$	plug in $P_1 = 1$	plug in $P_1 = 0$
$\{P_1, P_2\}$	$1 \vee P_2 \;=\; 1$	$0 \vee P_2$
$\{P_1, \neg P_2, \neg P_3\}$	$1 \vee \neg P_2 \vee \neg P_3 \;=\; 1$	$0 \vee \neg P_2 \vee \neg P_3$
$\{\neg P_1, P_2, \neg P_3\}$	$0 \vee P_2 \vee \neg P_3$	$1 \vee P_2 \vee \neg P_3 \;=\; 1$
result	$\Delta\|P_1 = \{\{P_2, \neg P_3\}\}$	$\Delta\|\neg P_1 = \{\{P_2\}, \{\neg P_2, \neg P_3\}\}$

Either way, **the atom P_1 is gone**: each conditioning step leaves a smaller problem.

Case analysis: the splitting rule

Theorem (splitting rule)

Δ is satisfiable $\iff$ $\Delta|_{l}$ is satisfiable **or** $\Delta|_{\neg l}$ is satisfiable

every model makes l either true or false, i.e., $\Delta \equiv (l \wedge \Delta_{l}) \vee (\neg l \wedge \Delta_{\neg l})$

Δ

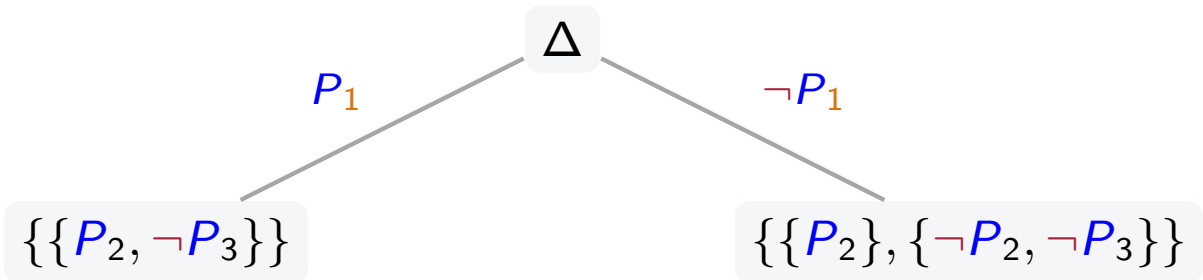
Pick any literal, and split the problem into two cases.

Case analysis: the splitting rule

Theorem (splitting rule)

Δ is satisfiable $\iff$ $\Delta|_l$ is satisfiable **or** $\Delta|_{\neg l}$ is satisfiable

every model makes l either true or false, i.e., $\Delta \equiv (l \wedge \Delta_l) \vee (\neg l \wedge \Delta_{\neg l})$



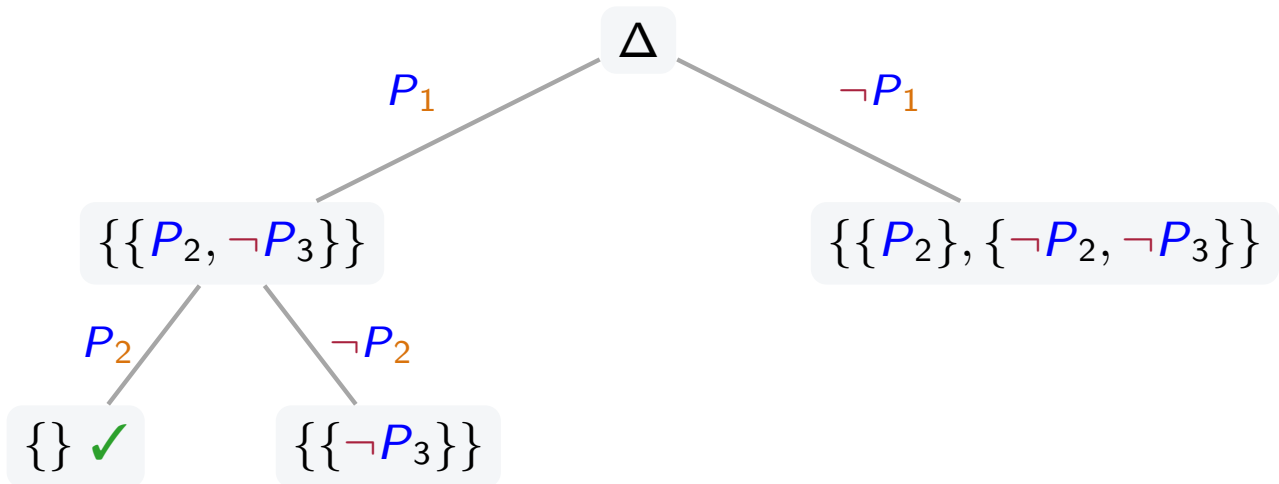
Each case has **one atom fewer**. Splitting all the way down is the truth table again...

Case analysis: the splitting rule

Theorem (splitting rule)

Δ is satisfiable $\iff$ $\Delta|_l$ is satisfiable **or** $\Delta|_{\neg l}$ is satisfiable

every model makes l either true or false, i.e., $\Delta \equiv (l \wedge \Delta_l) \vee (\neg l \wedge \Delta_{\neg l})$



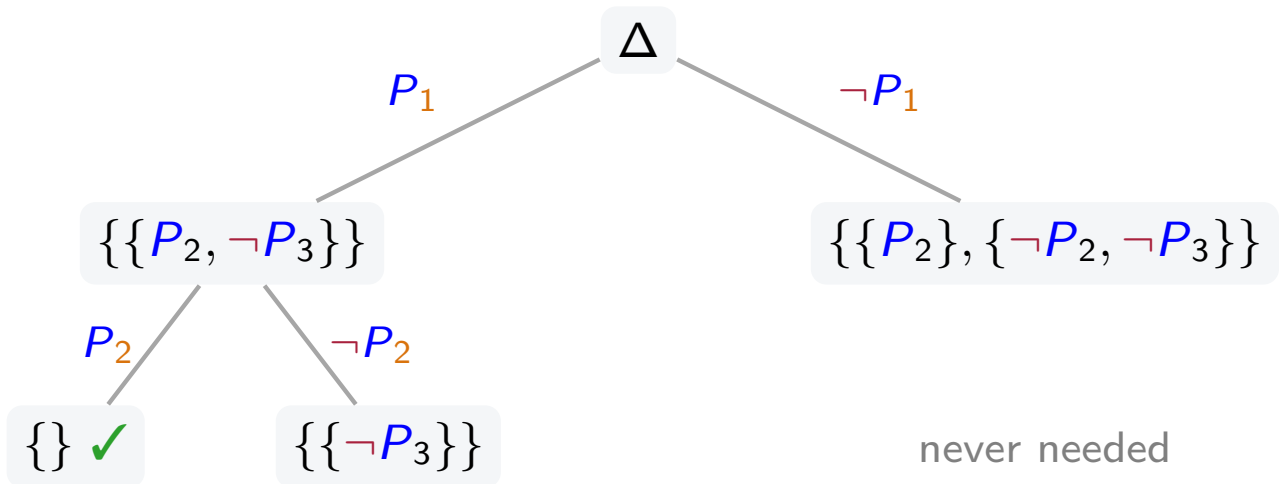
. . . but we can **stop** as soon as one branch reaches $\{\}$: Δ is satisfiable.

Case analysis: the splitting rule

Theorem (splitting rule)

Δ is satisfiable $\iff$ $\Delta|_l$ is satisfiable **or** $\Delta|_{\neg l}$ is satisfiable

every model makes l either true or false, i.e., $\Delta \equiv (l \wedge \Delta_l) \vee (\neg l \wedge \Delta_{\neg l})$



Model, read off the path: $P_1 = 1$, $P_2 = 1$ (and P_3 either way). The other half of the search is skipped.

No guessing needed: the unit rule

Corollary (unit rule)

for a **unit clause** $\{l\} \in \Delta$: $\quad \Delta$ is satisfiable $\iff$ $\Delta|l$ is satisfiable

the other case is dead: $\Delta|\neg l$ contains the empty clause

A clause with a single literal leaves no choice: that literal has to be true.

No guessing needed: the unit rule

Corollary (unit rule)

for a **unit clause** $\{l\} \in \Delta$: Δ is satisfiable $\iff$ $\Delta|l$ is satisfiable

the other case is dead: $\Delta|\neg l$ contains the empty clause

Propagation finds a model

$$\{\{P_2\}, \{\neg P_2, \neg P_3\}\}$$

Our case $\Delta|\neg P_1$ from before: each step creates the next unit clause.

No guessing needed: the unit rule

Corollary (unit rule)

for a **unit clause** $\{l\} \in \Delta$: Δ is satisfiable $\iff$ $\Delta|l$ is satisfiable

the other case is dead: $\Delta|\neg l$ contains the empty clause

Propagation finds a model

$$\{\{P_2\}, \{\neg P_2, \neg P_3\}\}$$
$$|\, P_2 \quad \{\{\neg P_3\}\}$$

Our case $\Delta|\neg P_1$ from before: each step creates the next unit clause.

No guessing needed: the unit rule

Corollary (unit rule)

for a **unit clause** $\{l\} \in \Delta$: Δ is satisfiable $\iff$ $\Delta|l$ is satisfiable

the other case is dead: $\Delta|\neg l$ contains the empty clause

Propagation finds a model

$\{\{P_2\}, \{\neg P_2, \neg P_3\}\}$

$|\ P_2$ $\{\{\neg P_3\}\}$

$|\ \neg P_3$ $\{\}$ satisfiable ✓

Our case $\Delta|\neg P_1$ from before: each step creates the next unit clause.

No guessing needed: the unit rule

Corollary (unit rule)

for a **unit clause** $\{l\} \in \Delta$: Δ is satisfiable $\iff$ $\Delta|l$ is satisfiable

the other case is dead: $\Delta|\neg l$ contains the empty clause

Propagation finds a model

$\{\{P_2\}, \{\neg P_2, \neg P_3\}\}$

$|\ P_2$ $\{\{\neg P_3\}\}$

$|\ \neg P_3$ $\{\}$ satisfiable ✓

Propagation finds a conflict

$\{\{P_1\}, \{\neg P_1, P_2\}, \{\neg P_2\}\}$

An empty clause appears: no model exists, and no case split was needed.

No guessing needed: the unit rule

Corollary (unit rule)

for a **unit clause** $\{l\} \in \Delta$: Δ is satisfiable $\iff$ $\Delta|l$ is satisfiable

the other case is dead: $\Delta|\neg l$ contains the empty clause

Propagation finds a model

$\{\{P_2\}, \{\neg P_2, \neg P_3\}\}$

$|\ P_2 \quad \{\{\neg P_3\}\}$

$|\ \neg P_3 \quad \{\}$ satisfiable ✓

Propagation finds a conflict

$\{\{P_1\}, \{\neg P_1, P_2\}, \{\neg P_2\}\}$

$|\ P_1 \quad \{\{P_2\}, \{\neg P_2\}\}$

An empty clause appears: no model exists, and no case split was needed.

No guessing needed: the unit rule

Corollary (unit rule)

for a **unit clause** $\{l\} \in \Delta$: Δ is satisfiable $\iff$ $\Delta|l$ is satisfiable

the other case is dead: $\Delta|\neg l$ contains the empty clause

Propagation finds a model

$\{\{P_2\}, \{\neg P_2, \neg P_3\}\}$

$|\ P_2$ $\{\{\neg P_3\}\}$

$|\ \neg P_3$ $\{\}$ satisfiable ✓

Propagation finds a conflict

$\{\{P_1\}, \{\neg P_1, P_2\}, \{\neg P_2\}\}$

$|\ P_1$ $\{\{P_2\}, \{\neg P_2\}\}$

$|\ P_2$ $\{\{\}\}$ unsatisfiable ✗

An empty clause appears: no model exists, and no case split was needed.

No guessing needed: the unit rule

Corollary (unit rule)

for a **unit clause** $\{l\} \in \Delta$: Δ is satisfiable $\iff$ $\Delta|l$ is satisfiable

the other case is dead: $\Delta|\neg l$ contains the empty clause

Propagation finds a model

$\{\{P_2\}, \{\neg P_2, \neg P_3\}\}$

$|\ P_2$ $\{\{\neg P_3\}\}$

$|\ \neg P_3$ $\{\}$ satisfiable ✓

Propagation finds a conflict

$\{\{P_1\}, \{\neg P_1, P_2\}, \{\neg P_2\}\}$

$|\ P_1$ $\{\{P_2\}, \{\neg P_2\}\}$

$|\ P_2$ $\{\{\}\}$ unsatisfiable ✗

Unit propagation: apply the unit rule until no unit clause is left. No case split, so it is cheap.

For Horn formulas (at most one positive literal per clause), propagation alone decides SAT.

Putting it together: the DPLL algorithm

Algorithm

```
1  function DPLL(Δ)
2
3
4
5      select a literal l in Δ
6      return DPLL(Δ|l)∨      splitting
                DPLL(Δ|¬l)    rule
```

The splitting rule gives the **recursion**...

Putting it together: the DPLL algorithm

Algorithm

```
1  function DPLL(Δ)
2
3      if {} ∈ Δ: return 0        empty
4      if Δ = {}: return 1        sets
5      select a literal l in Δ
6      return DPLL(Δ|l) ∨         splitting
                 DPLL(Δ|¬l)       rule
```

. . . the two empty sets tell us when to **stop**. . .

Putting it together: the DPLL algorithm

Algorithm

```
1  function DPLL(Δ)
2      Δ ← UnitProp(Δ)                unit rule
3      if {} ∈ Δ: return 0            empty
4      if Δ = {}: return 1            sets
5      select a literal l in Δ
6      return DPLL(Δ|l) ∨             splitting
                  DPLL(Δ|¬l)          rule
```

Unit propagation

```
function UnitProp(Δ)
    while {} ∉ Δ and
        some {l} ∈ Δ:
        Δ ← Δ|l
    return Δ
```

. . . and the unit rule **shortcuts** every case it can. The ∨ only runs its second call if the first one returns 0.

Putting it together: the DPLL algorithm

Algorithm

```
1  function DPLL(Δ)
2      Δ ← UnitProp(Δ)                 unit rule
3      if {} ∈ Δ: return 0             empty
4      if Δ = {}: return 1             sets
5      select a literal l in Δ
6      return DPLL(Δ|l) ∨              splitting
                  DPLL(Δ|¬l)           rule
```

Unit propagation

```
function UnitProp(Δ)
    while {} ∉ Δ and
        some {l} ∈ Δ:
        Δ ← Δ|l
    return Δ
```

Correctness: $\text{DPLL}(\Delta)$ returns 1 $\iff$ Δ is satisfiable. Davis, Putnam, Logemann and Loveland, 1962

Worst case: still exponentially many calls. In practice: far fewer.

DPLL in action

$$\Delta = \big\{\{\neg P_1, P_2\},\ \{\neg P_1, \neg P_2\},\ \{P_1, P_3\}\big\}$$

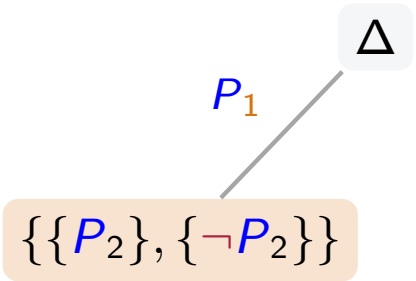
Δ

call	what happens
$\text{DPLL}(\Delta)$	no unit clause: select P_1

No unit clause, so DPLL has to guess: try $P_1 = 1$ first.

DPLL in action

$$\Delta = \big\{\{\neg P_1, P_2\},\ \{\neg P_1, \neg P_2\},\ \{P_1, P_3\}\big\}$$

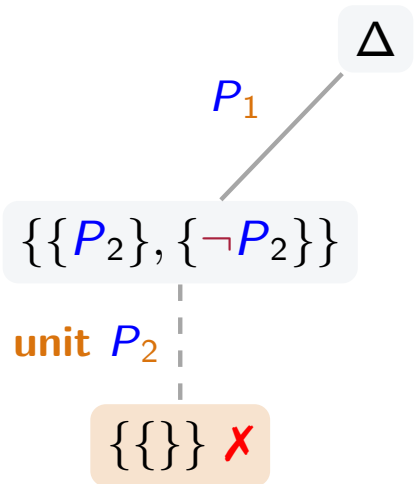


call	what happens
$\text{DPLL}(\Delta)$	no unit clause: select P_1
$\text{DPLL}(\Delta \mid P_1)$	unit clause $\{P_2\}$

Conditioning on P_1 creates two unit clauses that contradict each other...

DPLL in action

$$\Delta = \{\{\neg P_1, P_2\},\ \{\neg P_1, \neg P_2\},\ \{P_1, P_3\}\}$$

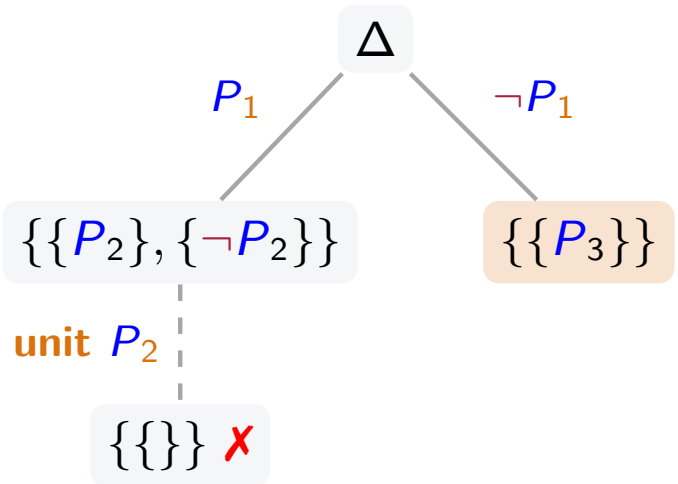


call	what happens
$\text{DPLL}(\Delta)$	no unit clause: select P_1
$\text{DPLL}(\Delta \mid P_1)$	unit clause $\{P_2\}$
	empty clause: **return** 0

. . . so propagation hits the empty clause: **backtrack**.

DPLL in action

$$\Delta = \{\{\neg P_1, P_2\},\ \{\neg P_1, \neg P_2\},\ \{P_1, P_3\}\}$$

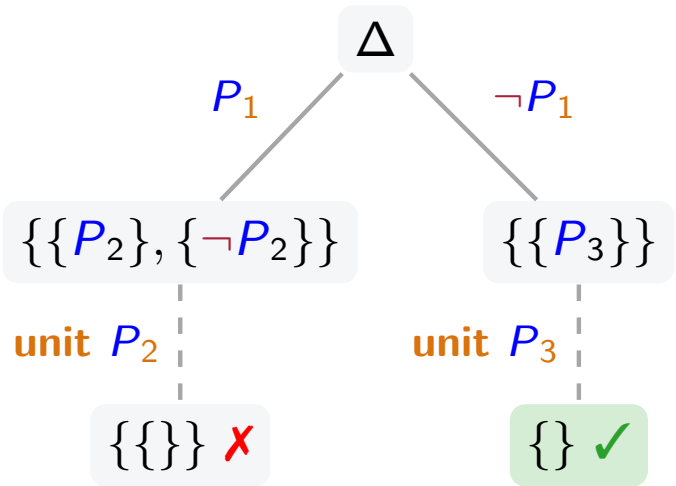


call	what happens
$\text{DPLL}(\Delta)$	no unit clause: select P_1
$\text{DPLL}(\Delta \mid P_1)$	unit clause $\{P_2\}$
	empty clause: **return** 0
$\text{DPLL}(\Delta \mid \neg P_1)$	unit clause $\{P_3\}$

Try $P_1 = 0$ instead: one unit clause, so no further guessing.

DPLL in action

$$\Delta = \{\{\neg P_1, P_2\},\ \{\neg P_1, \neg P_2\},\ \{P_1, P_3\}\}$$

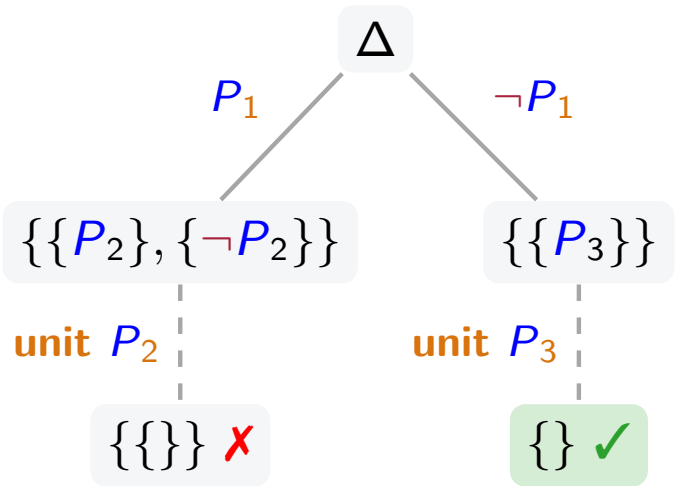


call	what happens
$\text{DPLL}(\Delta)$	no unit clause: select P_1
$\text{DPLL}(\Delta \mid P_1)$	unit clause $\{P_2\}$
	empty clause: **return** 0
$\text{DPLL}(\Delta \mid \neg P_1)$	unit clause $\{P_3\}$
	no clauses left: **return** 1

Satisfiable ✓ Model: $P_1 = 0,\ P_3 = 1$ (P_2 either way)

DPLL in action

$$\Delta = \{\{\neg P_1, P_2\},\ \{\neg P_1, \neg P_2\},\ \{P_1, P_3\}\}$$



call	what happens
$\text{DPLL}(\Delta)$	no unit clause: select P_1
$\text{DPLL}(\Delta \mid P_1)$	unit clause $\{P_2\}$
	empty clause: **return** 0
$\text{DPLL}(\Delta \mid \neg P_1)$	unit clause $\{P_3\}$
	no clauses left: **return** 1

One guess and two unit steps, instead of a truth table with **8 rows**.

Interim summary

1. **Normal forms**

$$\neg(P_1 \rightarrow P_2) \equiv P_1 \wedge \neg P_2$$

NNF: $\neg$ only on atoms

Interim summary

1. **Normal forms**

$$\neg(P_1 \rightarrow P_2) \equiv P_1 \wedge \neg P_2$$

NNF: $\neg$ only on atoms

2. **CNF and DNF**

$$(\neg P_1 \vee P_2) \wedge (\neg P_1 \vee P_3)$$

an $\wedge$ of clauses / an $\vee$ of terms

Interim summary

1. **Normal forms**

$$\neg(P_1 \rightarrow P_2) \equiv P_1 \wedge \neg P_2$$

NNF: $\neg$ only on atoms

2. **CNF and DNF**

$$(\neg P_1 \vee P_2) \wedge (\neg P_1 \vee P_3)$$

an $\wedge$ of clauses / an $\vee$ of terms

3. **CNF as sets**

$$\{\{\neg P_1, P_2\}, \{\neg P_1, P_3\}\}$$

$\{\} \in \Delta$: unsat; $\Delta = \{\}$: sat

Interim summary

1. Normal forms

$\neg(P_1 \rightarrow P_2) \equiv P_1 \wedge \neg P_2$

NNF: $\neg$ only on atoms

2. CNF and DNF

$(\neg P_1 \vee P_2) \wedge (\neg P_1 \vee P_3)$

an $\wedge$ of clauses / an $\vee$ of terms

3. CNF as sets

$\{\{\neg P_1, P_2\}, \{\neg P_1, P_3\}\}$

$\{\} \in \Delta$: unsat; $\Delta = \{\}$: sat

4. Conditioning

$\Delta|l$: assume l is true

one atom fewer

Interim summary

1. Normal forms

$\neg(P_1 \rightarrow P_2) \equiv P_1 \wedge \neg P_2$

NNF: $\neg$ only on atoms

2. CNF and DNF

$(\neg P_1 \vee P_2) \wedge (\neg P_1 \vee P_3)$

an $\wedge$ of clauses / an $\vee$ of terms

3. CNF as sets

$\{\{\neg P_1, P_2\}, \{\neg P_1, P_3\}\}$

$\{\} \in \Delta$: unsat; $\Delta = \{\}$: sat

4. Conditioning

$\Delta|l$: assume l is true

one atom fewer

5. Splitting rule

Δ sat $\iff$ $\Delta|l$ or $\Delta|\neg l$ sat

case analysis on one literal

Interim summary

1. Normal forms

$\neg(P_1 \to P_2) \equiv P_1 \wedge \neg P_2$

NNF: $\neg$ only on atoms

2. CNF and DNF

$(\neg P_1 \vee P_2) \wedge (\neg P_1 \vee P_3)$

an $\wedge$ of clauses / an $\vee$ of terms

3. CNF as sets

$\{\{\neg P_1, P_2\}, \{\neg P_1, P_3\}\}$

$\{\} \in \Delta$: unsat; $\Delta = \{\}$: sat

4. Conditioning

$\Delta|l$: assume l is true

one atom fewer

5. Splitting rule

Δ sat $\iff$ $\Delta|l$ or $\Delta|\neg l$ sat

case analysis on one literal

6. DPLL

splitting + unit propagation

a search tree that stops early

Quick check: true or false?

1. $\neg P_1 \vee (P_2 \wedge \neg P_3)$ is in NNF
2. $\neg(P_1 \wedge P_2)$ is in CNF
3. $P_1 \wedge \neg P_2$ is CNF *and* DNF
4. $\{\{P_1, \neg P_1\}\}$ is unsatisfiable
5. $\{\{\}\}$ and $\{\}$ mean the same
6. $\{\{P_1, P_2\}, \{\neg P_1\}\} | P_1 = \{\{\}\}$
7. Δ in 6 is solvable via unit prop.

Quick check: true or false?

1. $\neg P_1 \vee (P_2 \wedge \neg P_3)$ is in NNF — **true** — $\neg$ only on atoms
2. $\neg(P_1 \wedge P_2)$ is in CNF
3. $P_1 \wedge \neg P_2$ is CNF *and* DNF
4. $\{\{P_1, \neg P_1\}\}$ is unsatisfiable
5. $\{\{\}\}$ and $\{\}$ mean the same
6. $\{\{P_1, P_2\}, \{\neg P_1\}\} | P_1 = \{\{\}\}$
7. Δ in 6 is solvable via unit prop.

Quick check: true or false?

1. $\neg P_1 \vee (P_2 \wedge \neg P_3)$ is in NNF — **true** — $\neg$ only on atoms
2. $\neg(P_1 \wedge P_2)$ is in CNF — **false** — but $\neg P_1 \vee \neg P_2$ is
3. $P_1 \wedge \neg P_2$ is CNF *and* DNF
4. $\{\{P_1, \neg P_1\}\}$ is unsatisfiable
5. $\{\{\}\}$ and $\{\}$ mean the same
6. $\{\{P_1, P_2\}, \{\neg P_1\}\} | P_1 = \{\{\}\}$
7. Δ in 6 is solvable via unit prop.

Quick check: true or false?

1. $\neg P_1 \vee (P_2 \wedge \neg P_3)$ is in NNF — **true** — $\neg$ only on atoms
2. $\neg(P_1 \wedge P_2)$ is in CNF — **false** — but $\neg P_1 \vee \neg P_2$ is
3. $P_1 \wedge \neg P_2$ is CNF *and* DNF — **true** — two clauses / one term
4. $\{\{P_1, \neg P_1\}\}$ is unsatisfiable
5. $\{\{\}\}$ and $\{\}$ mean the same
6. $\{\{P_1, P_2\}, \{\neg P_1\}\} | P_1 = \{\{\}\}$
7. Δ in 6 is solvable via unit prop.

Quick check: true or false?

1. $\neg P_1 \vee (P_2 \wedge \neg P_3)$ is in NNF — **true** — $\neg$ only on atoms
2. $\neg(P_1 \wedge P_2)$ is in CNF — **false** — but $\neg P_1 \vee \neg P_2$ is
3. $P_1 \wedge \neg P_2$ is CNF *and* DNF — **true** — two clauses / one term
4. $\{\{P_1, \neg P_1\}\}$ is unsatisfiable — **false** — a single clause: valid
5. $\{\{\}\}$ and $\{\}$ mean the same
6. $\{\{P_1, P_2\}, \{\neg P_1\}\} | P_1 = \{\{\}\}$
7. Δ in 6 is solvable via unit prop.

Quick check: true or false?

1.	$\neg P_1 \vee (P_2 \wedge \neg P_3)$ is in NNF	**true**	$\neg$ only on atoms
2.	$\neg(P_1 \wedge P_2)$ is in CNF	**false**	but $\neg P_1 \vee \neg P_2$ is
3.	$P_1 \wedge \neg P_2$ is CNF *and* DNF	**true**	two clauses / one term
4.	$\{\{P_1, \neg P_1\}\}$ is unsatisfiable	**false**	a single clause: valid
5.	$\{\{\}\}$ and $\{\}$ mean the same	**false**	unsat vs. sat
6.	$\{\{P_1, P_2\}, \{\neg P_1\}\} \mid P_1 = \{\{\}\}$		
7.	Δ in 6 is solvable via unit prop.		

Quick check: true or false?

1.	$\neg P_1 \vee (P_2 \wedge \neg P_3)$ is in NNF	**true**	$\neg$ only on atoms
2.	$\neg(P_1 \wedge P_2)$ is in CNF	**false**	but $\neg P_1 \vee \neg P_2$ is
3.	$P_1 \wedge \neg P_2$ is CNF *and* DNF	**true**	two clauses / one term
4.	$\{\{P_1, \neg P_1\}\}$ is unsatisfiable	**false**	a single clause: valid
5.	$\{\{\}\}$ and $\{\}$ mean the same	**false**	unsat vs. sat
6.	$\{\{P_1, P_2\}, \{\neg P_1\}\} \mid P_1 = \{\{\}\}$	**true**	clause 1 drops, $\neg P_1$ goes
7.	Δ in 6 is solvable via unit prop.		

Quick check: true or false?

1.	$\neg P_1 \vee (P_2 \wedge \neg P_3)$ is in NNF	**true**	$\neg$ only on atoms
2.	$\neg(P_1 \wedge P_2)$ is in CNF	**false**	but $\neg P_1 \vee \neg P_2$ is
3.	$P_1 \wedge \neg P_2$ is CNF *and* DNF	**true**	two clauses / one term
4.	$\{\{P_1, \neg P_1\}\}$ is unsatisfiable	**false**	a single clause: valid
5.	$\{\{\}\}$ and $\{\}$ mean the same	**false**	unsat vs. sat
6.	$\{\{P_1, P_2\}, \{\neg P_1\}\}\|P_1 = \{\{\}\}$	**true**	clause 1 drops, $\neg P_1$ goes
7.	Δ in 6 is solvable via unit prop.	**true**	units $\neg P_1$, then P_2

More questions in the course notes

Propositional logic in four questions: done

$F := \ldots$	$\overline{v}(F)$	$\Gamma \models \alpha$	SAT
How do we **write** formulas?	When is a formula **true**?	What **follows** from what?	How do we **compute** it?

DPLL is the classic algorithm and the foundation of modern SAT solvers, which add more advanced techniques such as clause learning on top.

Propositional logic in four questions: done

$F ::= \ldots$ $\quad$ $\nu(F)$ $\quad$ $\Gamma \models \alpha$ $\quad$ SAT

How do we **write** formulas?

When is a formula true?

What **follows** from what?

How do we **compute** it?

DPLL is the classic algorithm and the foundation of modern SAT solvers, which add more advanced techniques such as clause learning on top.

Why did I tell you about this?

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

Three questions about F

SAT is there a model?

#SAT how many models?

WMC How much weight?

P_1	P_2	P_3	F
0	0	0	0
0	0	1	**1**
0	1	0	0
0	1	1	**1**
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	**1**

over the models

So far we only asked *whether* F has a model. What else could we ask?

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

Three questions about F

SAT is there a model? ✓

look for a single 1

#SAT how many models?

WMC How much weight?

P_1	P_2	P_3	F	
0	0	0	0	
0	0	1	**1**	
0	1	0	0	
0	1	1	**1**	
1	0	0	0	
1	0	1	0	
1	1	0	0	
1	1	1	**1**	

over the models

SAT: one model is enough. This is all DPLL tells us.

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

Three questions about F

SAT is there a model? ✓

look for a single 1

#SAT how many models? **3**

count the 1s

WMC How much weight?

P_1	P_2	P_3	F	
0	0	0	0	
0	0	1	**1**	
0	1	0	0	
0	1	1	**1**	
1	0	0	0	
1	0	1	0	
1	1	0	0	
1	1	1	**1**	
over the models			3	

#SAT, or **model counting**: *how many* worlds make F true? 3 of the 8.

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

weights: $w(P_1) = 0.8,\ w(P_2) = 0.75,\ w(P_3) = 0.4$

Three questions about F

SAT is there a model? ✓

look for a single 1

#SAT how many models? **3**

count the 1s

WMC How much weight? **0.32**

add up their weights

P_1	P_2	P_3	F	weight
0	0	0	0	0.03
0	0	1	**1**	0.02
0	1	0	0	0.09
0	1	1	**1**	0.06
1	0	0	0	0.12
1	0	1	0	0.08
1	1	0	0	0.36
1	1	1	**1**	0.24
over the models			3	**0.32**

WMC, or **weighted model counting**: worlds can matter more or less. A world weighs the product of its literals, e.g., $0.2 \cdot 0.25 \cdot 0.4 = 0.02$.

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

weights: $w(P_1) = 0.8,\ w(P_2) = 0.75,\ w(P_3) = 0.4$

Three questions about F

SAT is there a model? ✓

look for a single 1

#SAT how many models? **3**

count the 1s

WMC How much weight? **0.32**

add up their weights

P_1	P_2	P_3	F	weight
0	0	0	0	0.03
0	0	1	**1**	0.02
0	1	0	0	0.09
0	1	1	**1**	0.06
1	0	0	0	0.12
1	0	1	0	0.08
1	1	0	0	0.36
1	1	1	**1**	0.24
over the models			3	**0.32**

With $w(\neg P) = 1 - w(P)$, the weights of all 8 worlds sum to 1: WMC is the **probability** that F is true.

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

Three questions, one recipe

SAT

#SAT

WMC

P_1	P_2	P_3	F
0	0	0	0
0	0	1	**1**
0	1	0	0
0	1	1	**1**
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	**1**
sum over models			

So far we only asked *whether* a model exists. Now we ask more, always about the **same rows**: the models.

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

Three questions, one recipe

SAT $\bigvee_{v \models F} \bigwedge_{v \models l} 1$ ✓

#SAT

WMC

P_1	P_2	P_3	F	
0	0	0	0	
0	0	1	**1**	
0	1	0	0	
0	1	1	**1**	
1	0	0	0	
1	0	1	0	
1	1	0	0	
1	1	1	**1**	
sum over models				

SAT: **or** over the models: is there at least one?

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

Three questions, one recipe

SAT $\bigvee_{v \models F} \bigwedge_{v \models I} 1$ ✓

#SAT $\sum_{v \models F} \prod_{v \models I} 1$ **3**

WMC

P_1	P_2	P_3	F	
0	0	0	0	
0	0	1	**1**	
0	1	0	0	
0	1	1	**1**	
1	0	0	0	
1	0	1	0	
1	1	0	0	
1	1	1	**1**	
sum over models			3	

#SAT, or **model counting**: **add** 1 per model. 3 of the 8 worlds.

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

weights: $w(P_1) = 0.8,\ w(P_2) = 0.75,\ w(P_3) = 0.4$

Three questions, one recipe

SAT	$\bigvee_{v \models F} \bigwedge_{v \models I} 1$	✓
#SAT	$\sum_{v \models F} \prod_{v \models I} 1$	**3**
WMC	$\sum_{v \models F} \prod_{v \models I} w(I)$	**0.32**

P_1	P_2	P_3	F	weight
0	0	0	0	0.03
0	0	1	**1**	0.02
0	1	0	0	0.09
0	1	1	**1**	0.06
1	0	0	0	0.12
1	0	1	0	0.08
1	1	0	0	0.36
1	1	1	**1**	0.24
sum over models			3	**0.32**

WMC: a world weighs the **product** of its literals, e.g., $0.2 \cdot 0.25 \cdot 0.4 = 0.02$; **add** over the models.

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

weights: $w(P_1) = 0.8,\ w(P_2) = 0.75,\ w(P_3) = 0.4$

Three questions, one recipe

SAT	$\bigvee_{v \models F} \bigwedge_{v \models l} 1$	✓
#SAT	$\sum_{v \models F} \prod_{v \models l} 1$	**3**
WMC	$\sum_{v \models F} \prod_{v \models l} w(l)$	**0.32**

P_1	P_2	P_3	F	weight
0	0	0	0	0.03
0	0	1	**1**	0.02
0	1	0	0	0.09
0	1	1	**1**	0.06
1	0	0	0	0.12
1	0	1	0	0.08
1	1	0	0	0.36
1	1	1	**1**	0.24
sum over models			3	**0.32**

With $w(\neg P) = 1 - w(P)$, all weights sum to 1, so WMC = the **probability** that F is true.

From deciding to counting

$$F = (\neg P_1 \vee P_2) \wedge P_3$$

weights: $w(P_1) = 0.8,\ w(P_2) = 0.75,\ w(P_3) = 0.4$

Three questions, one recipe

SAT	$\bigvee_{v \models F} \bigwedge_{v \models l} 1$	✓
#SAT	$\sum_{v \models F} \prod_{v \models l} 1$	**3**
WMC	$\sum_{v \models F} \prod_{v \models l} w(l)$	**0.32**
general	$\bigoplus_{v \models F} \bigotimes_{v \models l} a(l)$	

P_1	P_2	P_3	F	weight
0	0	0	0	0.03
0	0	1	**1**	0.02
0	1	0	0	0.09
0	1	1	**1**	0.06
1	0	0	0	0.12
1	0	1	0	0.08
1	1	0	0	0.36
1	1	1	**1**	0.24
sum over models			3	**0.32**

Same rows, same recipe: combine the literals of a model with $\otimes$, the models with $\oplus$. Only the operations change.

Why counting? Logic meets probabilities

A classifier, and some background knowledge

predictions: P_j = "input j gets label **yes**", $\Pr(P_j) = 0.8,\ 0.75,\ 0.4$

knowledge: $F = (P_1 \rightarrow P_2) \wedge P_3$ (our formula from before)

Why counting? Logic meets probabilities

A classifier, and some background knowledge

predictions: P_j = "input j gets label **yes**", $\Pr(P_j) = 0.8,\ 0.75,\ 0.4$

knowledge: $F = (P_1 \rightarrow P_2) \wedge P_3$ (our formula from before)

How consistent?

$\Pr(F) = 0.32$

WMC: the chance that the predictions satisfy F

Why counting? Logic meets probabilities

A classifier, and some background knowledge

predictions: P_j = "input j gets label **yes**", $\Pr(P_j) = 0.8,\ 0.75,\ 0.4$

knowledge: $F = (P_1 \rightarrow P_2) \wedge P_3$ (our formula from before)

How consistent?

$\Pr(F) = 0.32$

WMC: the chance that the predictions satisfy F

A loss to learn from

$-\log \Pr(F) \approx 1.14$

the *semantic loss*: differentiable in the weights

Why counting? Logic meets probabilities

A classifier, and some background knowledge

predictions: P_j = "input j gets label **yes**", $\Pr(P_j) = 0.8,\ 0.75,\ 0.4$

knowledge: $F = (P_1 \rightarrow P_2) \wedge P_3$ (our formula from before)

How consistent?

$\Pr(F) = 0.32$

WMC: the chance that the predictions satisfy F

A loss to learn from

$-\log \Pr(F) \approx 1.14$

the *semantic loss*: differentiable in the weights

Best explanation

$P_1 = P_2 = P_3 = 1$

the most probable model (MPE): weight 0.24

Why counting? Logic meets probabilities

A classifier, and some background knowledge

predictions: P_j = "input j gets label **yes**", $\Pr(P_j) = 0.8,\ 0.75,\ 0.4$

knowledge: $F = (P_1 \rightarrow P_2) \wedge P_3$ (our formula from before)

How consistent?

$\Pr(F) = 0.32$

WMC: the chance that the predictions satisfy F

A loss to learn from

$-\log \Pr(F) \approx 1.14$

the *semantic loss*: differentiable in the weights

Best explanation

$P_1 = P_2 = P_3 = 1$

the most probable model (MPE): weight 0.24

Counting turns a formula into a **number** that we can optimize. Later: training neural networks with logic, and Bayesian networks.

Counting with DPLL: same search, different algebra

The splitting rule, three times

$$\textsc{sat}(\Delta) \;=\; \textsc{sat}(\Delta|l) \vee \textsc{sat}(\Delta|\neg l)$$

Counting with DPLL: same search, different algebra

The splitting rule, three times

$$\begin{aligned} \textsc{sat}(\Delta) &= \textsc{sat}(\Delta|l) \vee \textsc{sat}(\Delta|\neg l) \\ \#(\Delta) &= \#(\Delta|l) + \#(\Delta|\neg l) \end{aligned}$$

Counting with DPLL: same search, different algebra

The splitting rule, three times

$$\begin{aligned}
\textsc{sat}(\Delta) &= \textsc{sat}(\Delta|l) \vee \textsc{sat}(\Delta|\neg l) \\
\#(\Delta) &= \#(\Delta|l) + \#(\Delta|\neg l) \\
\textsc{wmc}(\Delta) &= w(l) \cdot \textsc{wmc}(\Delta|l) + w(\neg l) \cdot \textsc{wmc}(\Delta|\neg l)
\end{aligned}$$

Counting with DPLL: same search, different algebra

The splitting rule, three times

$$\begin{aligned}
\text{SAT}(\Delta) &= \text{SAT}(\Delta|l) \vee \text{SAT}(\Delta|\neg l) \\
\#(\Delta) &= \#(\Delta|l) + \#(\Delta|\neg l) \\
\text{WMC}(\Delta) &= w(l) \cdot \text{WMC}(\Delta|l) + w(\neg l) \cdot \text{WMC}(\Delta|\neg l) \\
S(\Delta) &= a(l) \otimes S(\Delta|l) \oplus a(\neg l) \otimes S(\Delta|\neg l) \quad \text{in general}
\end{aligned}$$

Only $\oplus$, $\otimes$ and a change

	$\oplus$	$\otimes$	$a(l)$
SAT	$\vee$	$\wedge$	1
#SAT	$+$	$\times$	1
WMC	$+$	$\times$	$w(l)$
MPE	max	$\times$	$w(l)$

Counting with DPLL: same search, different algebra

The splitting rule, three times

$$\begin{aligned}
\text{SAT}(\Delta) &= \text{SAT}(\Delta|l) \vee \text{SAT}(\Delta|\neg l) \\
\#(\Delta) &= \#(\Delta|l) + \#(\Delta|\neg l) \\
\text{WMC}(\Delta) &= w(l) \cdot \text{WMC}(\Delta|l) + w(\neg l) \cdot \text{WMC}(\Delta|\neg l) \\
S(\Delta) &= a(l) \otimes S(\Delta|l) \oplus a(\neg l) \otimes S(\Delta|\neg l) \quad \text{in general}
\end{aligned}$$

Only $\oplus$, $\otimes$ and a change

	$\oplus$	$\otimes$	$a(l)$
SAT	$\vee$	$\wedge$	1
#SAT	$+$	$\times$	1
WMC	$+$	$\times$	$w(l)$
MPE	max	$\times$	$w(l)$

One subtlety

$$\{\{P_1, P_2\}\}|P_1 = \{\}$$

but that branch has **2** models

P_2 was never split: it still counts

Counting with DPLL: same search, different algebra

The splitting rule, three times

$$\begin{aligned}
\text{SAT}(\Delta) &= \text{SAT}(\Delta|l) \vee \text{SAT}(\Delta|\neg l) \\
\#(\Delta) &= \#(\Delta|l) + \#(\Delta|\neg l) \\
\text{WMC}(\Delta) &= w(l) \cdot \text{WMC}(\Delta|l) + w(\neg l) \cdot \text{WMC}(\Delta|\neg l) \\
S(\Delta) &= a(l) \otimes S(\Delta|l) \oplus a(\neg l) \otimes S(\Delta|\neg l) \quad \text{in general}
\end{aligned}$$

Only $\oplus$, $\otimes$ and a change

	$\oplus$	$\otimes$	$a(l)$
SAT	$\vee$	$\wedge$	1
#SAT	$+$	$\times$	1
WMC	$+$	$\times$	$w(l)$
MPE	max	$\times$	$w(l)$

One subtlety

$$\{\{P_1, P_2\}\}|P_1 = \{\}$$

but that branch has **2** models

P_2 was never split: it still counts

DPLL is one instance of a general search over **semirings** (more later).

Reasoning over Boolean formulas (symprog)

```
## pip install symprog          (github.com/TuLLI-AI/symprog)
from symprog import Solver

solver = Solver()
solver.add(["Or(Not(p1), p2)", "p3"])            ## F
solver.add_weights({"p1": 0.8, "p2": 0.75, "p3": 0.4})

solver.query(mode="SAT").inference         ## => yes
solver.query(mode="Count").inference       ## => 3.0
solver.query(mode="Prob").inference        ## => 0.32  WMC
-solver.query(mode="LogProb").inference  ## => 1.14  loss
solver.query(mode="ViterbiAssignment").variables   ## MPE
## => {'p1': True, 'p2': True, 'p3': True}
```

One formula, many questions: `mode` picks the reasoning type.

Conclusion: what you should know

Syntax

- ✓ formulas are trees
- ✓ NNF, CNF, DNF
- ✓ CNF as a set of clauses

Conclusion: what you should know

Syntax

- ✓ formulas are trees
- ✓ NNF, CNF, DNF
- ✓ CNF as a set of clauses

Semantics

- ✓ worlds: each atom **1** or **0**
- ✓ truth tables, bottom-up
- ✓ equivalence and the laws

Conclusion: what you should know

Syntax

- ✓ formulas are trees
- ✓ NNF, CNF, DNF
- ✓ CNF as a set of clauses

Semantics

- ✓ worlds: each atom **1** or **0**
- ✓ truth tables, bottom-up
- ✓ equivalence and the laws

Inference

- ✓ satisfiable, valid, $\Gamma \models \alpha$
- ✓ $\Gamma \models \alpha$ *iff* $\Gamma \cup \{\neg\alpha\}$ is unsat
- ✓ truth tables need 2^n worlds

Conclusion: what you should know

Syntax

- ✓ formulas are trees
- ✓ NNF, CNF, DNF
- ✓ CNF as a set of clauses

Inference

- ✓ satisfiable, valid, $\Gamma \models \alpha$
- ✓ $\Gamma \models \alpha$ *iff* $\Gamma \cup \{\neg\alpha\}$ is unsat
- ✓ truth tables need 2^n worlds

Semantics

- ✓ worlds: each atom **1** or **0**
- ✓ truth tables, bottom-up
- ✓ equivalence and the laws

Computation

- ✓ conditioning $\Delta|_l$
- ✓ DPLL: splitting, unit rule
- ✓ #SAT, WMC: $\bigoplus$ of $\bigotimes$

Conclusion: what you should know

Syntax

- ✓ formulas are trees
- ✓ NNF, CNF, DNF
- ✓ CNF as a set of clauses

Inference

- ✓ satisfiable, valid, $\Gamma \models \alpha$
- ✓ $\Gamma \models \alpha$ *iff* $\Gamma \cup \{\neg\alpha\}$ is unsat
- ✓ truth tables need 2^n worlds

Semantics

- ✓ worlds: each atom **1** or **0**
- ✓ truth tables, bottom-up
- ✓ equivalence and the laws

Computation

- ✓ conditioning $\Delta|_l$
- ✓ DPLL: splitting, unit rule
- ✓ #SAT, WMC: $\bigoplus$ of $\bigotimes$

Next time: language modeling background, symbolic transformer computation.

References I

Beurer-Kellner, L., Fischer, M., and Vechev, M. (2023). Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969.

Bogin, B., Yang, K., Gupta, S., Richardson, K., Bransom, E., Clark, P., Sabharwal, A., and Khot, T. (2024). Super: Evaluating agents on setting up and executing tasks from research repositories. *Proceedings of EMNLP*.

Bragg, J., D'Arcy, M., Balepur, N., Bareket, D., Dalvi, B., Feldman, S., Haddad, D., Hwang, J. D., Jansen, P., Kishore, V., Majumder, B. P., Naik, A., Rahamimov, S., Richardson, K., Singh, A., Surana, H., Tiktinsky, A., Vasu, R., Wiener, G., Anastasiades, C., Candra, S., Dunkelberger, J., Emery, D., Evans, R., Hamada, M., Huff, R., Kinney, R., Latzke, M., Lochner, J., Lozano-Aguilera, R., Nguyen, C., Rao, S., Tanaka, A., Vlahos, B., Clark, P., Downey, D., Goldberg, Y., Sabharwal, A., and Weld, D. S. (2026). Astabench: Rigorous benchmarking of ai agents with a scientific research suite. *Proceedings of ICLR*.

Chen, J., Yuan, S., Ye, R., Majumder, B. P., and Richardson, K. (2023). Put your money where your mouth is: Evaluating strategic planning and execution of llm agents in an auction arena. *arXiv preprint arXiv:2310.05746*.

Cheng, J., Clark, P., and Richardson, K. (2025). Language modeling by language models. *Proceedings of Neurips*.

Cheng, J., Richardson, K., and Chin, P. (2026). Analytica: Soft propositional reasoning for robust and scalable llm-driven analysis. *Proceedings of ICLR*.

References II

De Raedt, L. and Kimmig, A. (2015). Probabilistic (logic) programming concepts. *Machine Learning*, 100:5–47.

De Raedt, L., Kimmig, A., and Toivonen, H. (2007). Problog: A probabilistic prolog and its application in link discovery. In *IJCAI 2007, Proceedings of the 20th international joint conference on artificial intelligence*, pages 2462–2467. IJCAI-INT JOINT CONF ARTIF INTELL.

Dohan, D., Xu, W., Lewkowycz, A., Austin, J., Bieber, D., Lopes, R. G., Wu, Y., Michalewski, H., Saurous, R. A., Sohl-Dickstein, J., et al. (2022). Language model cascades. *arXiv preprint arXiv:2207.10342*.

Dries, A., Kimmig, A., Meert, W., Renkens, J., Van den Broeck, G., Vlasselaer, J., and De Raedt, L. (2015). Problog2: Probabilistic logic programming. In *Joint european conference on machine learning and knowledge discovery in databases*, pages 312–315. Springer.

Fierens, D., Van den Broeck, G., Renkens, J., Shterionov, D., Gutmann, B., Thon, I., Janssens, G., and De Raedt, L. (2015). Inference and learning in probabilistic logic programs using weighted boolean formulas. *Theory and Practice of Logic Programming*, 15(3):358–401.

Garg, P., Geh, R. L., Israel, D., Millstein, T., Richardson, K., and Broeck, G. V. d. (2026). Probabilistic programs of thought. *arXiv preprint arXiv:2604.17290*.

References III

Huang, X., Bakalova, A., Bhattamishra, S., Merrill, W., and Hahn, M. (2026). Discovering interpretable algorithms by decompiling transformers to rasp. *arXiv preprint arXiv:2602.08857*.

Mandel, L., Baudart, G., Vaziri, M., and Hirzel, M. (2026). Ppdl: Llm-based flows as probabilistic programs. In *Forty-third International Conference on Machine Learning*.

Manhaeve, R., Dumancic, S., Kimmig, A., Demeester, T., and De Raedt, L. (2018). Deepproblog: Neural probabilistic logic programming. *Advances in neural information processing systems*, 31.

Richardson, K., Anderson, C., Balakrishnan, P., Ban, T., Ladia, D., Gupta, A., and Hudspeth, M. (2026a). From Token Probabilities to Semantic Constraints: Towards Declarative Probabilistic Evaluation of Language Models. *to appear at Proceedings of EMNLP*.

Richardson, K., Feng, Y., Garg, P., Cheng, J., and Roth, D. (2026b). CoTs as Tractable Probabilistic Programs. *Proceedings of TPM*.

Richardson, K., Srikumar, V., and Sabharwal, A. (2025). Understanding the logic of direct preference alignment through logic. *Proceedings of ICML*.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. (2022). Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

References IV

Weiss, G., Goldberg, Y., and Yahav, E. (2021). Thinking like transformers. In *International Conference on Machine Learning*, pages 11080–11090. PMLR.

Wu, C., Yang, S., and Murali, A. (2026). Imprompt: A language framework for prompt programming. *arXiv preprint arXiv:2607.22683*.

Yang, A. and Chiang, D. (2024). Counting like transformers: Compiling temporal counting logic into softmax transformers. *arXiv preprint arXiv:2404.04393*.

Yang, A., Chiang, D., and Angluin, D. (2024). Masked hard-attention transformers recognize exactly the star-free languages. *Advances in Neural Information Processing Systems*, 37:10202–10235.

Yang, R., Chen, J., Zhang, Y., Yuan, S., Chen, A., Richardson, K., Xiao, Y., and Yang, D. (2025). Selfgoal: Your language agents already know how to achieve high-level goals. *Proceedings of NAACL*.

Yu, H., Xuan, K., Li, F., Zhu, K., Lei, Z., Zhang, J., Qi, Z., Richardson, K., and You, J. (2025). Tinyscientist: An interactive, extensible, and controllable framework for building research agents. *Proceedings of EMNLP*.

Yu, H., You, J., Clark, P., Majumder, B. P., and Richardson, K. (2026). Artifactlinker: Linking scientific artifacts for automatic state-of-the-art discovery. *Proceedings of COLM*.

Zhang, Y., Yuan, S., Hu, C., Richardson, K., Xiao, Y., and Chen, J. (2024). Timearena: Shaping efficient multitasking language agents in a time-aware simulation. *Proceedings of ACL*.